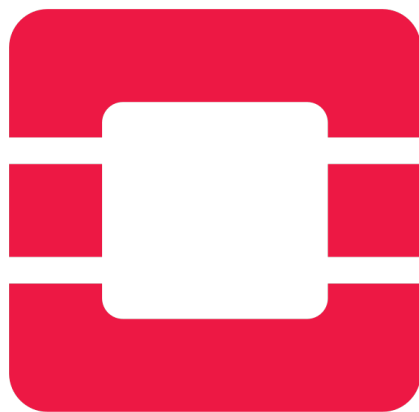




openstack®

목차

오픈스택 101

목차

1. 왜 다시 오픈소스인가

- 오픈소스의 철학과 라이선스
- 클라우드 벤더 종속의 현실
- AI 시대에 오픈소스가 다시 중요한 이유
 - 데이터 주권
 - 모델 주권
 - 인프라 주권



목차

2. 클라우드 서비스 모델 다시 보기

- IaaS / PaaS / SaaS 정리
- Public vs Private vs Hybrid
- AI 워크로드 관점에서의 IaaS
 - GPU/CPU
 - 격리
 - 네트워크

3. OpenStack 101 아키텍처

- Nova/Neutron/Glance/Cinder/Keystone
- OpenStack의 API 중심 설계
- "OpenStack은 API 플랫폼이다"



목차

4. 4장. OpenStack 운영과 자동화

- CLI/API/Ansible
- Heat/Terraform(OpenTofu)
- Infrastructure as Code 개념



목차

5. 5장. LLM이 등장한다

A. 5-1. LLM / SLM 기본 개념 (운영자 관점)

- LLM vs SLM
- 왜 SLM이 온프레미스에 적합한가
- CPU 기반 모델의 현실적 가치

A. 5-2. LLM이 이해해야 할 인프라 언어

- OpenStack API
- Ansible Playbook
- CLI 결과(JSON)
- LLM+OpenStack 연결 구조



목차

6. 6장. OpenStack 위에서 AI 인프라 만들기

- AI 실습 환경으로서의 OpenStack
- Kubernetes/Kubeflow/Jupyter
- GPU 패스스루 or CPU 기반 추론

7. 7장. LLM 기반 운영 자동화 시나리오

- "VM 하나 만들어줘"
- "이 프로젝트 리소스 사용량 분석해줘"
- "30일 지난 랩 자동 정리해줘"
- AI Ops Lite 개념



목차

8. 8장. 실습 또는 데모 구성(선택)

- OpenStack+Ansible
- OpenStack+LLM API (개념 데모)
- 내부 LLM+로그분석

9. 9장. 정리

- OpenStack은 끝난 기술이 아니다
- AI 시대의 인프라 백본
- 사람이 운영하던 인프라 → AI가 이해하는 인프라



소개

과정

강사

소개

01

오픈스택 처음 사용자

02

오픈스택 설치 및 기본 서비스
운영 및 명령어 학습



소개

이 과정은 오픈스택 기본 서비스 구성 및 구축 과정입니다. 다루는 내용은 다음과 같습니다.

1. 오픈스택 설치(kolla-ansible)
2. 오픈스택 주요 컴포넌트
3. AWS와 오픈스택 비교(교안에 미포함)
4. 오픈스택 명령어 사용 방법
5. 자원 자동화
6. 오픈스택과 AIOPs 활용

확장 서비스 과정 및 프로비저닝 자동화(openstack-helm, OpenStack Kayobe)과정은 별도로 제공할 예정입니다.



랩 정보

대시보드: <https://vlab.dustbox.kr>

웹 콘솔: <https://vconsole.vlab.dustbox.kr>

접근 시 사용 가능한 계정은 다음과 같음.

- 사용자 계정: `user1~20`
- 비밀번호: `vlabhorizon`
- 도메인: `vlab-users`
- 콘솔정보: `bastion`



강사

강사

2026-01-31

강사

이름: 최국현

메일: tang@linux.com, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

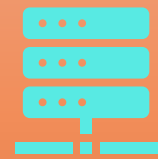
언제든지 질문 및 요청 환영 입니다.



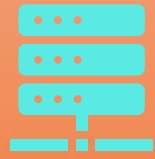
LAB

랩 및 교육대상

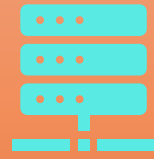
랩 다이어그램



control



compute



compute



storage

오픈스택 랩 정보

- 웹 주소: <https://vlab.dustbox.kr>
- 사용자 계정: user1~20
- 비밀번호: vlabhorizon
- 도메인: lab user

위 정보는 강의 진행 시 접근이 가능하며, 이후에는 접근이 불가능 합니다.



오픈스택

전체적인 설명

오픈소스란 무엇인가?

오픈소스(Open Source)란?

소스 코드가 공개되어 있으며 누구나 열람, 수정, 재배포가 가능한 소프트웨어 개발 방식 단순히 "무료"가 아닌 개발 철학과 생태계.

핵심 키워드

1. 투명성 (Transparency)
2. 검증 가능성 (Auditability)
3. 공동 개발 (Collaboration)
4. 선택권 (Freedom of Choice)



왜 오픈소스가 중요한가 (지금 시점에서)

과거의 오픈소스

- 비용 절감
- 대안 소프트웨어
- 커뮤니티 중심 개발

현재의 오픈소스

- 클라우드 인프라의 표준
- 대규모 시스템의 기반
- 벤더 종속을 피하기 위한 핵심 전략



왜 오픈소스가 중요한가 (지금 시점에서)

현실적인 이유

- 특정 벤더의 정책 변경 리스크
- 라이선스 비용 증가
- 폐쇄형 플랫폼의 한계



오픈소스와 클라우드 인프라

클라우드의 내부를 들여다보면 대부분 오픈소스 기반 소프트웨어로 구성이 되어 있다.

- Linux
- KVM
- OpenStack
- Kubernetes
- Ceph

현재 OpenStack의 위치는 다음과 같다.

- 퍼블릭 클라우드의 내부 구조를
- 기업이 직접 통제 가능한 형태로 구현한 플랫폼
- 프라이빗 / 하이브리드 클라우드의 핵심 구성요소



AI · LLM 시대의 오픈소스 의미

AI 시대에 오픈소스가 더 중요해진 이유

- 데이터 주권
- 모델 주권
- 인프라 주권

폐쇄형 AI vs 오픈소스 AI

- 외부 API 의존
- 내부 데이터 노출 위험
- 비용 예측 불가



AI · LLM 시대의 오픈소스 의미

오픈소스 인프라 + SLM

- 온프레미스 AI
- CPU 기반 모델
- 폐쇄망 환경에서도 활용 가능



왜 OpenStack은 한국에서 외면 받았다가, 다시 주목받는가?

과거: 한국에서 OpenStack이 어려웠던 이유 (2014~2020)

운영 난이도

- 설치·업그레이드 복잡
- 숙련된 운영 인력 부족

퍼블릭 클라우드 급성장

- AWS/GCP/Azure의 빠른 도입
- “직접 운영할 이유” 감소

국내 기업 문화

- 장애 리스크를 내부에서 감당하기 어려움
- 벤더 책임을 선호하는 구조

비용 오해

- “오픈소스 = 공짜”라는 잘못된 기대
- 실제 운영 비용 대비 효과를 증명하기 어려웠음



전환점: 다시 OpenStack을 검토하게 된 배경 (2021~현재)

클라우드 비용 폭증

- 장기 사용 시 OPEX 급증
- 예측 불가능한 과금 구조

벤더 종속(Vendor Lock-in) 현실화

- 특정 클라우드 기능 의존
- 마이그레이션 비용 증가

AI·데이터 주권 이슈

- 내부 데이터의 외부 반출 제한
- 온프레미스 AI 인프라 필요

기술 성숙

- Kolla-Ansible, 자동화 도구 안정화
- Kubernetes·AI 플랫폼과의 결합 용이

하이브리드 클라우드 전략 확산

- 퍼블릭 + 프라이빗 병행이 현실적인 선택



지금의 OpenStack은 무엇이 다른가?

단독 플랫폼 ❌

→ 하이브리드·AI 인프라의 기반

수동 운영 ❌

→ 자동화·IaC 중심 운영

VM 전용 ❌

→ Kubernetes·AI 워크로드 공존



오픈스택의 시작 및 역사

1. 오픈스택은 본래 미국 NASA에서 프로젝트용 웹 사이트 관리를 효율적으로 하기 위해서 Rackspace에 위탁 및 운영.
2. Rackspace는 여러 프로젝트 및 자원의 회수 및 재활용을 효율적으로 하기 위해서 리눅스 기반으로 관리도구를 만들기 시작.
3. 도구가 점점 발전이 되면서 최종적으로 "오픈스택"이라는 이름으로 커뮤니티에 기여 및 공개가 되었다.

자세한 내용은 아래 사이트에서 확인이 가능하다.

<https://docs.openstack.org/project-team-guide/introduction.html>



오픈스택 구성원

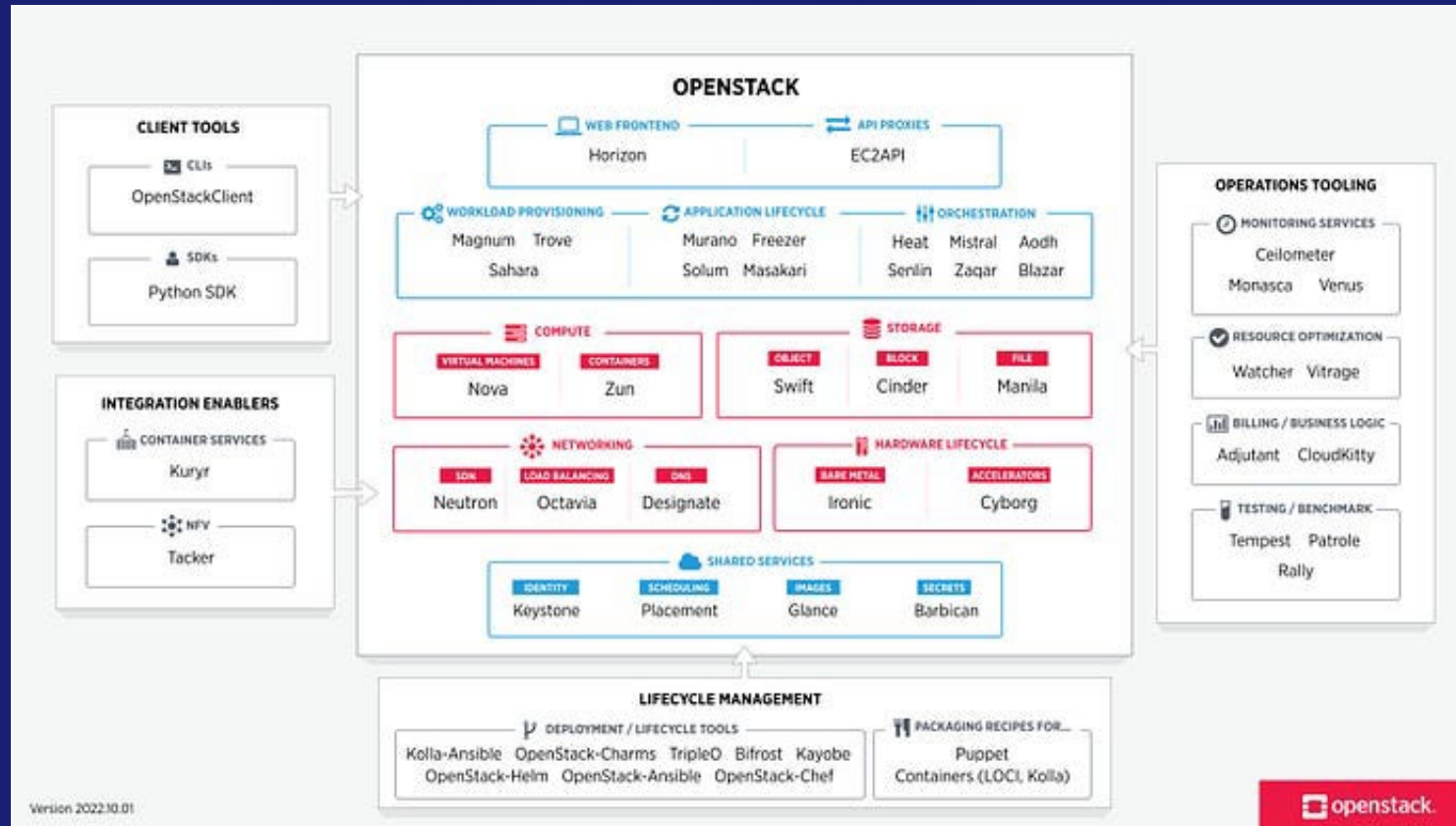
오픈스택은 파이썬 기반으로 작성이 되어 있으며, 이때 오픈스택은 Nova밖에 없었다. 이를 통해서 네트워크/볼륨/이미지/가상머신/보안과 같은 자원을 모두 다루다가 하나씩 서비스가 분리가 되었다.

현재 오픈스택 핵심 서비스 다음과 같다.

1. Nova
2. Keystone
3. Cinder
4. Glance
5. Heat
6. Horizon



주요 구성원



오픈스택 커뮤니티 버전

현재 오픈스택은 코드/년도 기반 릴리즈 이름을 따르고 있다. 릴리즈 주기는 보통 6개월에 한번씩 릴리즈가 되며, 2025년도 첫 번째 릴리즈는 2025.1, 두 번째 릴리즈는 2025.2 이와 같은 방식으로 네이밍을 결정한다.

코드 네임은 알파벳 A부터 도시/동물 이름으로 시작하나, 현재는 보조적으로 사용하고 년도 기반 릴리즈가 관리 번호로 명명 및 명시되고 있다.

레드햇 오픈스택 경우에는 별도로 버전 릴리즈 관리 및 운영하고 있다.



커뮤니티 버전 릴리즈

연도 기반 버전	코드네임	릴리즈 시기
2018.1	Queens	2018년 2월
2018.2	Rocky	2018년 8월
2019.1	Stein	2019년 4월
2019.2	Train	2019년 10월
2020.1	Ussuri	2020년 5월
2023.2	Bobcat	2023년 10월
2024.1	Caracal	2024년 4월
2024.2	Dalmatian	2024년 10월
2025.1	Flamingo	2025년 4월 (예정)
2025.2	G... (미정, G 시작 이름 예정)	2025년 10월 (예정)



레드햇 버전 비교

레드햇 경우에는 독립적으로 릴리즈 번호를 구성 및 사용하고 있다.

구분	RHEL (레드햇)	OpenStack
표기 방식	RHEL 7, RHEL 8, RHEL 9 ...	2025.1 "Dalmatian", 2025.2 "Flamingo"
릴리즈 주기	보통 3~5년마다 메이저, 10년까지 지원	6개월마다 새 시리즈 (상반기/하반기)
이름 부여	숫자 기반	알파벳 순서 기반 이름(Albany, Bobcat, Caracal ...) + 연도 기반 버전
지원 정책	메이저 버전 단위로 LTS 제공	stable branch 유지보수(보통 18개월~24개월), Extended Maintenance/Unmaintained 전환



레드햇 버전 비교

연도	RHEL 메이저 GA(있을 때)	OpenStack 상반기	OpenStack 하반기
2014	RHEL 7 — 2014-06-10	—	—
2018	—	Queens — 2018-02	Rocky — 2018-08
2019	RHEL 8 — 2019-05-07	Stein — 2019-04	Train — 2019-10
2020	—	Ussuri — 2020-05	Victoria — 2020-10
2021	—	Wallaby — 2021-04	Xena — 2021-10



커뮤니티/레드햇 버전 비교

연도	RHEL 메이저 GA(있을 때)	OpenStack 상반기	OpenStack 하반기
2022	RHEL 9 — 2022-05-17	Yoga — 2022-03	Zed — 2022-10
2023	—	Antelope — 2023-03-22 (OpenStack Releases)	Bobcat — 2023-10
2024	—	Caracal — 2024-04 (SLURP)	Dalmatian — 2024-10-02 (OpenStack Releases)
2025	RHEL 10 — 2025-05 (10.0)	Epoxy — 2025-04-02 (SLURP) (lists.openstack.org)	Flamingo — 2025-10-01(예정) (OpenStack Releases)



사과의 말씀

10년전에 오픈스택 강의 시, "버전 릴리즈 네이밍 방식이 더 이상 알파벳 방식으로 구성하지 않는다."이야기 하였습니다. 하지만, 그 예상을 뒤집고 다시 알파벳 기준으로 시작이 되었습니다.

하지만, 년도날짜 기준으로 릴리즈는 예상대로 나왔지만, 나머지는 메일링에서 토론되었던 내용과 다르게 진행이 되었습니다.



오픈스택 용도 및 목적

오픈스택은 퍼블릭 클라우드에서 대표적인 **AWS/GCP**와 같은 API기반의 플랫폼 구성 및 운영이 가능하다. 서비스 확장은 사용자 혹은 오픈스택 릴리즈에 맞추어서 가능하며, 이를 통해서 다양한 서비스를 사용자에게 제공이 가능하다.

오픈스택은 플랫폼 기반으로 서비스가 확장이 되는 구성을 가지고 있다. 이를 통해서 **IaaS** 및 **XaaS** 서비스 개념을 구현 및 구성하고 있다. 또한, 오픈스택은 **OpenInfra**의 핵심 구성원이다. 오픈 소스에서는 인프라 및 응용 분야에 대해서 보통 두 가지 영역으로 구별하고 있다.

1. Open-Infra
2. CNCF

OpenInfra는 말 그대로 열린 인프라 아키텍처를 지향하고 있으며, **CNCF**는 인프라 위에서 동작하는 컴퓨팅 서비스에 대해서 정의하고 있다.



설치

현재 오픈스택은 다양한 방법으로 설치를 제공 및 지원하고 있다.

1. kolla-ansible
2. OpenStack Kayobe
3. OpenStack Bifrost
4. Helm-OpenStack

일반적으로 많이 사용하는 구성은 **Kolla-Ansible** 기반으로 많이 사용하며, 하드웨어 및 운영체제 관리까지 포함이 되는 경우 **Kayobe**, **Bifrost**를 많이 사용한다.

좀 더 유연한 컨트롤러 및 쿠버네티스와 통합을 위해서 **helm-openstack** 사용이 가능하지만, 레드햇 오픈 시프트를 제외하고 아직 실 환경에서 사용하는 사례는 드물다. (2025년 기준)



설치 방법

현재 오픈스택은 다양한 방식으로 패키지 및 설치 방식을 제공하고 있다. 기본적으로 패키지는 아래와 같은 형식으로 제공한다.

1. deb

2. rpm

시스템 패키지가 아닌, 컨테이너 기반으로 구성을 원하는 경우, **OpenStack Kolla Ansible**를 통해서 설치가 가능하다. 이는 컨테이너 이미지를 통해서 런타임을 제공하며, 다음과 같은 형식의 배포판을 지원하고 있다.

1. Debian 계열

2. Redhat 계열

아쉽게도 수세/슬랙웨어 리눅스 계열에 대한 패키지 및 런타임 이미지는 아직 제공하지 않는다.



설치 방법

베어메탈 기반으로 구성 시, 자동화 도구가 필요한데 오픈스택은 다음과 형식으로 제공한다.

1. **Ironi**c
2. **Bifstor** (ansible+Ironi)c
3. **Kayobe**

레드햇 오픈스택 경우에는 현재 Director 및 OpenShift 기반으로 설치 지원.

대용량 및 프로덕트 용도로 사용을 원하는 경우 **Kayobe**기반으로 물리적 인프라 구성 및 컨테이너화 된 오픈스택 서비스 배포 사용을 권장한다.



설치(Bifrost vs Kayobe)

구분	Bifrost	Kayobe
정의	Ironic(베어메탈 서비스)를 스탠드얼론으로 설치·활용하는 Ansible 플레이북 모음	Kolla-Ansible을 기반으로 컨테이너화된 OpenStack을 베어메탈에 풀스택 배포·운영하는 오케스트레이션
주 목적	베어메탈 OS 프로비저닝(디스커버리, 인스펙션, 이미지 디플로이) 중심의 경량 자동화	컨트롤 플레인/워크로드 노드 OS 프로비저닝 + 컨테이너화 OpenStack 배포 + Day2 운영 작업까지 포괄
핵심 구성	Ansible Roles/Playbooks + Ironic standalone	Seed/Seed-hypervisor, bifrost(내장 컨테이너), Kolla/Kolla-Ansible, 호스트 구성(네트워크·OS)
동작 방식	Ironic을 단독으로 올려 알려진 하드웨어에 베이스 이미지 배포	Seed 호스트에서 bifrost 컨테이너로 노드 등록·프로비저닝 → Kolla-Ansible로 OpenStack 서비스 컨테이너 배포
의존/연계	Ironic, Ansible. OpenStack의 다른 컴포넌트 없이 독립 운용 가능	내부적으로 bifrost를 호출해 오버클라우드 노드를 PXE로 발견/등록 후, Kolla-Ansible로 서비스 배포



설치(Bifrost vs Kayobe)

구분	Bifrost	Kayobe
적용 규모	랩/POC/경량 베어메탈 설치, 대량 OS 설치 자동화	엔터프라이즈급 OpenStack 풀스택 배포·업그레이드·스케일링
강점	단순·경량, Ironic 기능을 빨리 체험/활용, 독립 실행	설계→프로비저닝→컨테이너 배포→Day2까지 엔드투엔드 자동화, 대규모 운영에 적합
한계	OpenStack 전체 스택을 다루지 않음 (컨트롤 플레인/서비스 배포는 범위 밖)	학습 곡선 존재, 초기 설계/네트워킹 준비 요구사항이 큼
대표 워크플로	(1) Ironic standalone 설치 → (2) 하드웨어 등록·인스펙션 → (3) 이미지 배포	(1) Seed/네트워크 준비 → (2) bifrost로 오버클라우드 디스커버리/프로비저닝 → (3) Kolla-Ansible로 OpenStack 컨테이너 배포 → (4) Day2 운영
산출물	베어메탈 서버에 OS가 배포된 상태(이미지 설치 완료)	컨트롤/컴퓨트 노드에 컨테이너화된 OpenStack 완전체 (업그레이드/스케일 지원)
누구에게 적합?	"OS만 깔아서 바로 쓰고 싶다", "Ironic을 가볍게 테스트/활용"	"OpenStack 풀스택을 베어메탈에 일관되게 배포/운영하고 싶다"



OpenStack-Helm

항목	helm-openstack (OpenStack-Helm)	Kayobe
한줄 정의	Kubernetes 위에 OpenStack을 Helm 차트로 배포·운영하는 커뮤니티 프로젝트	베어메탈에 Kolla/Kolla-Ansible 컨테이너형 OpenStack을 배포·운영하는 자동화 프레임 워크(Ansible 중심)
주요 목적/범위	OpenStack 서비스와 필수 컴포넌트(예: MariaDB, RabbitMQ, Ceph 등)를 K8s 네이티브하게 설치·유지	Seed → Bifrost → Kolla-Ansible의 체인으로 호스트 프로비저닝 + 컨테이너형 OpenStack 배포 + Day2 운영 까지 포괄
기반/제어면	Kubernetes/Helm (차트, values, 릴리스로 상태 관리)	Ansible + Bifrost + Kolla-Ansible (Seed 호스트에서 PXE/등록/이미징, 이후 컨테이너형 OpenStack 배포)
설치 흐름(요지)	K8s 클러스터 준비 → OSH 차트로 Keystone/Nova/Neutron/Cinder 등 배포 → Helm 릴리스로 구성·업그레이드	Ansible Control/Seed 준비 → Seed에서 Bifrost로 오버클라우드 노드 프로비저닝 → Kolla-Ansible로 OpenStack 컨트롤/컴퓨트 배포



OpenStack-Helm

항목	helm-openstack (OpenStack-Helm)	Kayobe
Day-2(운영) 접근	Helm 업그레이드 모델(차트/ConfigMap/이미지 롤링 업데이트로 일관된 방식)	Kayobe + Kolla-Ansible 플레이북으로 업그레이드/스케일/재구성. 호스트 OS/네트워크 구성까지 관리 범위에 포함
적용 지형	K8s 인프라가 이미 표준인 조직, OpenStack을 K8s 워크로드로 통합하려는 팀	새 하드웨어 또는 기존 베어메탈에 풀스택 OpenStack을 표준화해 올리고 싶은 팀(대규모·장기 운영)
강점	K8s 에코체인(Helm)과 친화적, 차트 기반 선언형 관리, 일부 컴포넌트만 선택적 배포도 용이	엔드-투-엔드 자동화(호스트 준비→프로비저닝→컨테이너 배포→Day2), 대규모 운영 경험 축적, 업그레이드 경로 명확
한계/유의	K8s 레이어와 OpenStack 복잡도가 겹쳐 네트워킹/스토리지 설계 난이도가 높아질 수 있음	초기 네트워크/시딩 설계(Seed, PXE, L2/L3 플랜)와 Ansible 구성 학습 비용이 필요



최종 비교

항목	helm-openstack (OpenStack-Helm)	Kayobe
산출물	K8s 상에서 구동되는 OpenStack 서비스(네임스페이스/릴리스로 관리)	베어메탈 노드 위에 컨테이너형 OpenStack 완전체 (컨트롤/컴퓨트/네트워킹), 운영용 플레이북 세트
어떤 환경에 적절한가?	"우리의 표준은 Kubernetes이고, 그 위에서 OpenStack을 서비스처럼 다루고 싶다"	"베어메탈부터 표준 절차로 OpenStack을 올리고, Day-2까지 Ansible로 일관 관리하고 싶다"



준비 및 설계 단계(Operation Model)

오픈스택은 준비 및 설계 단계는 보통 다음과 같이 진행한다. 이 교육에서는 DAY 0/1/2 기준으로 진행한다.

단계	주요 목적	작업 예시 (OpenStack)
Day 0(설계/플랜)	요구사항 분석 및 아키텍처 설계	- 컨트롤러/컴퓨트/스토리지 노드 수량 계획 - 네트워크 토폴로지, VLAN/IP 범위 설계 - 인증·보안 정책 수립- 스토리지 선택(Ceph/NFS 등)
Day 1(배포/구축)	실제 클러스터 배포 및 초기 구성	- Kolla-Ansible, TripleO, Helm으로 OpenStack 서비스 배포 - Keystone 기본 계정 생성 - 네트워크/프로젝트 초기 설정
Day 2(운영/관리)	운영·유지보수, 업그레이드, 최적화	- Nova Compute 노드 확장 - Keystone 사용자/역할 관리 - Cinder 백엔드 교체- 보안 패치 및 업그레이드



오픈스택 설치

배포판

RPM/DEB

kolla-ansible

배포판 지원

현재 오픈스택 kolla-ansible에서 지원하는 배포판은 공식적으로 다음과 같다.

1. Debian
2. Ubuntu
3. Rocky/CentOS-Stream

이와 같이 지원한다. 레드햇 계열은 실제로 Rocky, CentOS-Stream 계열이 아니어도 사용이 가능하다.

수세 리눅스는 SUSE OpenStack Cloud(SOC)라는 이름으로 지원하였다가, 2021년도에 공식적으로 더 이상 지원하지 않는다.



수세 리눅스 배포판

수세 오픈스택 지원 버전 및 릴리즈 연도

SUSE OpenStack Cloud 9

- 기반: OpenStack Pike (2017 릴리즈)
- 출시: 2017년 말

SUSE OpenStack Cloud 10

- 기반: OpenStack Queens (2018 릴리즈)
- 출시: 2018년 10월



수세 리눅스 배포판

SUSE OpenStack Cloud 11

- 기반: OpenStack Rocky (2018 릴리즈)
- 출시: 2019년 3월

SUSE OpenStack Cloud 12

- 기반: OpenStack Train (2019 릴리즈)
- 출시: 2020년 초

SOC 12 (OpenStack Train) 이 마지막 릴리즈. 이후 SUSE는 오픈스택을 더 이상 지원하지 않고, Rancher/Harvester/Kubernetes 기반으로 PaaS/IaaS 플랫폼 전략을 변경.

현재 수세는 더 이상 공식적으로 RPM 및 컨테이너 이미지를 제공하지 않는다.



RPM/DEB

현재 대다수 오픈스택은 Hosted에서 RPM, DEB 기반으로 설치를 하지 않는다. 이 문서에서는 더 이상 RPM/DEB 기반으로 설치하는 다루지 않는다.

다만, `kolla-build`를 사용시 RPM를 사용하기 때문에, RPM 및 DEB 패키지는 모든 배포판에서 별도의 저장소로 제공하고 있다. 저장소를 검색하면 다음과 같이 화면에 출력이 된다.

레드햇 호환 계열 사용자는 더 이상 레드햇과 동일한 품질의 RPM은 사용하기 어렵다. centos-stream-release RPM 기반으로 여전히 사용이 가능하다.(RDO는 CentOS-Stream으로 전환)

```
centos-release-openstack-antelope.noarch
centos-release-openstack-bobcat.noarch
centos-release-openstack-caracal.noarch
centos-release-openstack-dalmatian.noarch
```



설치 준비

간단하게 SNO(Single Node OpenStack)를 구성하기 위해서 아래와 같이 사양이 필요하다. 이 사양은 절대 실제 서비스 제품 환경(Service Product)에서 사용하는 사양이 아니다.

- CPU: 16 core이상
- MEM: 64기가 권장. 최소 32기가(각 노드)
- DISK: 100기가 이상의 SSD 혹은 NVME디스크
- NIC: 1GiB 이상 네트워크 인터페이스
- 런타임: Podman, Docker-CE 기반으로 설치 및 구성 가능

모든 랩의 기본 사양은 오픈스택 랩 기반으로 설계가 되었으며, 일반 워크스테이션 및 노트북 기반으로 사용하는 경우, 꼭 메인보드 및 CPU에서 **nested**기능이 활성화 되어 있어야 한다.



설치 사양

오픈스택 인 오픈스택(OIO)를 구성하기 위해서는 다음과 같은 조건으로 가상머신 생성 및 구성 한다. 네트워크는 구성에 따라서 다르지만, 서비스 네트워크는 VLAN 혹은 VXLAN/Geneve 기반으로 구성한다.

아래 내용은 **오픈스택 랩 기반**에서 구성 시 네트워크 및 볼륨 사양이다.

- 볼륨 디스크: 10GiB
- 네트워크:
 - infra-net: 192.168.10.0/24, vlan10
 - net-osp-internal: 192.168.20.0/24, vlan20
 - net-osp-storage: 192.168.30.0/24, vlan30(선택사항)
 - infra-net: 10.0.0.0/8
- 사양: t1.openstack-controller, t1.openstack-compute



오픈스택 랩 진행 시 주의사항

오픈스택에서 오픈스택을 구성 및 구현 시, 포트 보안문제로 올바르게 동작하지 않는다. 다음과 같이 포트 보안(Port Security) 설정을 다음과 같이 한다.

Edit Port

Info

Name

☒ Enable Admin State ⓘ

Binding: VNIC Type

Normal

☐ Port Security

You can edit the properties of your port here.

Enable Admin State
When the admin state of the port is enabled, the networking service forward packets on the port. Otherwise, it does not forward any packets on the port.

Binding: VNIC Type
It specifies the VNIC type bound to the networking port.

Port Security
Enables anti-spoofing rules for the port if enabled. In addition, if port security is disabled, security groups on the port will be automatically cleared. When you enable port security of the port, you may want to associate some security groups on the port.

Security Groups
You can add or remove security groups associated with the port in the next tab (if the port security is enabled for the port).

Cancel Update



간단한 구성 영상



설치

클러스터 프로비저닝

kolla-ansible 구성

오픈스택 설치

현재 오픈 스택은 RPM/DEB 기반으로 설치보다는 고수준 컨테이너 기반으로 설치를 권장한다. 고수준 컨테이너 런타임으로 설치시, 다음과 같은 조건이 따른다.

1. Podman

2. Docker-CE(Containerd)

오픈스택에서 제공하는 모든 기능을 사용하기 위해서는 **Docker(Containerd)** 기반으로 구성을 권장. Podman 기반으로 설치를 지원하고 있지만, 몇몇 기능은 **Podman**으로 완전히 지원하지 않는다.

앞으로 대다수 **Kolla** 기반의 컨테이너 이미지 및 애플리케이션은 **Podman**기반으로 마이그레이션 예정.



파이썬과 Kolla 관계

`kolla-ansible`를 사용하기 위해서, 가급적이면 Python 3.11버전 이상 사용을 권장한다. 레드햇 기준으로는 기본 Python 3.9으로는 `kolla-ansible` 구형 그리고, `kolla-ansible` 명령어를 올바르게 사용하기가 어렵다.

이러한 이유로 레드햇 계열은 가급적이면 Rocky 9.x버전 기반으로 설치한다. Kolla 매뉴얼에는 Debian 리눅스에서 `venv` 사용을 권장하지만, 레드햇 계열 배포판도 가급적이면 `venv` 기반으로 `kolla`, `kolla-ansible` 설치 및 사용을 권장한다.



python, pip

다음과 같은 과정으로 파이썬 패키지를 설치 및 준비한다.

```
dnf install -y git python3.11 python3.11-pip dbus-devel glib2-devel  
pkgconf-pkg-config python3.11-devel  
python3.11 -m venv ~/openstack  
source ~/openstack/bin/activate  
dnf groupinstall -y "Development tools"  
pip3.11 install dbus-python docker kolla-ansible==19.5.0  
pip3.11 install dbus-python podman kolla-ansible==19.5.0
```



kolla-ansible(통합모드)

/etc/kolla/위치에 inventory 파일 복사한다.

```
mkdir -p /etc/kolla
cp -a ~/openstack/share/kolla-ansible/etc_examples/kolla/* /etc/kolla
cp -a ~/openstack/share/kolla-ansible/ansible/inventory/all-in-one ~
ls -l /etc/kolla
ls -l ~/all-in-one
```



kolla-ansible(다중노드)

/etc/kolla/위치에 inventory 파일 복사한다.

```
mkdir -p /etc/kolla
cp -a ~/openstack/share/kolla-ansible/etc_examples/kolla/* /etc/kolla
cp -a ~/openstack/share/kolla-ansible/ansible/inventory/multinode ~
ls -l /etc/kolla
ls -l ~/multinode
```



docker-registry 구성(옵션)

내부 컨테이너 이미지 저장소가 필요한 경우, docker-registry 서버를 구성한다.

```
REGISTRY_HOST=$(ip -4 addr show ens3 | grep -oP '(?<=inet\s)\d+(\.\d+){3}')
REGISTRY_DIR=/opt/registry
CERT_DIR=/etc/docker/certs
mkdir -p ${REGISTRY_DIR} ${CERT_DIR}
openssl req -newkey rsa:2048 -nodes -keyout ${CERT_DIR}/domain.key \
    -x509 -days 365 -out ${CERT_DIR}/domain.crt \
    -subj "/CN=${REGISTRY_HOST}"
```



docker-registry 구성(옵션)

포드만 버전

```
dnf install -y podman
podman run -d --name registry \
  -p 5000:5000 \
  -v ${REGISTRY_DIR}:/var/lib/registry:z \
  -v ${CERT_DIR}:/certs:z \
  -e REGISTRY_HTTP_TLS_CERTIFICATE=/certs/domain.crt \
  -e REGISTRY_HTTP_TLS_KEY=/certs/domain.key \
  docker.io/library/registry:2
```



docker-registry 구성(옵션)

도커 버전

```
dnf config-manager --add-repo \  
https://download.docker.com/linux/rhel/docker-ce.repo  
dnf install -y docker-ce docker-ce-cli containerd.io  
systemctl enable --now docker.service  
docker run -d --name registry \  
  -p 5000:5000 \  
  -v ${REGISTRY_DIR}:/var/lib/registry:z \  
  -v ${CERT_DIR}:/certs:z \  
  -e REGISTRY_HTTP_TLS_CERTIFICATE=/certs/domain.crt \  
  -e REGISTRY_HTTP_TLS_KEY=/certs/domain.key \  
  docker.io/library/registry:2
```



docker-registry 구성(옵션)

다른 노드도 접근 시, TLS문제가 발생하지 않도록 키를 복사한다. 모든 노드에 복사를 한다.

```
scp /etc/docker/certs/domain.crt root@control1:/etc/pki/ca-  
trust/source/anchors/  
scp /etc/docker/certs/domain.crt root@node4:/etc/pki/ca-  
trust/source/anchors/  
update-ca-trust extract
```



docker-registry 구성(옵션)

컨테이너에 TLS키 오류가 발생하지 않도록 복사한다.

```
mkdir -p /etc/containers/certs.d/${REGISTRY_HOST}:5000  
cp /etc/docker/certs/domain.crt \  
/etc/containers/certs.d/${REGISTRY_HOST}:5000/ca.crt
```

```
mkdir -p /etc/docker/certs.d/${REGISTRY_HOST}:5000  
cp /etc/docker/certs/domain.crt \  
/etc/docker/certs.d/${REGISTRY_HOST}:5000/ca.crt
```



버전별로 설치 명령어(2024)

PIP 기준으로 2024년 릴리즈 설치 시, 다음과 같이 구별 및 설치 진행이 가능하다.

연도	버전	릴리즈 날짜	설치 명령어
2024	18.0.0	2024-04-24	<code>pip install kolla-ansible==18.0.0</code>
	18.1.0	2024-06-26	<code>pip install kolla-ansible==18.1.0</code>
	18.2.0	2024-08-14	<code>pip install kolla-ansible==18.2.0</code>
	18.3.0	2024-10-23	<code>pip install kolla-ansible==18.3.0</code>



버전별로 설치 명령어(2025)

PIP 기준으로 2025년 릴리즈 설치 시, 다음과 같이 구별 및 설치 진행이 가능하다.

연도	버전	PyPI 릴리즈 날짜	설치 명령어
2025	20.0.0	2025-05-14	<code>pip install kolla-ansible==20.0.0</code>
	20.1.0	2025-06-24	<code>pip install kolla-ansible==20.1.0</code>
	20.2.0	2025-09-11	<code>pip install kolla-ansible==20.2.0</code>
	20.3.0	2025-11-10	<code>pip install kolla-ansible==20.3.0</code>



PRODUCTION OPENSTACK

MULTI NODES

환경준비

시작 전, 아래와 같이 파이썬 환경을 준비한다. 기본적인 부분은 위에서 이미 수행하였다.

```
dnf install -y lvm2
vgcreate cinder-volumes /dev/vdb
vgdisplay
```

서버에 사용하는 인터페이스 카드는 총 두 개가 필요하다. 사용자 랩 환경에 따라서 이 부분은 달라진다.

1. 내부용(1개, 터널링)
2. 외부용(1개, 오픈스택 기반으로 2개)
3. 스토리지(1개)
4. 관리자 네트워크(1개)
5. 백업 네트워크(1개)



inventory(다중노드)

/root/inventory 설치 시, 사용할 서버를 아래와 같이 입력한다.

```
# vi /root/inventory
[control]
control1 ansible_host=192.168.10.11
control2 ansible_host=192.168.10.12
control3 ansible_host=192.168.10.13

[compute]
compute1 ansible_host=192.168.10.14
compute2 ansible_host=192.168.10.15
```



inventory(다중노드)

위의 내용 계속 이어서...

```
[storage]
```

```
storage ansible_host=192.168.10.16
```

```
[network]
```

```
network ansible_host=192.168.10.17
```

```
[monitoring]
```

```
control1
```

```
[deployment]
```

```
localhost ansible_connection=local
```



globals.yml(다중노드)

/etc/kolla/globals.yml에 다음처럼 작성한다.

```
# vi /etc/kolla/globals.yml
---
kolla_base_distro: "rocky"
kolla_install_type: "binary"
openstack_tag: "2025.01"

# VIP 설정
kolla_internal_vip_address: "192.168.10.250"
kolla_external_vip_address: "10.0.X.Y"
keepalived_virtual_router_id: 51
keepalived_password: "keepalivedpass"
```



globals.yml(다중노드)

위의 내용 계속 이어서...

네트워크 인터페이스

network_interface: "eth0"

neutron_external_interface: "eth1"

storage_interface: "eth2"

kolla_enable_tls_external: "yes"

kolla_enable_tls_internal: "yes"

kolla_internal_fqdn: "int.aio.local"

kolla_external_fqdn: "ext.aio.local"

internal

compute, storage 노드에서만 존재



globals.yml(다중노드)

위의 내용 계속 이어서...

Docker Registry 설정 (내부 미리)

```
docker_registry: "192.168.20.100:5000" # storage 노드 external IP
docker_namespace: "kolla"
docker_registry_insecure: "yes"
kolla_copy_ca_into_containers: "no"
```



globals.yml(다중노드)

/etc/kolla/globals.yml에 다음처럼 작성한다.

MTU 설정

```
network_mtu: 9000
```

```
neutron_global_physnet_mtu: 9000
```

서비스 활성화

```
enable_cinder: "yes"
```

```
enable_cinder_backend_nfs: "yes"
```

```
enable_heat: "yes"
```

```
enable_horizon: "yes"
```

```
enable_keepalived: "yes"
```



kolla-ansible 실행 준비

다음 명령어로 설치를 준비한다.

```
# kolla-ansible install-deps
# kolla-genpwd
# kolla-ansible pull
# kolla-ansible bootstrap-servers -i /etc/kolla/ansible/inventory/multimode
# kolla-ansible certificates -i /etc/kolla/ansible/inventory/multimode
# kolla-ansible prechecks -i /etc/kolla/ansible/inventory/multinode
# kolla-ansible deploy -i /etc/kolla/ansible/inventory/multimode
# kolla-ansible post-deploy
```



SINGLE NODE LAB

All In One OpenStack

환경준비

시작 전, 아래와 같이 파이썬 환경을 준비한다.

```
dnf install -y lvm2  
vgcreate cinder-volumes /dev/vdb  
vgdisplay
```

서버에 사용하는 인터페이스 카드는 총 두 개가 필요하다. 사용자 랩 환경에 따라서 이 부분은 달라진다.

1. 내부용(1개, 터널링)
2. 외부용(1개, 오픈스택 기반으로 2개)



AIO INVENTORY

AIO 기반으로 설치 시, 아래와 같이 구성 및 설정한다. 예제로 제공하는 파일을 그대로 활용한다.

```
mkdir -p /etc/kolla/ansible/inventory/  
cp openstack/share/kolla-ansible/ansible/inventory/all-in-one  
/etc/kolla/ansible/inventory/  
hostnamectl set-hostname aio.local
```



globals.yml

AIO의 설정 및 설치의 다음과 같이 진행한다.

```
# vi /etc/kolla/globals.yml
kolla_base_distro: "rocky"
openstack_release: "2025.1"
node_custom_config: "/etc/kolla/config"
config_strategy: COPY_ALWAYS

# 단일 NIC
network_interface: "ens3"    # 내부/관리망 인터페이스(192.168.10.0/24)

# internal/external 둘 다 내부 인터페이스로 설정
kolla_internal_vip_address: "192.168.10.X" # 내부/관리망 인터페이스
kolla_external_vip_address: "192.168.10.X"
```



globals.yml

위의 내용 계속 이어서...

```
# 단일 컨트롤 노드에서는 아래 옵션은 끄기
enable_haproxy: "no"
enable_keepalived: "no"
enable_proxysql: "no"
enable_cinder_backend_lvm: "yes"
cinder_volume_group: "cinder-volumes"
```

```
# Neutron
```

```
neutron_plugin_agent: "openvswitch"
neutron_external_interface: "ens4"
neutron_bridge_name: "br-ex"
```

```
## 외부네트워크 인터페이스(10.0.0.0/8)
## OVS 외부 브릿지 이름
```



globals.yml

위의 내용 계속 이어서...

```
kolla_enable_tls_external: "no"  
kolla_enable_tls_internal: "no"  
kolla_internal_fqdn: "aio.local"  
kolla_external_fqdn: "aio.local"
```

```
enable_openstack_core: "yes"  
enable_cinder: "yes"  
enable_cinder_backup: "no"  
enable_heat: "yes"  
enable_magnum: "yes"  
enable_octavia: "yes"  
enable_barbican: "yes"
```



globals.yml(덜어내기)

좀 더 가볍게 사용하기 위해서 아래와 같이 옵션이 조정이 가능하다.

선택: 텔레메트리/로그 끄기(가벼움)

enable_central_logging: "no"

enable_prometheus: "no"

enable_grafana: "no"

선택: 모니터링 비활성

enable_central_logging: "no"

enable_prometheus: "no"

enable_grafana: "no"



globals.yml(네트워크)

만약, 점보 프레임 및 cinder 필요한 경우, 아래와 같이 MTU 설정 및 구성한다.

```
# Nova
```

```
nova_compute_virt_type: "kvm"
```



kolla-ansible 실행 준비

아래와 같이 설치를 진행한다.

```
kolla-ansible install-deps
kolla-genpwd
kolla-ansible bootstrap-servers
kolla-ansible pull
kolla-ansible certificates
kolla-ansible octavia-certificates
kolla-ansible prechecks
kolla-ansible deploy
kolla-ansible post-deploy
pip3.11 install python-openstackclient
```



설치 확인 및 테스트

테스트

최종 환경 테스트

오픈스택 랩에서 진행하는 경우, 아래와 같이 올바르게 구성이 되었는지 확인이 가능하다. 혹은 아래 주소에서 내려받기가 가능하다.

```
curl http://10.0.0.250/verify-openstack.sh | sh -  
curl http://10.0.0.250/delete-openstack.sh | sh -
```


최종 환경 테스트

10.0.0.0/8 대역으로 통신이 올바르게 되지 않는 경우, 아래와 같이 명령어를 실행한다.

```
# ip route delete default via 192.168.10.1 dev ens4 proto \
dhcp src 192.168.10.229 metric 101
# ip route delete default via 192.168.20.1 dev ens9 proto \
dhcp src 192.168.20.193 metric 103
# ip addr flush ens8          ## br-ex가 사용하는 인터페이스
# ip link set br-ex up
# ip addr add 10.0.X.Y/8 dev br-ex
# ip addr show br-ex
```

오픈스택 기본 컴포넌트 소개

오픈스택 코어

Oslo/RabbitMQ

기본 기능

Oslo란 무엇인가?

Oslo – OpenStack 공통 유틸리티 라이브러리

- OpenStack 서비스 간 중복 기능을 표준화한 Python 라이브러리 모음
- 코드 재사용, 유지보수성, 일관성 확보 목적

주요 모듈

- oslo.config: 설정 파일 파싱
- oslo.db: DB 연결 및 마이그레이션
- oslo.log: 로깅 통합
- oslo.messaging: 메시징 추상화 (→ RabbitMQ 연동)
- oslo.concurrency, oslo.serialization 등



코어 서비스 개요

OpenStack의 기원은 2010년, **NASA**와 **Rackspace**의 협업에서 시작이 되었음. 이 시점에는 오픈스택에 Oslo, RabbitMQ와 같은 기능은 포함이 되지 않았음.

1. NASA는 Nebula라는 내부 클라우드 프로젝트를 통해 VM 관리를 자동화 목적
2. Rackspace는 자체적인 오브젝트 스토리지 시스템(Swift)을 가지고 있었고, 이를 공개하면서 협업 시작

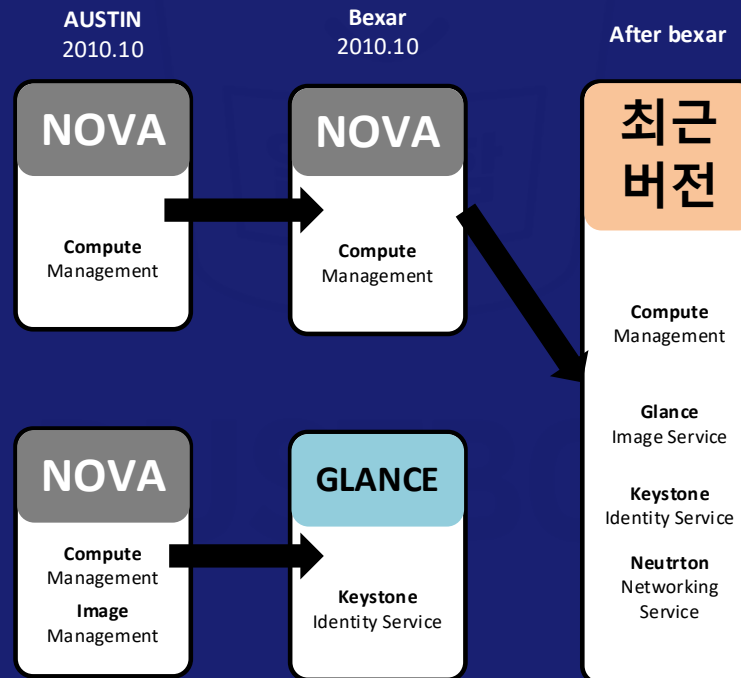
초기 목적은 **"가상머신 인프라 자동화"**였고, 이후 커뮤니티 확장을 거치며 IaaS 전체 영역(컴퓨트, 네트워크, 스토리지 포함)으로 발전.

NASA는 초기 버전의 Nova(컴퓨트 서비스)를 개발하면서, **"VM 인스턴스의 프로비저닝, 스케줄링, 라이프사이클 관리"**에 집중.

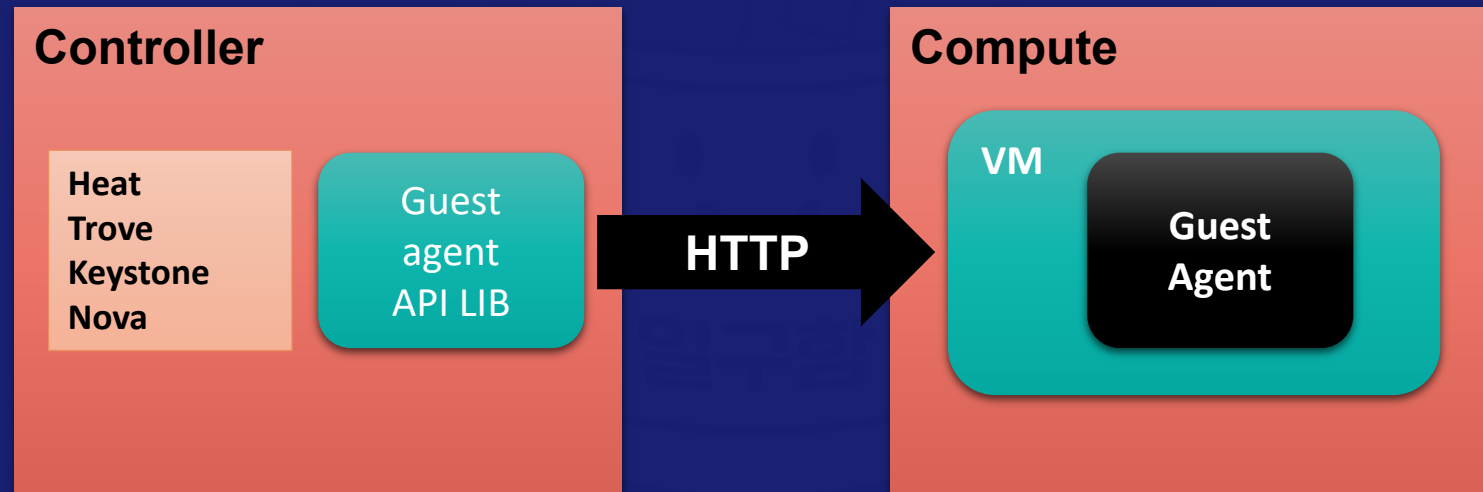


코어 서비스 개요

코어 서비스는 본래 Nova에서 시작하여, 확장성을 고려하면서 점점 아키텍처 확장이 되면서 특정 기능이 컴포넌트 (구성원)으로 분리가 되기 시작하였다. 기본적인 모습은 Bexar에서 어느정도 갖추기 시작했다.



Oslo CORE



RabbitMQ란 무엇인가?

RabbitMQ – 메시지 브로커

- AMQP 기반 비동기 메시징 시스템
- OpenStack 서비스 간 RPC 호출, 알림 처리 담당

주요 구성 요소

- Producer/Consumer
- Exchange → Queue → 메시지 전달
- 서비스 간 메시지의 중간자 역할

사용 예시

Nova API → 메시지 전송 → RabbitMQ → Nova Compute



Oslo.messaging + RabbitMQ

Oslo와 RabbitMQ의 관계

- oslo.messaging이 메시지 송수신 로직을 추상화
- 실제 메시지 전송은 RabbitMQ가 수행

메시지 흐름

1. Oslo는 인터페이스
2. RabbitMQ는 운송 수단



요약 및 비교

Oslo = 뼈대 / RabbitMQ = 혈관

항목	Oslo	RabbitMQ
역할	OpenStack 공통 라이브러리 (설정, 메시징 등)	메시지 브로커 (서비스 간 비동기 통신 중계)
기능	설정 관리, DB 연결, 메시징 인터페이스, 로그 등	큐잉, 라우팅, 메시지 저장 및 전달
구성	Python 패키지 모듈 (예: oslo.config, oslo.messaging 등)	AMQP 기반 구성 (Exchange, Queue, Producer 등)
주요 예시	oslo.messaging → 메시지 큐 인터페이스 제공	Nova API → 메시지 전송 → Nova Compute 처리
연관성	oslo.messaging이 메시지를 추상화하고 RabbitMQ가 처리함	실제 메시지 운송 및 수신자 전달 담당
비유	OpenStack의 뼈대 역할 (공통 구조)	OpenStack의 혈관 역할 (데이터 흐름)



메세지 브로커

이 교육에서는 메세지 브로커 구조/구성 및 명령어는 이 과정에서 학습 하지 않는다. 별도 과정이 필요한 경우 강사에게 따로 문의.



명령어

openstack-client

로그인

오픈스택 명령어를 사용하기 위해서 **openstack** 명령어 혹은 컴포넌트 명령어에 API 및 SECRET키를 전달해야 한다. 여기서 말하는 SECRET은 아이디 및 비밀번호까지 포함이다. 전달하기 위해서 두 가지 방법을 제공하고 있다.

1. BASH기반의 환경 변수 전달
2. YAML기반의 환경 변수 전달

초창기 및 기존 사용자는 1번 방법을 많이 사용하였으나, 여러 클러스터를 관리하면서 1번으로 여러 클러스터 관리가 어려운 부분이 있다. 이러한 이유로 여러 클러스터를 접근하기 위해서 YAML기반으로 작성 및 접근 관리 방법이 제안이 되었고, 현재 최근 릴리즈는 YAML기반으로 CLI사용을 권장하고 있다.



앞에서 다루었던 내용

앞에 잠깐 다루었던 로그인 부분을 다시 한 번 집고 넘어간다.



로그인 구성(셸 변수; 키스톤)

셸 기반은 다음과 같이 구성한다. 셸 기반으로 구성하는 경우 스크립트로 작성한다.

```
# vi project-openrc.sh
export OS_AUTH_URL="https://keystone.example.com:5000/v3"
export OS_USERNAME="your_user"
export OS_PASSWORD="your_password"
export OS_PROJECT_NAME="your_project"
export OS_USER_DOMAIN_NAME="vlab-users"           # ex) Default, vlab-users
export OS_PROJECT_DOMAIN_NAME="vlab-users"         # 프로젝트가 속한 도메인
export OS_IDENTITY_API_VERSION=3
export OS_REGION_NAME="RegionOne"
export OS_INTERFACE="public"
export OS_CACERT="/path/to/ca.crt"
# openstack token issue
```



로그인 구성(셸 변수; 애플리케이션 자격)

애플리케이션 자격으로 접근하는 경우 다음과 같이 작성한다.

```
# vi appcred-openrc.sh
export OS_AUTH_URL="https://keystone.example.com:5000/v3"
export OS_AUTH_TYPE="v3applicationcredential"
export OS_APPLICATION_CREDENTIAL_ID="APP_CRED_ID"
export OS_APPLICATION_CREDENTIAL_SECRET="APP_CRED_SECRET"

export OS_IDENTITY_API_VERSION=3
export OS_REGION_NAME="RegionOne"
export OS_INTERFACE="public"
export OS_CACERT="/path/to/ca.crt"    # 필요 시
# openstack token issue
```



로그인 구성(클라우드 셸; 키스톤)

cloud shell 기반으로 구성 시 다음과 같이 구성한다.

```
# vi ~/.config/openstack/clouds.yaml
clouds:
  mycloud:
    region_name: RegionOne
    interface: public
    verify: true                # TLS 검증
    cacert: /path/to/ca.crt    # 자체서명 CA일 때 추가
    auth:
      auth_url: https://keystone.example.com:5000/v3
      username: your_user
      password: your_password
      project_name: your_project
      user_domain_name: vlab-users
      project_domain_name: vlab-users
    identity_api_version: 3
```



로그인 구성(클라우드 셸; 애플리케이션 자격)

cloud shell 기반으로 구성 시 다음과 같이 구성한다.

```
# vi ~/.config/openstack/clouds.yaml
clouds:
  mycloud-app:
    region_name: RegionOne
    interface: public
    verify: true
    cacert: /path/to/ca.crt
    auth_type: v3applicationcredential
    auth:
      auth_url: https://keystone.example.com:5000/v3
      application_credential_id: APP_CRED_ID
      application_credential_secret: APP_CRED_SECRET
    identity_api_version: 3
```



오픈스택 명령어

오픈스택 명령어는 기본적으로 파이선으로 작성되어 있다. 기능이 추가가 되면, PIP3를 통해서 명령어를 확장해야 한다. 다만, **openstack** 통합 명령어는 아직 모든 컴포넌트의 명령어를 다 수행하지 않기 때문에, 종종 컴포넌트 명령어를 통해서 구성 및 관리가 필요한 경우가 있다.

오픈스택 명령어 사용 방법은 다음과 같다.

openstack <COMPONENT_NAME> <VERBE> <OPTIONS> <OBJECT_NAME>

보통 위와 같은 형식으로 명령어를 작성 및 수행한다.



명령어 확장

오픈스택 명령어 설치하는 다음과 같은 방법으로 진행한다. 다만, 배포판 및 구성에 따라서 조금씩 다르다.

```
# python3 --version
# pip3 --version
# sudo apt update
# sudo apt install python3 python3-pip -y
# python3 -m venv openstack-env
# source openstack-env/bin/activate
# pip install python-openstackclient
```



명령어 확장

명령어를 확장하기 위해서 다음과 같이 추가가 가능하다. 다만, 구성에 따라서 조금씩 달라진다.

```
# pip install python-glanceclient python-neutronclient
```

서비스	패키지 이름
Compute(Nova)	python-novaclient
Image(Glance)	python-glanceclient
Network(Neutron)	python-neutronclient
Block Storage(Cinder)	python-cinderclient
Identity(Keystone)	python-keystoneclient
Object Storage(Swift)	python-swiftclient
Heat	python-heatclient



명령어 보안(clouds.yaml)

오픈스택 명령어를 좀 더 안전하게 사용하기 위해서 컨테이너 기반으로 실행이 가능하다.

다음과 같이 포드만에서 사용이 가능하다. 아래와 같이 실행하면, 사용자가 사용하는 `clouds.yaml` 파일 기반으로 오픈스택 API에 접근이 가능하다.

```
# podman run -it --rm \  
-v $HOME/.config/openstack:/root/.config/openstack \  
docker.io/openstackclients/openstackclient:latest
```



클라이언트 명령어 보안(ENV)

혹은 기존 방식을 사용하는 경우, 다음과 같이 적용이 가능하다.

```
# vi $HOME/.config/openstack
export OS_AUTH_URL=https://<API_URL>:5000/v3
export OS_PROJECT_NAME=admin
export OS_USERNAME=admin
export OS_PASSWORD=<PASSWORD>
export OS_USER_DOMAIN_NAME=Default
export OS_PROJECT_DOMAIN_NAME=Default
# podman run -it --rm \
-v $HOME/.config/openstack:/root/.config/openstack \
docker.io/openstackclients/openstackclient:latest
```



클라이언트 명령어 보안(ENV)

혹은 기존 방식을 사용하는 경우, 다음과 같이 적용이 가능하다.

```
# podman run -it --rm \  
-e OS_AUTH_URL \  
-e OS_PROJECT_NAME \  
-e OS_USERNAME \  
-e OS_PASSWORD \  
-e OS_USER_DOMAIN_NAME \  
-e OS_PROJECT_DOMAIN_NAME \  
docker.io/openstackclients/openstackclient:latest
```



클라이언트 명령어 보안(alias)

영구적으로 사용하기 위해서 최종적으로 아래와 같이 스크립트에 적용한다.

```
# alias osc="podman run -it --rm \  
-v $HOME/.config/openstack:/root/.config/openstack \  
docker.io/openstackclients/openstackclient:latest"
```



연습문제

구성된 오픈스택 클러스터에 `openstack` 명령어로 접근이 되도록 구성한다.

1. 사용자는 `clouds.yaml` 파일 기반으로 접근
2. `openstack token issue` 명령어로 올바르게 접근이 되었는지 확인
3. `openstack project list`로 올바르게 프로젝트 목록이 조회가 되는지 확인

오픈스택 CLI 및 대시보드

Horizon/Skyline

OpenStack CLI

운영자는 무엇을 사용해야 하는가?

오픈스택 클러스터를 운영하는 관리자에게 있어 CLI(Command Line Interface)는 필수적인 도구로 간주된다. CLI는 단순한 명령 실행을 넘어서, **스크립트 기반 자동화, 배치 작업, 그리고 대규모 리소스 관리**에 최적화되어 있다.

예를 들어 수십 개의 인스턴스를 생성하거나, 다수의 네트워크와 볼륨을 조작하는 작업은 CLI를 통해 몇 줄의 스크립트로 처리할 수 있으며, 이는 운영의 일관성과 재현성을 크게 향상시킨다.



운영자는 무엇을 사용해야 하는가?

반면, **Horizon 대시보드**는 웹 기반 그래픽 사용자 인터페이스(GUI)로, 리소스의 상태를 직관적으로 확인하거나 간단한 조작을 수행할 때 유용하다. 간단한 작업이나 혹은 관리를 위해서 대시보드로 처리가 가능하다. 하지만, **대규모/반복적인 작업**에는 매우 부적절하다.

CLI는 실질적인 인프라 운영의 핵심이며, Horizon은 시각적 모니터링이나 간단한 작업 처리, 교육 목적 등에 적합한 도구이다. 이 둘을 적절히 병행하면, 운영자는 신속성과 안정성을 동시에 확보할 수 있다.

결론적으로, 오픈스택 운영자는 **CLI를 기본 도구로 사용**하되, Horizon 대시보드는 **보조 수단**으로 활용하는 것이 바람직하다.



대시보드 허용 범위

약 40~60% 수준의 CLI 기능만 대시보드(Horizon)에서 제공됩니다.

- 사용자 권한에 따라 약간의 차이는 있으나, 관리자 기준으로 전체 기능 중 절반을 넘기 어려움

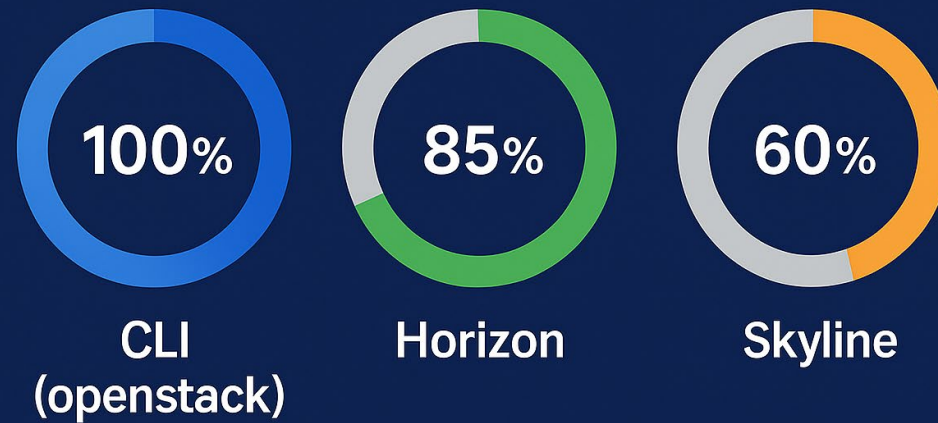
Horizon은 전체 OpenStack CLI 기능의 절반 가량만 제공하며, 특히 고급 기능과 자동화, 디버깅, 세밀한 설정은 대부분 CLI 또는 REST API를 통해서만 가능하다.

따라서 운영자 입장에서는 대시보드는 시각적 보조 수단으로 활용하고, 실제 운영 핵심은 CLI를 중심으로 구성해야 한다.



관리도구 비교

OpenStack 관리도구 비교 (100% 기준)



항목 살펴보기

다음과 같은 항목에 대해서 비교가 가능하다.

기능 영역	CLI 지원	Horizon 지원	비고
인스턴스 생성/삭제	지원	지원	기본 기능
네트워크, 서브넷, 라우터	지원	지원	다소 제한된 옵션
보안 그룹 설정	지원	지원	고급 규칙 작성 어려움
사용자/프로젝트 관리	지원	지원	도메인 기반 권한 제어는 CLI 권장
스택(Heat) 관리	지원	네니요	템플릿 등록/로그 확인 어려움
볼륨(Cinder) 조작	지원	지원	세부 옵션 미지원
이미지 등록/관리	지원	지원	CLI는 glance 옵션 더 풍부



항목 살펴보기

다음과 같은 항목에 대해서 비교가 가능하다.

기능 영역	CLI 지원	Horizon 지원	비고
Floating IP 연결	지원	지원	제한 없음
서비스 상태 확인	지원	미지원	openstack service list, status 불가
정책, RBAC, Role 설정	지원	미지원	CLI 전용
고급 디버깅/로그 조회	지원	미지원	Horizon은 대부분 표시 불가
메타데이터, Flavor 수정	지원	미지원	CLI/REST API만 가능
대규모 반복 작업	지원	미지원	GUI는 자동화 불가



Horizon

기본 기능

개요

Horizon은 OpenStack에서 제공하는 기본 웹 기반 UI 관리 도구로, 관리자와 일반 사용자 모두 GUI를 통해 손쉽게 자원을 생성하고 관리할 수 있다.

하지만 **Horizon**은 CLI나 API와 비교해 일부 고급 기능이 제공되지 않으며, 전체 기능의 약 절반 이상만 제공되는 평가가 있다.

최근에는 **Skyline**이라는 현대적인 UI 대시보드도 함께 제공되고 있지만, 아직은 핵심 서비스 위주의 기능만을 지원하며, 실제 운영 환경에서는 여전히 Horizon이 기본 선택지로 널리 사용되고 있다.



Horizon

OpenStack Horizon은 웹 기반 대시보드로, OpenStack 사용자가 **프로젝트 자원(가상 머신, 네트워크, 이미지 등)**을 시각적으로 관리할 수 있도록 도와주는 프론트-엔드 인터페이스이다.

Django 프레임워크 기반으로 개발되었으며, **관리자(Admin)**와 **일반 사용자(User)** 모두가 사용할 수 있는 구조를 갖는다. CLI나 API에 익숙하지 않은 사용자들에게 직관적인 사용자 경험을 제공한다는 점에서 높은 접근성을 가진다.

Horizon은 OpenStack 초기부터 존재해 온 핵심 컴포넌트 중 하나이며, 다양한 플러그인을 통해 **Nova, Neutron, Glance, Cinder, Heat** 등의 서비스와 통합된다.

다만 구조상 확장성이 제한적이며, 현대적인 UX/UI 트렌드에는 다소 뒤처진다는 평가를 받는다. 특히 대규모 클라우드 환경이나 복잡한 역할 기반 접근 제어(RBAC)가 필요한 경우에는 Horizon만으로 충분하지 않은 경우도 있다.



Horizon 인터페이스

openstack

Defaultlab.pacemaker

Project

API Access

Compute

Overview

Instances

Images

Key Pairs

Server Groups

Rating

Volumes

Container

Container Infra

Network

Orchestration

Databases

DNS

Share

Reservations

Identity

Workflow

NFV

Project / Compute / Instances

Instances

Instance ID ▾

Filter

Launch Instance

Delete Instances

More Actions ▾

Displaying 6 items

☐	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions	
☐	pacemaker-small-server-03	lab-rocky-9	net-infra 10.0.2.98 iac-private 192.168.100.12	m1.medium	-	Active	us-east-1a	nova	None	Running	5 minutes	Create Snapshot ▾
☐	pacemaker-small-server-06	lab-rocky-9	iac-private 192.168.100.15 net-infra 10.0.5.140	m1.medium	-	Active	us-east-1a	nova	None	Running	5 minutes	Create Snapshot ▾
☐	pacemaker-small-server-05	lab-rocky-9	net-infra 10.0.1.171 iac-private 192.168.100.14	m1.medium	-	Active	us-east-1a	nova	None	Running	5 minutes	Create Snapshot ▾
☐	pacemaker-small-server-04	lab-rocky-9	iac-private 192.168.100.13 net-infra 10.0.5.50	m1.medium	-	Active	us-east-1a	nova	None	Running	5 minutes	Create Snapshot ▾
☐	pacemaker-small-server-02	lab-rocky-9	iac-private 192.168.100.11 net-infra 10.0.4.36	m1.medium	-	Active	us-east-1a	nova	None	Running	5 minutes	Create Snapshot ▾
☐	pacemaker-small-server-01	lab-rocky-9	net-infra 10.0.4.247 iac-private 192.168.100.10	m1.medium	-	Active	us-east-1a	nova	None	Running	5 minutes	Create Snapshot ▾

Displaying 6 items



Skyline

Skyline은 Horizon의 한계를 극복하고자 새롭게 등장한 OpenStack 대시보드 프로젝트로, React 기반의 프론트엔드와 FastAPI 기반의 백-엔드를 사용한다.

모던 웹 기술을 채택하여 성능 개선과 사용자 경험 향상을 목표로 하며, Horizon보다 빠른 렌더링과 가벼운 인터페이스를 제공한다. 그러나 Skyline은 아직 비교적 초기 단계에 있으며, 기능 커버리지가 Horizon에 비해 부족한 편이다.

특히, 세밀한 프로젝트·도메인 관리, 사용자 생성, 로깅 설정 등의 고급 관리 기능은 아직 구현되지 않았거나 제한적으로 지원된다.

그럼에도 불구하고, 향후 Horizon을 완전히 대체할 수 있는 후보로 간주되고 있으며, OpenStack 커뮤니티에서도 적극적으로 발전시키고 있다.



PODMAN/DOCKER

런타임

런타임 환경(Docker)

OpenStack에서 **Docker**는 전통적으로 컨테이너 관리에 사용되어 왔으며, 특히 **Kolla-Ansible**과 같은 프로젝트에서는 Docker를 전제로 많은 템플릿과 스크립트가 구성되어 있다.

Docker는 백그라운드에서 dockerd라는 **데몬이 항상 실행** 중이며, 시스템 전체 컨테이너를 중앙 집중식으로 관리한다. 이로 인해 OpenStack 서비스에서 컨테이너 상태를 모니터링하거나 제어하는 데에 편리하다.

OpenStack에서는 Dockerd가 아닌, **containerd** 기반으로 오픈스택 컨테이너를 구성 및 사용한다.

참고: 2025.1 릴리즈 이후로 podman으로 대다수 컴포넌트 운영이 가능하다. 다만, 플레이북에 사소한 오류들이 있으니, 적절하게 수정 및 운영이 필요하다.



런타임 환경(Podman)

Podman은 **데몬리스** 방식으로 각 컨테이너가 사용자 프로세스로 실행되며, **rootless** 모드도 지원한다.

이는 보안상 이점을 제공하지만, OpenStack처럼 시스템 단에서 여러 계정이 동시에 컨테이너를 제어해야 하거나, 중앙에서 로그를 수집하고 상태를 모니터링해야 하는 환경에서는 일부 제약이 생길 수 있다.

또한, Podman은 일부 기능(예: volume plugin, swarm, 일부 Compose 기능 등)이 Docker와 완전히 호환되지 않는다.

가볍게 오픈스택 클러스터를 구성하는 경우 권장하며, 모든 기능을 사용하고 싶은 경우, 가급적이면 Containerd 기반으로 구성을 권장한다.



오픈스택 런타임 비교

오픈스택에서 런타임 선택 시, 다음과 같은 부분을 고려해서 선택한다.

기능/특성	Docker	Podman	비고
데몬 구조	dockerd 데몬 항상 실행	무데몬(Daemonless)	시스템 통합에 차이 발생
rootless 실행	제한적 지원	완전한 지원	Podman이 유리함
systemd 통합	별도 설정 필요	기본 통합 가능	Podman이 유리함
Kolla-Ansible 호환성	공식 지원	실험적 (추가 설정 필요)	Docker 권장
docker-compose 지원	기본 지원	별도 도구 필요	일부 기능 차이 있음
swarm 클러스터링	기본 지원	미지원	Docker 우위
volume plugin 지원	다양한 플러그인 지원	제한적 지원 (rootless에 서는 미지원)	일부 기능 제한



오픈스택 런타임 비교

위의 내용 계속 이어서...

기능/특성	Docker	Podman	비고
컨테이너 이미지 관리	docker pull/push	동일	이미지 포맷 동일
SELinux 및 AppArmor 통합	기본 지원	기본 지원 + fine-grained control	Podman이 강력함
컨테이너 디버깅	지원	대부분 동일하게 지원	유사
컨테이너 네트워크 드라이버	bridge, host, overlay 등 다양	기본 bridge, slirp4netns (rootless)	일부 제한
OCI 호환성	지원	완전 지원	동일



명령어 확인

아래 명령어로 Docker/Podman에 구성된 컨테이너 자원을 확인한다.

```
# docker ps
# podman container ls
# docker network ls
# podman network ls
# docker volume ls
# podman volume ls
```



결론

OpenStack에서 운영 시 Docker는 여전히 표준 컨테이너 런타임으로서 가장 널리 사용되며, 대부분의 오픈스택 구성 도구(Kolla, Triple-O, Magnum 등)와 완전한 호환성을 보장한다.

반면 Podman은 보안과 유연성 면에서는 더 우수하지만, OpenStack의 기존 도구 체인과 완벽히 호환되지 않거나 일부 기능에서 추가 설정이 필요하다.



결론

Podman을 사용할 계획이라면 다음과 같은 점을 고려해야 한다.

1. rootless Podman을 사용할 경우, OpenStack이 사용하는 로그/볼륨/네트워크 통합에 제한이 생길 수 있음
2. Kolla 기반 구성에서는 docker → podman 마이그레이션이 어렵고, 호환되지 않는 CLI나 시스템 단 의존성 이슈가 존재함
3. podman generate systemd 기능 등을 활용해 자동화는 가능하지만, OpenStack의 기존 운영 모델과 통합되기 위해서는 추가적인 조정이 필요함
4. 인프라 엔지니어는 docker/podman 차이점 및 기술에 대해서 반드시 학습해야 함

필요하다면 Podman 기반으로 OpenStack 환경을 테스트/학습용으로 운영할 수는 있지만, 프로덕션 환경에서는 Docker 사용이 현재로서는 더 안전하고 안정적인 선택지라고 볼 수 있다.



오픈스택+LLM

오픈스택과 AI Assistant 기능

LLM과 SLM 개요

현재 LLM 시장에서는 다음과 같이 구별이 되고 있다.

1. 대형 및 여러 모델을 함께 사용하는 LLM 모델
2. 특정 기능 위주로 구성된 SLM 모델

인프라 경우에는 보통 여러 기능이 필요 없기 때문에 SLM 모델을 사용해서 오픈스택을 좀 더 쉽게 사용이 가능하다. 하지만, LLM, SLM에 대해서 아직 익숙하지 않는 인프라 엔지니어를 위해서 간단하게 설명을 하고 진행한다.



LLM과 SLM 개요

LLM (Large Language Model)

- 대규모 범용 언어 모델
- 외부 API 기반 활용이 일반적
- 높은 성능/높은 비용/낮은 통제력
- GPU는 필수

SLM (Small Language Model)

- 특정 목적에 최적화된 경량 모델
- 온프레미스/폐쇄망 환경에 적합
- 운영 보조 역할에 현실적인 선택
- CPU 혹은 가속기에 따라서 GPU LESS로 구성 가능



온프레미스 환경에서 SLM의 역할

왜 SLM이 필요한가

- 내부 로그·설정 정보 외부 반출 제한
- 비용 예측 가능한 AI 필요
- 네트워크 단절 환경 고려

SLM의 현실적인 역할

- 시스템 상태 요약
- 반복 로그 패턴 분석
- 운영자 질문에 대한 설명 제공



OpenStack 운영에 AI를 활용하는 방식

두 가지 방법으로 접근이 가능하다.

1. 오픈스택 CLI 명령어 생성/조회/실행
2. IaC 기반으로 코드 생성 후 자원 관리

오픈스택 CLI를 사용하는 경우, 종종 존재하지 하위 명령어 및 자원을 접근하는 경우가 있다. 이러한 이유로 CLI 기반으로 AI 도움을 받는 경우, 반드시 명령어에 대해서 학습해야 한다.

IaC도 역시 동일한 문제가 있으며, 이 부분도 스스로 어느정도 IaC 문법 및 모듈에 대해서 이해가 되어야지 사용이 가능하다.



OpenStack 운영에 AI를 활용하는 방식

AI가 개입할 수 있는 지점

- 대규모 로그에서 핵심 내용 요약
- 장애 징후에 대한 설명 제공
- 자연어 → 명령어 변환 보조

활용 예시

- 이 에러 로그가 의미하는 게 뭐지?
- 이 상태에서 다음으로 확인할 명령은?
- 이 설정이 왜 필요한가?



AI SHELL

오픈스택 쉽게 운영하기 위해서 AI Assistant가 통합된 KIKI SHELL를 설치 후 적용한다. KIKI SHELL은 아래 주소에서 다운로드 후 사용이 가능하다.

```
# curl -fsSL http://10.0.0.250/kiki-shell -o /usr/local/bin/kiki
# usermod -s /usr/local/bin/ki2sh <USERNAME>
# su - <USERNAME>
kiki> alias os='openstack'
kiki> alias osvm='openstack server'
kiki> osvm list
kiki> ? 내가 방금 만든 alias(os, osvm)를 기준으로 자주 쓰는 운영 명령 10개를 추천
해줘
kiki> :stream on
kiki> ? 지금부터 "Glance 업로드 실패" 상황을 가정하고, 내가 하나씩 실행할 명령을 단
계별로 안내해줘. 다음 단계로 넘어갈 때마다 '다음'이라고 할게.
```



연습문제

KIKI SHELL 기반으로 간단하게 셸 및 오픈스택 자원을 검색해본다.

1. 키스톤 자원에서 사용자를 자연어로 조회한다.
2. 인스턴스가 구성이 되어 있는지 자연어로 조회한다.
3. 네트워크가 구성이 되어 있는지 자연어로 조회한다.

Keystone

기본 기능

Keystone 개요

Keystone은 OpenStack의 인증(Identity) 서비스를 담당하며, 사용자 인증, 권한 관리, 서비스 등록 및 서비스 접근 위치(Service Endpoint) 정보 제공 등을 수행한다.

Keystone은 다음과 같은 주요 기능을 제공한다:

1. 사용자(User), 프로젝트(Project), 역할(Role)의 생성 및 권한 할당

- 이를 통해 OpenStack 자원에 대한 접근 제어를 수행할 수 있다.

2. 인증 토큰 발급 및 검증

- 사용자가 OpenStack API에 접근하기 위해서는 Keystone으로부터 토큰을 발급받아야 하며, 이 토큰은 요청 시 인증 수단으로 사용된다.



Keystone 개요

3. 서비스 카탈로그(Service Catalog) 제공

- 등록된 OpenStack 서비스들의 목록과 해당 서비스의 접근 엔드포인트(Endpoint) 정보를 포함하며, 토큰 발급 시 함께 반환된다.
- 엔드포인트는 보통 public (외부 접근용), internal (내부 서비스간 호출용), admin (관리자용) 세 가지 URL로 구성된다.

4. 서비스 등록 및 서비스 계정(Service Account) 관리

- OpenStack의 각 서비스(Nova, Glance, Neutron 등)는 Keystone에 등록되어야 하며, 이때 해당 서비스에 접근할 수 있는 전용 사용자 계정(서비스 계정)이 생성된다.
- 서비스 계정은 일반 사용자와는 별도로 서비스 간 통신 및 인증을 위해 사용된다.



Keystone 개요

도메인(Domain) 개념

Keystone은 사용자 및 프로젝트를 도메인 단위로 관리할 수 있으며, 멀티 도메인 환경에서 권한과 자원 관리를 유연하게 할 수 있다.

정책 관리 (Policy)

Keystone은 서비스 전반의 접근 정책을 policy.json 혹은 policy.yaml을 통해 정의하여, 역할 기반 접근 제어 (RBAC)를 적용할 수 있다.

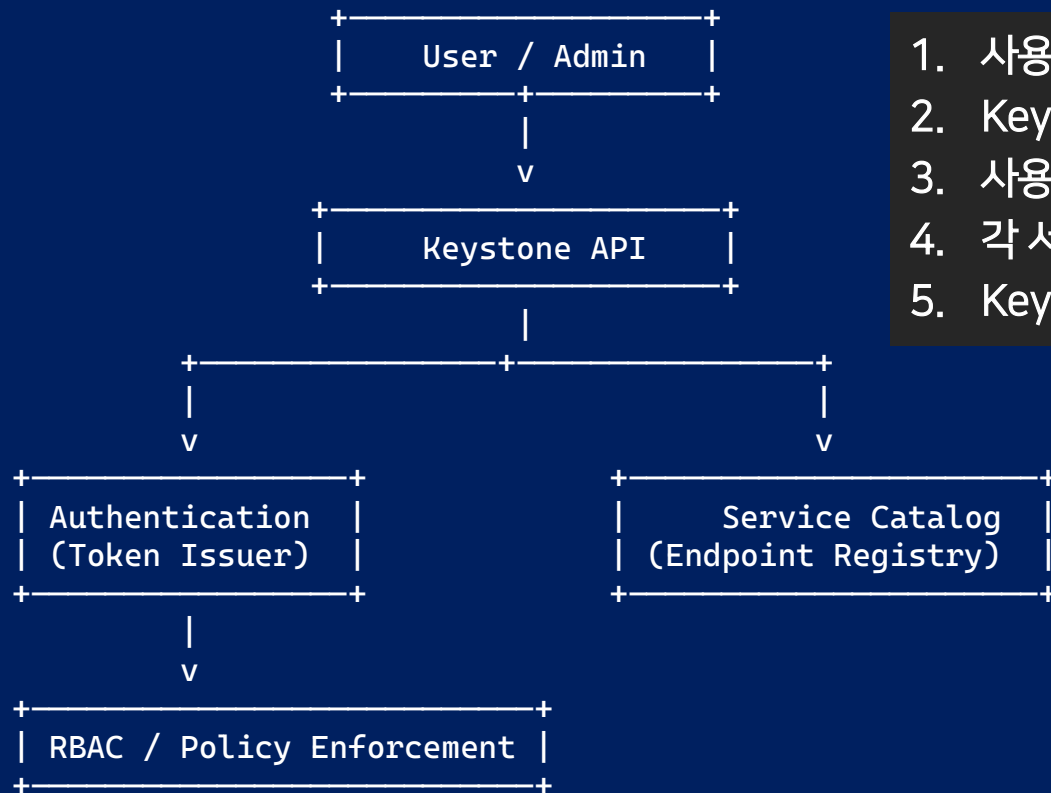
Federation & 외부 인증

LDAP, SAML, OIDC 등 외부 인증 시스템과의 연동이 가능하며, 이를 통해 싱글사인온(SSO) 환경도 구성할 수 있다.



Keystone 개요

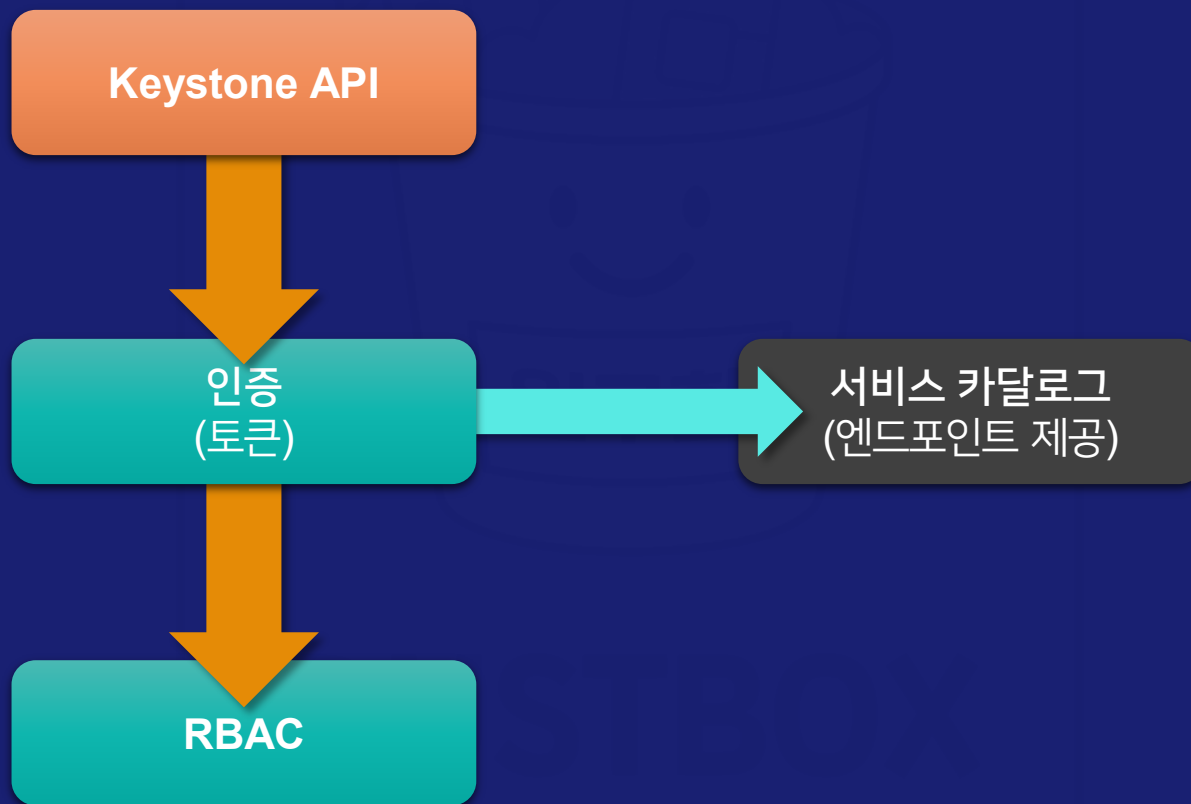
아래와 같이 Keystone 동작한다.



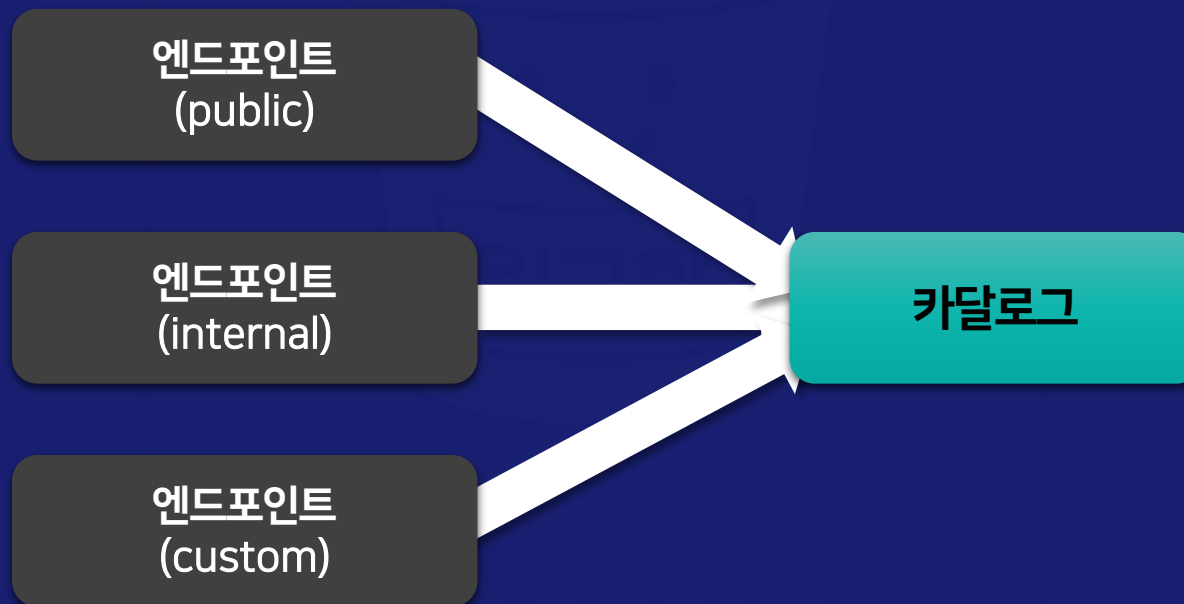
1. 사용자가 Keystone에 사용자명/비밀번호로 로그인 요청
2. Keystone이 사용자 인증 후 토큰 발급
3. 사용자는 이 토큰을 가지고 다른 OpenStack 서비스에 접근
4. 각 서비스는 Keystone에 토큰 유효성 확인 요청
5. Keystone이 유효한 토큰임을 확인하고 응답



Keystone 계층



Keystone 계층



Keystone 주요 기능 정리

기능을 표로 정리하면 다음과 같다. 자세한 설명은 뒤에서 좀 더 다룬다.

기능 영역	설명
인증(Authentication)	사용자 인증 후 토큰 발급
권한(Authorization)	역할 기반 권한 부여 (RBAC)
서비스 카탈로그	API 서비스 목록 및 위치 정보 제공
정책 엔진	policy.yaml 기반 권한 규칙
멀티도메인 지원	도메인별 사용자/프로젝트 분리 관리
외부 연동	LDAP, SAML, OAuth, Kerberos 등과 통합 가능



Keystone 인증 흐름 요약

1. 사용자가 Keystone에 사용자명/비밀번호로 로그인 요청
2. Keystone이 사용자 인증 후 토큰 발급
3. 사용자는 이 토큰을 가지고 다른 OpenStack 서비스에 접근
4. 각 서비스는 Keystone에 토큰 유효성 확인 요청
5. Keystone이 유효한 토큰임을 확인하고 응답



컨테이너 프로그램

Keystone은 호스트에서 더 이상 실행하지는 않는다. 여전히 호스트에서 사용이 가능하나, 업그레이드 및 업데이트를 안전하게 하기 위해서 컨테이너 기반으로 사용을 권장한다.

`docker`, `podman`으로 사용이 가능하며 다음 명령어로 동작중인 런타임 컨테이너 확인이 가능하다.

```
docker ps | grep keystone  
podman ps | grep keystone
```

```
docker volume ls
```



설정파일

Keystone 설정은 설치 시, 자동으로 앤서블 템플릿으로 생성 및 배포가 된다. 이러한 이유로 별도로 사용자가 구성할 필요는 없다. 다만, 사용자가 설정을 변경 원하는 경우 `/etc/kolla/config/`에서 변경이 가능하다.

파일 형식은 INI으로 생성 및 구성해야 한다.

```
/etc/kolla/config/  
└─ keystone/  
    └─ keystone.conf           # Keystone 메인 오버라이드  
    └─ domains/  
        └─ keystone.vlab-users.conf # 도메인별 드라이버(예: LDAP/별도DB)  
    └─ keystone-paste.ini      # 필요 시 파이프라인 조정(선택)
```



설정파일

아래와 같이 생성 및 구성된다.

```
vi /etc/kolla/config/keystone/keystone.conf
[DEFAULT]
# 로깅 레벨(선택)
debug = false

[identity]
# 도메인별 드라이버 활성화 + 도메인별 설정 디렉터리
domain_specific_drivers_enabled = true
domain_config_dir = /etc/keystone/domains
```



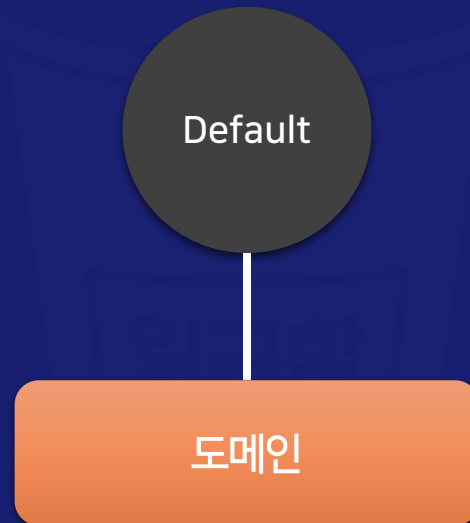
전체 명령어 구조

Domain: vlab-users

- |— **Project:** vlab-project
 - | |— User: vlab-user1
 - | | |— Role: student
 - | |— Group: training-group
- |— **Endpoint:** Glance; Cinder; Nova...
- |— **Service:** Glance; Cinder; Nova...
- |— **Region:** RegionOne / RegionTwo
 - |— Service: compute (nova)
 - |— Service: image (glance)
 - |— Service: network (neutron)



도메인 구성

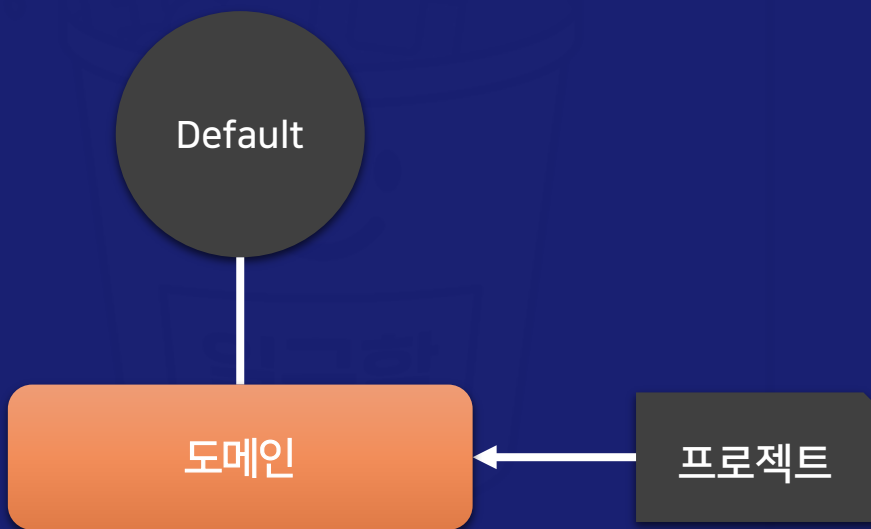


도메인 생성

```
# openstack domain create vlab-users --description "VLabs User Domain"  
# openstack domain list
```



프로젝트 구성

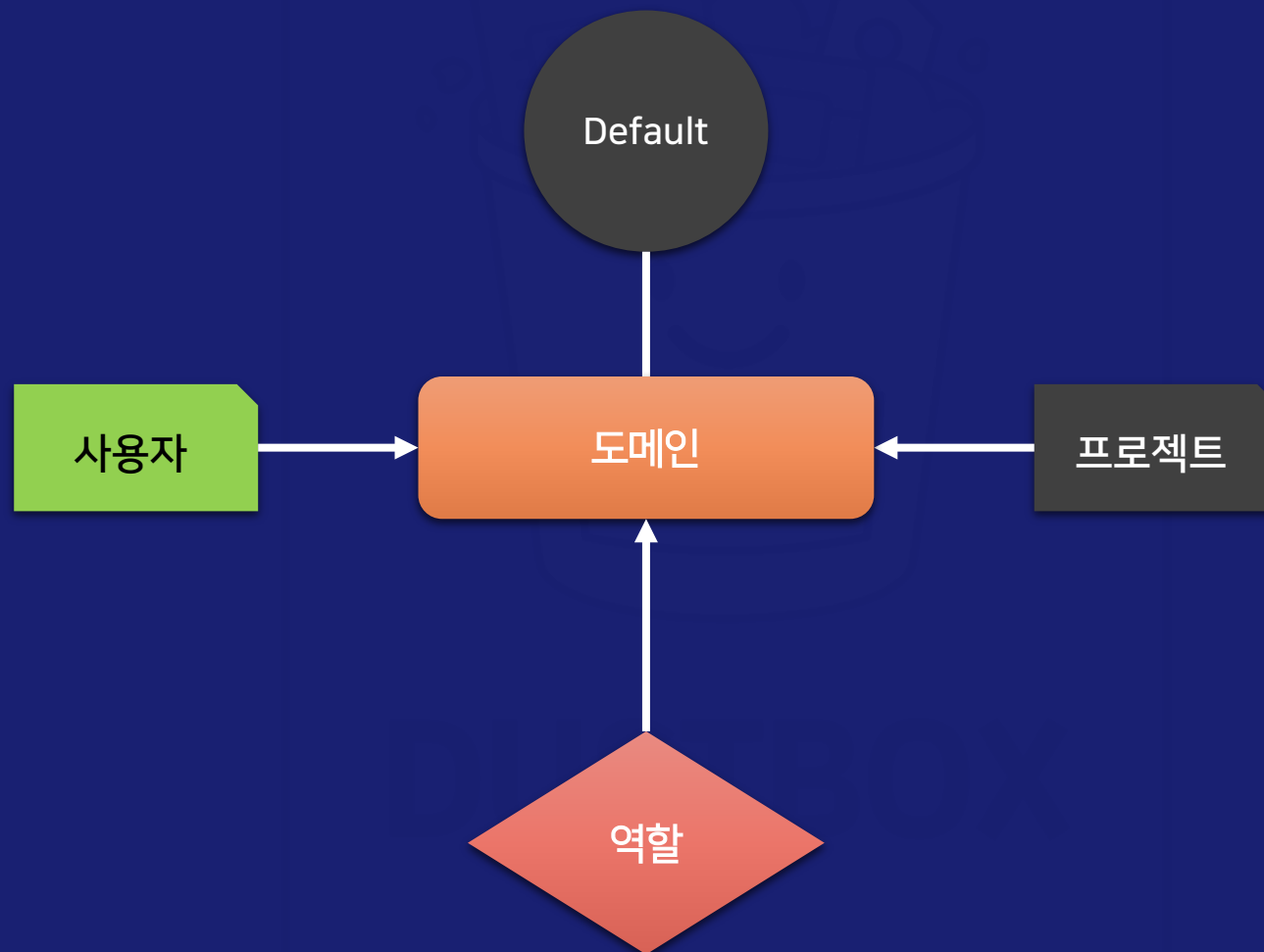


프로젝트 생성

```
# openstack project create --domain vlab-users vlab-project --description  
"VLabs Training Project"  
  
# openstack project list --domain vlab-users
```



역할/사용자 구성



사용자 생성

```
# openstack user create --domain vlab-users --project vlab-project --  
password 'Training123!' vlab-user1  
  
# openstack user list --domain vlab-users
```

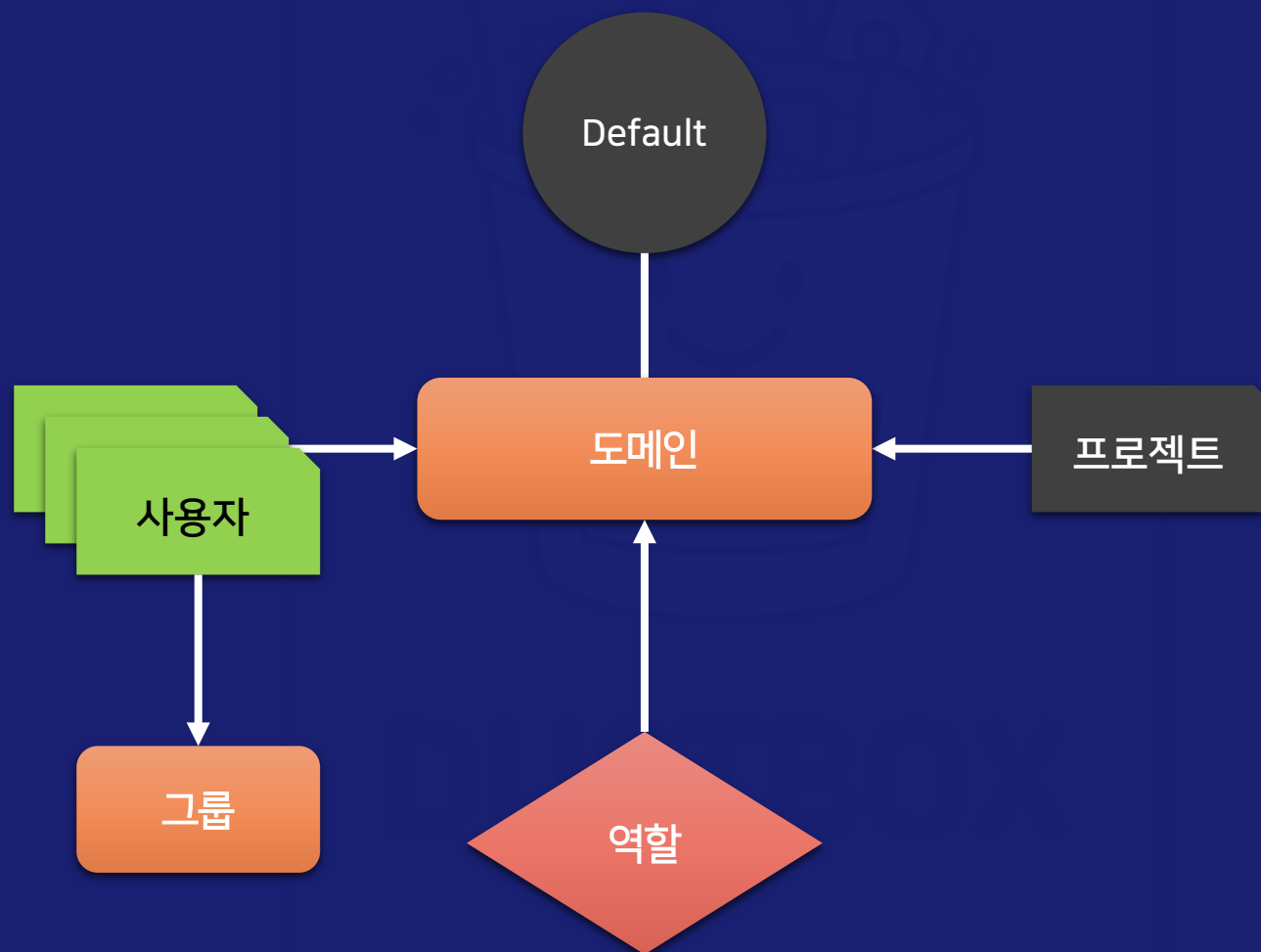


역할 생성

```
# openstack role create trainer
# openstack role create student
# openstack role add --project vlab-project --user vlab-user1 student
# openstack role assignment list --user vlab-user1 --project vlab-project --names
```



그룹 구성



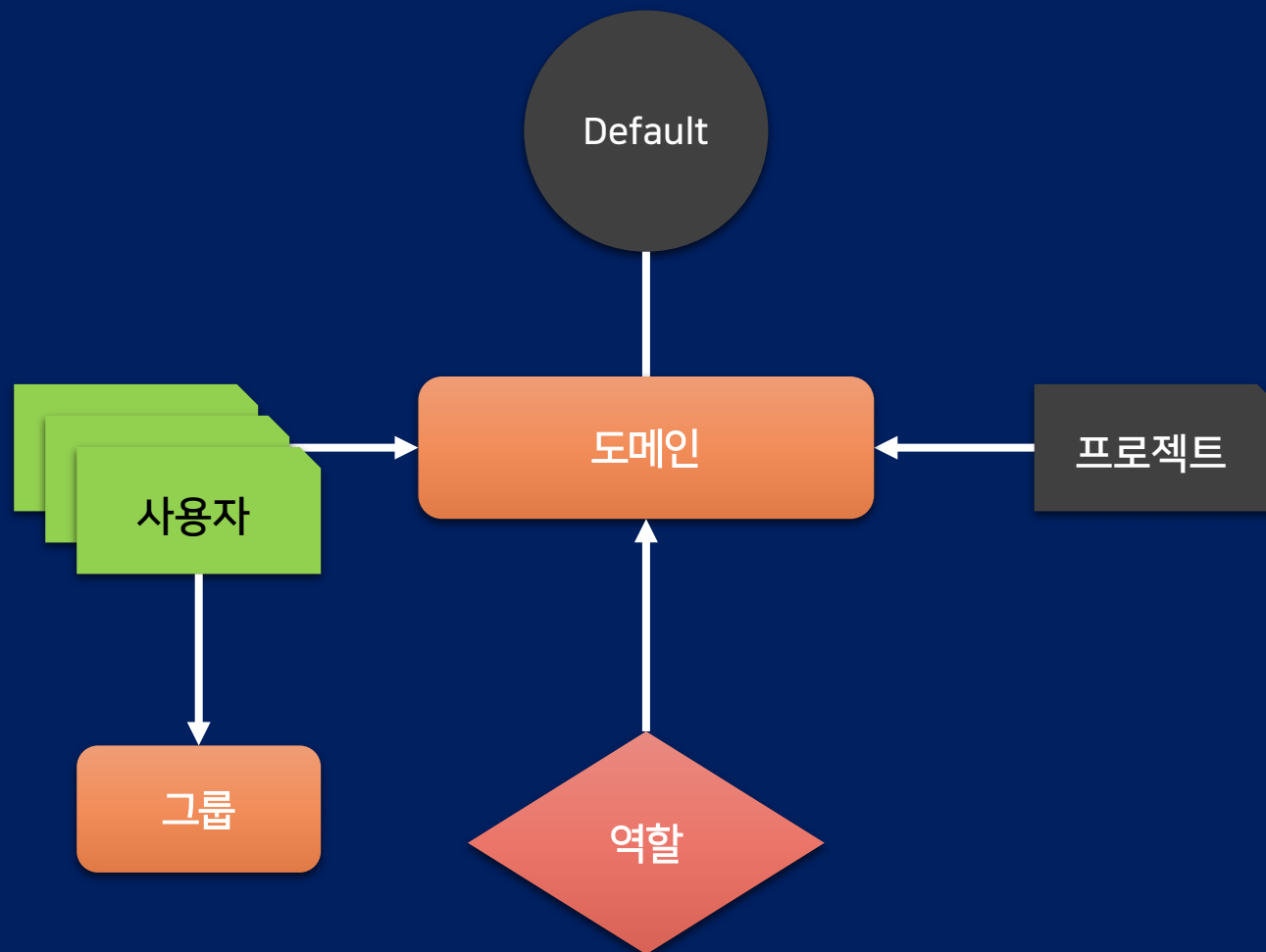
그룹 생성

```
# openstack group create --domain vlab-users training-group --description  
"Training User Group"  
  
# openstack group add user training-group vlab-user1  
  
# openstack group show training-group
```



리전 구성

리전



리전 생성

```
# openstack region create RegionTwo --description "Secondary Region for  
VLabs"
```



서비스 등록

```
# openstack service create --name nova --description "OpenStack Compute Service" compute
```

```
# openstack service create --name neutron --description "OpenStack Networking Service" network
```

```
# openstack service create --name glance --description "OpenStack Image Service" image
```

```
# openstack service list
```



엔드포인트 등록

```
# openstack endpoint create --region RegionOne compute public  
http://controller:8774/v2.1
```

```
# openstack endpoint create --region RegionOne compute internal  
http://controller:8774/v2.1
```

```
# openstack endpoint create --region RegionOne compute admin  
http://controller:8774/v2.1
```



엔드포인트 등록

```
# openstack endpoint create --region RegionOne image public  
http://controller:9292
```

```
# openstack endpoint create --region RegionOne image internal  
http://controller:9292
```

```
# openstack endpoint create --region RegionOne image admin  
http://controller:9292
```



엔드포인트 등록

```
# openstack endpoint create --region RegionOne network public  
http://controller:9696
```

```
# openstack endpoint create --region RegionOne network internal  
http://controller:9696
```

```
# openstack endpoint create --region RegionOne network admin  
http://controller:9696
```

```
# openstack endpoint list
```



도메인 개념 및 설명

여러 사용자가 혹은 그룹이 사용하는 IaaS 플랫폼은 하나의 도메인에서 구성하지 않는다. 도메인 보통 다음과 같은 부분을 도메인으로 일반적으로 분류한다.

주요 적용 범위

Keystone(인증 서비스)의 관리 영역.

포함되는 구성 요소

사용자(User), 그룹(Group), 프로젝트(Project), 역할(Role).

활용 방법

하나의 OpenStack 환경 안에서 여러 조직(예: 회사, 부서, 기관)을 분리하고 싶을 때 사용



도메인 개념 및 설명

각 도메인마다 별도의 관리자 계정(domain admin)을 두어 독립적인 사용자 관리 가능.

Domain A → 국방 교육용 (userA, projectA)

Domain B → 기업 PoC용 (userB, projectB)

서비스 카탈로그(예: Glance, Nova)는 도메인 전체에서 공용으로 접근 가능하게 설정할 수도 있고, 특정 도메인 전용 서비스로도 제한도 가능하다.



전체 명령어 구조

Domain: vlab-users

└─ Roles:

│ └─ trainer

│ └─ student

└─ Groups:

│ └─ trainers → [trainer1]

│ └─ students → [student1, student2]



전체 명령어 구조

```
├─ Projects:
|   ├─ lab-kubernetes
|   |   ├─ trainer1 (trainer)
|   |   ├─ student1 (student)
|   |   └─ student2 (student)
|   ├─ lab-openstack
|   |   ├─ trainers group (trainer)
|   |   └─ students group (student)
|   └─ lab-linux (empty for now)
```



도메인 생성 명령어

```
# openstack domain create vlab-users --description "VLabs Training Domain"  
# openstack domain list
```



역할 생성 명령어

```
# openstack role create --domain vlab-users trainer  
# openstack role create --domain vlab-users student  
# openstack role list --domain vlab-users
```



그룹 생성

```
# openstack group create --domain vlab-users trainers --description  
"Instructor Group"
```

```
# openstack group create --domain vlab-users students --description  
"Student Group"
```

```
# openstack group list --domain vlab-users
```



사용자 생성

트레이너 사용자 생성

```
openstack user create --domain vlab-users --password "Trainer123!" trainer1
```

학생 사용자 생성

```
openstack user create --domain vlab-users --password "Student123!" student1
```

```
openstack user create --domain vlab-users --password "Student123!" student2
```

사용자 목록 확인

```
openstack user list --domain vlab-users
```



그룹 생성

트레이너 그룹에 trainer1 추가

```
openstack group add user trainers trainer1
```

학생 그룹에 student1, student2 추가

```
openstack group add user students student1
```

```
openstack group add user students student2
```

그룹 상세 보기

```
openstack group show trainers
```

```
openstack group show students
```



프로젝트 생성

트레이너 그룹에 trainer1 추가

```
openstack group add user trainers trainer1
```

학생 그룹에 student1, student2 추가

```
openstack group add user students student1
```

```
openstack group add user students student2
```

그룹 상세 보기

```
openstack group show trainers
```

```
openstack group show students
```



역할 할당(사용자)

```
# trainer1에게 lab-kubernetes 프로젝트 trainer 역할 부여  
openstack role add --domain vlab-users --user trainer1 --project lab-  
kubernetes trainer  
  
# student1, student2에게 lab-kubernetes 프로젝트 student 역할 부여  
openstack role add --domain vlab-users --user student1 --project lab-  
kubernetes student  
  
openstack role add --domain vlab-users --user student2 --project lab-  
kubernetes student
```



역할 할당(그룹)

trainers 그룹 전체에 lab-openstack 프로젝트 trainer 역할 부여

```
openstack role add --domain vlab-users --group trainers --project lab-openstack trainer
```

students 그룹 전체에 lab-openstack 프로젝트 student 역할 부여

```
openstack role add --domain vlab-users --group students --project lab-openstack student
```



역할 확인

역할 할당 상태 보기 (이름 기반)

```
openstack role assignment list --names --domain vlab-users
```

특정 사용자 권한 확인

```
openstack role assignment list --user trainer1 --names
```

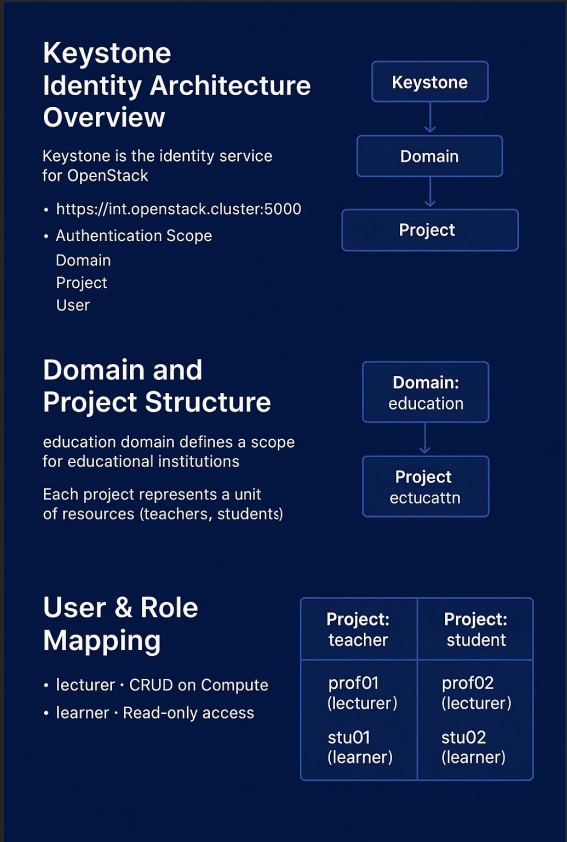
특정 프로젝트 권한 확인

```
openstack role assignment list --project lab-kubernetes --names
```



연습문제

다음과 같이 사용자를 구성한다.



연습문제

이 시스템은 교육자(교수)와 수강생(학생)이 함께 사용하는 구조이며, 각자 권한이 다르게 동작해야 한다.
기관의 인증 구조는 다음 요구사항을 가진다.

1. 교육 기관별로 독립된 도메인(domain)을 구성한다.
 - education : 전체 교육 기관
2. 각 도메인 안에는 프로젝트(project)가 존재한다.
 - teacher 프로젝트 : 교수진 리소스
 - student 프로젝트 : 학생 리소스

연습문제

3. 각 프로젝트에 속한 사용자는 다음과 같다.
 - 교수: prof01, prof02
 - 학생: stu01, stu02
4. 역할(role)은 다음과 같이 구성한다.
 - lecturer: 프로젝트 내 인스턴스 생성 가능
 - learner: 읽기만 가능
5. prof01은 teacher 프로젝트의 lecturer, stu01은 student 프로젝트의 learner로 지정한다.
6. 인증 서버는 내부망 int.openstack.cluster:5000을 사용한다.

연습문제

시나리오에 따라 education이라는 도메인을 생성.

```
# openstack domain create education --description "Education domain"
```

연습문제

도메인 education 안에 teacher와 student 프로젝트를 생성.

```
# openstack project create --domain education teacher --description  
"Teacher project"  
# openstack project create --domain education student --description  
"Student project"
```

연습문제

다음 사용자 4명을 생성. 비밀번호는 모두 edu@1234로 설정.

- prof01, prof02 → teacher 프로젝트
- stu01, stu02 → student 프로젝트

```
# openstack user create --domain education --password edu@1234 prof01
# openstack user create --domain education --password edu@1234 prof02
# openstack user create --domain education --password edu@1234 stu01
# openstack user create --domain education --password edu@1234 stu02
```

연습문제

다음 역할(role)을 생성.

- lecturer
- learner

```
# openstack role create lecturer  
# openstack role create learner
```

연습문제

다음 조건에 맞게 역할을 부여하시오.

- prof01, prof02 → teacher 프로젝트의 lecturer 역할
- stu01, stu02 → student 프로젝트의 learner 역할

```
# openstack role add --user prof01 --project teacher lecturer
# openstack role add --user prof02 --project teacher lecturer
# openstack role add --user stu01 --project student learner
# openstack role add --user stu02 --project student learner
```

연습문제

다음 조건에 맞게 역할을 부여하시오.

- prof01, prof02 → teacher 프로젝트의 lecturer 역할
- stu01, stu02 → student 프로젝트의 learner 역할

```
# openstack role add --user prof01 --project teacher lecturer
# openstack role add --user prof02 --project teacher lecturer
# openstack role add --user stu01 --project student learner
# openstack role add --user stu02 --project student learner
```


연습문제

prof01 계정으로 로그인하여 토큰을 발급받고, 발급된 토큰의 프로젝트 이름을 확인.

```
# source prof01-openrc.sh  
# openstack token issue | grep project_name
```

연습문제

learner 역할이 이미지 목록 조회만 가능하도록 정책을 수정하려면 어떤 파일을 편집해야 하는가?

```
/etc/kolla/keystone/policy.yaml
```

연습문제

도메인 education 내 모든 사용자와 프로젝트를 조회.

```
# openstack user list --domain education  
# openstack project list --domain education
```

Glance

기본 기능

설명

Glance는 OpenStack에서 이미지 관리 서비스를 담당하며, 가상 머신 이미지의 등록, 저장, 검색 및 전송 기능을 제공한다.

기본적으로 Glance는 파일 시스템을 백엔드로 사용하며, 이는 `/var/lib/glance/images/` 디렉터리에 UUID 형식의 파일로 저장된다.

다른 저장 백엔드(예: Swift, Ceph RBD, S3 등)를 사용하는 경우, 해당 스토리지 방식에 맞게 이미지가 저장된다.

Glance는 본래부터 Nova와 별개의 서비스로 존재하며, Nova는 Glance API를 통해 이미지를 불러와 인스턴스를 부트한다.

모든 이미지는 **disk_format**, **container_format** 등을 포함한 메타데이터를 가지며, 이 정보는 이미지의 저장 방식과 부팅 가능 여부를 결정한다.



설명

Glance는 구조적으로 단일 백엔드만 선택적으로 사용하는 방식.

1. **데이터 일관성 유지:** 여러 백엔드에 이미지를 분산 저장하면 동기화 및 정합성 관리가 복잡해짐.
2. **단순한 관리:** 운영자가 스토리지 위치나 방식에 대해 명확히 알고 있어야 하기 때문에, 단일 백엔드가 운영 및 마이그레이션에 유리함.
3. **디자인 철학:** 초기 Glance 설계는 "하나의 이미지 = 하나의 저장소"라는 단일 경로 접근을 따름.
4. **멀티 백엔드 기능은 존재하지만 기본은 아님.**
 - multi_store 기능이 존재하여 여러 백엔드를 선언할 수 있지만, 운영자는 각 이미지를 특정 스토어에 직접 지정해야 하며, 기본적으로 자동 분산은 불가능함.
 - 일반적으로 많이 사용하는 백-엔드는 파일 시스템/NFS와 같은 NAS(Network Attached Storage)와 같은 형식을 사용한다.



Glance 기능 분리

Glance가 Nova에서 다음과 같이 릴리즈 되면서 분리가 되었다.

릴리즈	이미지 관리 방식
Austin	Nova 내부 기능 (이미지 관리 포함)
Bexar	Glance 등장! Nova와 분리된 이미지 서비스
이후	Glance가 Nova와 분리된 채로 표준 이미지 API 제공



Glance가 지원하는 주요 백엔드

Glance에서 지원하는 백-엔드는 다음과 같다.

항목	내용
지원 백엔드	file, http, swift, rbd, cinder(기타: s3, gridfs, sheepdog 등 deprecated)
기본 백엔드	file (경로: /var/lib/glance/images)
설정 위치	glance-api.conf의 default_store 항목
멀티 백엔드 지원	enabled_backends 및 stores 설정으로 가능(예: default_store = fast-ceph, enabled_backends = fast-ceph:rbd, backup-swift:swift)
멀티 백엔드 제한 이유	▪ 이미지 정합성 및 동기화 문제 발생 가능성 ▪ 관리 복잡성 증가 ▪ 설계 철학: "1 이미지 = 1 저장소" ▪ 자동 분산 불가, 직접 저장소 지정 필요



Glance가 지원하는 가상머신 이미지 유형

Glance에서 다음과 같은 이미지들을 지원 및 제공한다.

이미지 유형	설명	주요 사용처
QCOW2	QEMU Copy-on-Write. 스냅샷, 압축, 공간 절약 기능 지원	KVM 기반 가상머신 (주로 사용됨)
Raw	비압축 순수 이미지. 가장 단순함	성능 우선 환경, Ceph RBD 등
VHD/VHDX	Microsoft Hyper-V 이미지 형식	이종 플랫폼 간 이전 목적
VMDK	VMware의 가상 디스크 포맷	VMware에서 생성된 이미지 호환
ISO	부팅 가능한 설치 이미지	OS 설치용
AMI/AKI/ARI	EC2 호환 이미지 포맷	클라우드 호환 환경
Docker	OCI 기반 컨테이너 이미지 지원 (Glance + Glare 확장 필요 시)	컨테이너 오케스트레이션 연동 목적



VMDK가 권장되지 않는 이유

VMDK사용은 가능하나, 올바르게 기능 및 성능을 기대할 수 없다.

이유	설명
변환 필요	KVM, QEMU 등에서 직접 사용 불가. QCOW2나 RAW로 변환해야 함
호환성 문제	VMDK는 VMware 전용 포맷으로, OpenStack에서의 기능 (스냅샷, resize 등)과 충돌 우려
스토리지 최적화 미지원	Ceph, Cinder 등 백엔드 스토리지에서 비효율적
보안 위험	VMDK 내부에 포함된 메타데이터 구조가 표준화되지 않아 보안 취약점 가능성 존재
운영 복잡성	여러 하이퍼바이저 포맷 지원 시 관리 부담 증가



컨테이너 프로그램

Glance는 호스트에서 더 이상 실행하지는 않는다. 여전히 호스트에서 사용이 가능하나, 업그레이드 및 업데이트를 안전하게 하기 위해서 컨테이너 기반으로 사용을 권장한다.

`docker`, `podman`으로 사용이 가능하며 다음 명령어로 동작중인 런타임 컨테이너 확인이 가능하다.

```
docker ps | grep glance  
podman ps | grep glance
```

```
docker volume ls
```



QCOW2/3 vs RAW: 성능 비교

RAW 이미지 형식은 압축이나 메타데이터 구조 없이 순수한 디스크 블록만을 포함하는 가장 단순한 형식. 반면, QCOW2/3는 스냅샷, 압축, 암호화, 백링크 기능을 포함하는 고급 기능 지원 이미지 포맷.

1. 성능 (I/O throughput 및 latency)

- RAW 형식은 오버헤드가 전혀 없어, 디스크 I/O 성능 측면에서 가장 빠름. 특히, Ceph RBD나 직접 블록 디바이스에 연결된 스토리지에서 그 차이는 더 두드러짐.
- QCOW2/3는 내부 메타데이터 구조로 인해 디스크 접근 시마다 주소 변환, 압축 해제, 스냅샷 관리 작업이 동반되어 쓰기 성능이 저하. 특히 랜덤 I/O 환경에서는 latency 증가.



QCOW2/3 vs RAW: 성능 비교

2. 공간 효율

- QCOW2/3는 **sparse allocation**과 **압축** 기능 덕분에 실제 디스크 공간을 절약할 수 있다. 작은 테스트 VM이 나 일시적인 이미지에 유리.
- RAW는 항상 **전체 용량**을 미리 할당해야 하기 때문에 디스크를 더 많이 소모.

3. 기능성

- QCOW2/3는 스냅샷, 백링크 이미지 체인 생성 등 가상화 환경에서 필요한 다양한 기능을 제공.
- RAW는 단순하지만 기능이 제한되어 있으며, snapshot은 외부 도구(KVM snapshot, Ceph snapshot 등)에 의존.



무엇이 더 나은 형태인가?

Ceph 기반이거나 혹은 EMC와 같은 SAN 장치를 사용하는 경우, QCOW2보다는 RAW사용하는게 더 효율적이고 더 빠르다. QCOW2는 외부 스토리지가 없거나 혹은 NAS형태로 구성하는 경우, QCOW2로 구성하는 더 빠른 프로 비저닝 속도를 기대할 수 있다.

항목	RAW	QCOW2/3
I/O 성능	★★★★★ (매우 우수)	★★ (메타데이터로 인한 오버헤드)
공간 효율	★ (전체 공간 할당)	★★★★ (sparse + 압축 지원)
스냅샷	외부 의존	자체 지원
VM 부팅 속도	빠름	느림 (특히 첫 부팅 시)



백엔드 추가 설명

"Ceph 기반이거나 혹은 EMC와 같은 SAN 장치를 사용하는 경우, QCOW2보다는 RAW 사용하는 게 더 효율적이고 더 빠르다."

이유:

1. Ceph RBD, EMC VMAX/VNX, NetApp 등 블록 스토리지 기반 시스템은 RAW 이미지와 직접 매핑 가능
2. QCOW2는 로컬 파일 기반의 레이어드 포맷이기 때문에, 블록 스토리지 위에 다시 파일시스템 레벨이 올라가면 중복 I/O 및 latency가 발생
3. 특히 Ceph의 경우, Glance에서 rbd 백엔드를 사용하는 경우 RAW 이미지는 RBD object로 바로 전송되고, VM에서 그대로 attach되므로 zero-copy 방식에 가까운 효율



백엔드 추가 설명

"QCOW2는 외부 스토리지가 없거나 혹은 NAS형태로 구성하는 경우, QCOW2로 구성하면 더 빠른 프로비저닝 속도를 기대할 수 있다."

1. NAS(Network Attached Storage) 또는 로컬 디스크에 이미지 저장하는 구조.
 - QCOW2는 sparse allocation 덕분에 이미지를 빠르게 복제(clone)
 - copy-on-write 방식으로 VM 프로비저닝 시간이 단축
2. Glance + Nova + libvirt의 QCOW2 snapshot chain을 활용하면 빠른 VM 생성 가능
3. I/O가 많은 VM이 다수 실행되면 QCOW2는 RAW보다 성능이 저하됨
 - 즉, 빠른 초기 프로비저닝은 QCOW2가 유리하지만, 운영 중 성능은 RAW가 우세



백엔드 정리

위의 내용을 정리하면 다음과 같다.

환경	권장 이미지 포맷	이유
Ceph / SAN (EMC 등)	RAW	블록 스토리지에 직접 매핑, I/O 효율 및 zero-copy 가능
NAS / 로컬 디스크 기반	QCOW2	빠른 clone, thin provisioning, 적은 초기 공간 사용
성능 위주 장기 운영	RAW	지속적인 랜덤 I/O, 낮은 latency
프로비저닝 속도 위주	QCOW2	빠른 VM 배포 및 초기 실행



명령어 사용 방법

openstack에서 이미지 관리를 위한 명령어는 다음과 같이 사용한다.

```
# openstack image <subcommand> [options]
```



이미지 목록

이미지 목록을 확인 하기 위해서 다음과 같이 한다.

```
# openstack image list
```



이미지 정보 확인

이미지 정보를 자세히 확인하기 위해서 아래와 같이 실행한다.

```
# openstack image show <IMAGE_NAME_OR_ID>
# openstack image show rocky9-base
```



로컬 이미지 업로드

이미지 업로드는 다음과 같다. create 명령어는 이미지 생성이 아니라 업로드이다!

```
# openstack image create "rocky9-base" \  
  --disk-format qcow2 \  
  --container-format bare \  
  --file /var/images/rocky9.qcow2 \  
  --public
```

옵션	설명
--disk-format qcow2	이미지 파일 형식 (qcow2, raw, vmdk, vdi, iso 등)
--container-format bare	추가 메타데이터 없는 일반 이미지
--file	로컬 경로 지정
--public	모든 프로젝트에서 접근 가능
--protected	관리자 외 삭제 불가 (선택)



로컬 이미지 업로드(특정 프로젝트)

이미지를 비공개로 생성 후, 특정 프로젝트에 사용을 원하는 경우 아래와 같이 구성이 가능하다.

```
# 비공개로 생성
# openstack image create "rhel9-internal" \
  --file /var/images/rhel9.qcow2 \
  --disk-format qcow2 \
  --container-format bare

# 특정 프로젝트에 접근 허용
# openstack image add project rhel9-internal <PROJECT_ID>
```



원격에서 이미지 업로드

원격에서 이미지를 다운로드 및 업로드 가능하다.

```
# openstack image create "ubuntu-cloud" \  
  --disk-format qcow2 \  
  --container-format bare \  
  --location "https://cloud-images.ubuntu.com/noble/current/noble-server-  
cloudimg-amd64.img" \  
  --public
```



이미지 수정

이미지 보호가 걸려 있는 경우, 제거가 되지 않는다.

```
# openstack image set --unprotected rocky9-base
# openstack image delete rocky9-base

# openstack image set --property os_distro='rocky' \
--property hw_disk_bus='virtio' rocky9-base
```



이미지 다운로드

이미지를 로컬로 다운로드 받기 위해서 아래와 같이 실행한다.

```
# openstack image save --file rocky9.qcow2 rocky9-base
```



명령어 정리

최종적으로 이미지 관리 명령어는 아래와 같은 단계로 많이 사용한다.

1. qcow2 이미지 업로드

```
openstack image create "rocky9-base" \  
  --disk-format qcow2 \  
  --container-format bare \  
  --file /srv/images/rocky9.qcow2 \  
  --public
```

2. 속성 추가

```
openstack image set rocky9-base \  
  --property os_distro='rocky' \  
  --property hw_disk_bus='virtio'
```

3. 확인

```
openstack image show rocky9-base -f yaml
```



컨테이너 레지스트리 서버

Glance

컨테이너 레지스트리

내부 서비스에 컨테이너 이미지를 배포 및 관리하기 위해서 반드시, 최소 한 개의 레지스트리 서버가 필요하다. 일반적으로 사용하는 소프트웨어는 두 가지가 있다.

1. `docker-registry`

2. Harbor

3. Gitlab

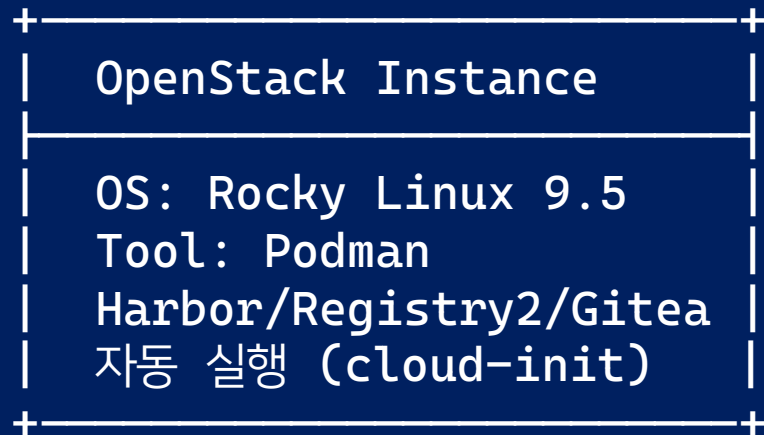
`docker-registry`는 가볍지만, 명령어 위주로 사용해야 되기 때문에, 일반적인 사용자 및 개발자는 어렵다. 이러한 이유로 많은 기업 혹은 단체는 harbor, Gitlab를 선호한다. 다만, Gitlab은 자원을 많이 사용하기 때문에 레지스트리 서버 목적으로는 Harbor를 권장한다.

기초 과정에서는 해당 부분에 대해서 구축은 필요하지 않는다.



컨테이너 레지스트리

구성 아키텍처는 다음과 같다.



|
(expose HTTP/HTTPS)

↓

Browser/User → [http\(s\)://harbor.vlab.dustbox.kr](http(s)://harbor.vlab.dustbox.kr)



인스턴스 기반 레지스트리 서버 구성

인스턴스 기반으로 레지스트리 서버를 구성하기 위해서 아래와 같이 명령어를 실행한다. 다만, 여기에서는 레지스트리 서버는 직접 구성하지는 않는다. 아래 내용은 cloud-init Customization Script에 추가한다.

```
# curl -fsSL http://10.0.0.250/deploy_gitea.sh | bash
```

자동으로 gitea가 자동으로 설치가 완료가 된다.



이미지 빌드

Glance

이미지 빌드

가상머신에 사용하는 이미지는 보통, 각 회사나 커뮤니티에서 구성 후 배포한다. 하지만, 해당 이미지는 표준적인 이미지로 구성이 되어 있기 때문에, 서비스 혹은 회사에서 사용하는 용도와 맞지 않는 경우가 있다.

이러한 이유로 오픈스택 사용자는 별도로 이미지를 구성 및 배포하는 경우가 많다. 여기서, 많이 사용하는 두 가지 이미지 빌드 도구를 가지고 가상머신 이미지 구성에 대해서 다루도록 한다.

1. **disk-image-builder**(많이 사용하는 도구)

2. **oz**(사용하기 편함)

3. **packer**(테라폼 기반)

3번 packer는 오픈소스 저장소에는 등록이 안되어 있기 때문에, 3번은 제외하고 1/2번 기준으로 디스크 이미지 빌드 방식에 대해서 설명한다. 여기서는 자세히 설명하지 않는다. 추가적인 내용은 아래 사이트에서 확인한다.

<https://github.com/openstack/diskimage-builder>



이미지 빌드(diskimage-builder)

현재 모든 가상머신 시스템은 diskimage-builder를 많이 사용한다. 구성 및 사용하기 위해서 아래와 같이 설정한다.

```
# dnf install -y python3-pip
# pip install diskimage-builder
export DIB_RELEASE=9
export DISTRO_NAME=rocky
export ELEMENTS_PATH=elements

# vi elements/rocky/root.d/50-enable-agent
#!/bin/bash
set -eux
dnf install -y qemu-guest-agent cloud-init
# disk-image-create rocky vm cloud-init -o rocky9-cloudimg
```



diskimage-builder 옵션

위에서 사용한 옵션은 다음과 같다. 배포판은 언제든지 변경이 가능하다.

여기서는 Rocky Linux 9으로 구성하였다. 오픈스택 및 대다수 IaaS를 지원하는 플랫폼들은 cloud-init를 사용하기 때문에, 부팅 시 활성화 되도록 구성한다.

옵션	설명
rocky	OS 베이스
vm	가상화용 이미지
cloud-init	cloud-init 활성화
-o	출력 파일명 지정 (rocky9-cloudimg.qcow2 생성됨)



oz로 Rocky 이미지 빌드하기

oz를 사용하는 경우, 아래와 같이 **템플릿(template)**를 생성 후, 부팅을 시작하여 이미지 빌드를 시작한다. 이 방식은 시간이 오래 걸리지만, 제일 안전하고 깔끔한 이미지를 생성 및 구성하는 장점이 있다.

```
# vi rocky.tdl
<template>
  <name>rocky9</name>
  <os>
    <name>Rocky</name>
    <version>9</version>
    <arch>x86_64</arch>
    <install type='url'>
      <url>http://mirror.rockylinux.org/rocky/9/BaseOS/x86_64/os/</url>
    </install>
```



oz로 Rocky 이미지 빌드하기

위의 내용 계속 이어서...

```
<rootpw>rockypass</rootpw>
</os>
<packages>
  <package name="cloud-init" />
  <package name="qemu-guest-agent" />
</packages>
</template>
# dnf install oz
# oz-install -d3 -t /etc/oz/templates/rocky-9.tdl
# openstack image create "rocky9-custom" \
--file rocky9-cloudimg.qcow2 --disk-format qcow2 --container-format bare \
--public
```



앤서블 예제

만약, 이미지를 자동으로 Glance에 배포하기 위해서 아래와 같이 앤서블 작성 후, Glance에 배포가 가능하다.

```
- name: Upload image to Glance
  openstack.cloud.image:
    name: rocky-9.5
    disk_format: qcow2
    container_format: bare
    filename: /root/rocky-server.qcow2
    is_public: true
    state: present
```



연습문제

Cinder

기본 기능

Cinder 설명

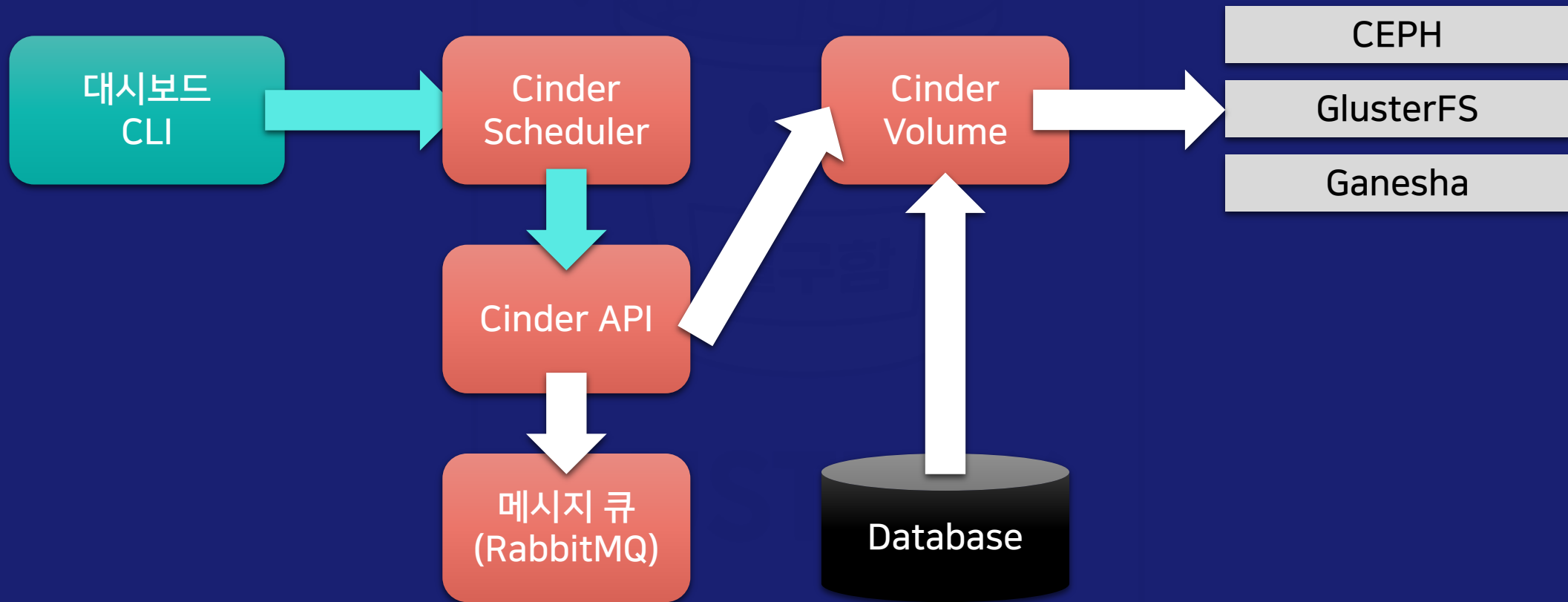
Cinder는 OpenStack에서 블록 스토리지 서비스를 제공하는 컴포넌트로, 가상 머신 인스턴스에 디스크 형태의 볼륨을 생성, 첨부, 관리할 수 있도록 지원한다.

초기에는 nova-volume이라는 Nova의 내부 컴포넌트였지만, 2012년 Folsom 릴리즈부터 Cinder라는 별도 서비스로 분리되었다.

Cinder는 다양한 백엔드 드라이버를 지원하며, Ceph RBD/LVM/NFS/NetApp/iSCSI등의 스토리지 솔루션과 연동이 가능하다.



Cinder 아키텍처



Cinder 설명

Cinder는 다음과 같은 주요 기능을 제공한다.

1. 볼륨 생성, 삭제 및 연결/분리
2. 스냅샷 및 볼륨 복제
3. 볼륨 백업/복원
4. 이기종 간 스토리지 마이그레이션
5. 공유 볼륨(multi-attach) 기능



Cinder의 주요 기능

1. 인스턴스와 분리된 블록 스토리지
2. 다중 백-엔드 지원(Multi Backend)
3. 스냅샷 기반 복제 및 백업
4. 보안 그룹 및 액세스 제어 연동
5. API를 통한 자동화된 볼륨 관리



랩 에서 사용하는 백엔드

랩에서는 Ceph/GlusterFS와 같은 백 엔드 구성 및 활용하지 않는다. 많은 자원 및 시간이 발생하기 때문에, 추후에 고급과정에서 다룬다. 랩에서는 NFS(Ganesha)기반으로 사용한다.

일반적으로 오픈소스 버전으로 오픈스택을 구성하는 경우 스토리지 백-엔드는 다음과 같이 구성한다.

종류	백엔드 프로그램	Cinder 연동 여부	특징 및 비고
블록	LVM2	공식 드라이버 있음	가장 간단한 로컬 디스크 백엔드
블록	DRBD	드라이버 있음	고가용성 구성
블록	ZFS zvol	비공식/커스텀	zvol로 블록 제공
블록	GlusterFS	공식 드라이버	Gluster 블록 또는 NFS 형태
블록	NFS (Ganesha 등)	공식 드라이버	NFS 공유 디렉터리 기반
블록	Sheepdog	비활성화	유지보수 중단 주의



비공식 백엔드

아래 스토리지는 사용은 가능하나, 공식적인 Cinder 드라이버가 없다. 이러한 이유로 직접 인스턴스와 연결 및 구성해야 한다.

종류	프로젝트명	Cinder 연동 여부	특징 및 비고
오브젝트	Ceph (RBD)	공식 지원	가장 널리 사용됨
오브젝트	MinIO	커스텀 드라이버 가능	S3 API 기반, 중간 계층 필요
오브젝트	OpenIO, Swift, Walrus		Glance에서 Swift 사용 가능
파일	MooseFS	NFS 통해 간접 연동 가능	FUSE 또는 NFS를 통해 접근 가능
파일	Lustre, BeeGFS	-	HPC 환경에 주로 사용, 직접 연동 없음



프로젝트에서 많이 사용하는 백엔드

주로 많이 사용하는 백-엔드는 다음과 같다.

백엔드	공식 지원	유형	특징
LVM2	네	블록	단일 노드, 테스트에 적합
Ceph RBD	네	블록	확장성, 성능 우수, 가장 많이 사용됨
NFS	네	파일	NAS 기반, 간편 구성
GlusterFS	네	파일	분산 파일 기반, 중소규모 적합
NetApp/EMC 등	네	블록/파일	기업용 상용 스토리지 연동



더 줄이면...

더 범주 및 범위를 줄이면 아래와 같이 백엔드를 많이 사용합니다.

환경	권장 백엔드	이유
대규모, 고성능	Ceph RBD	고가용성, 분산 구조
테스트/개발	LVM	구성 간단
NAS 기반 환경	NFS	외부 NFS 서버 연동
전용 파일 스토리지	GlusterFS	가볍고 유연한 분산 파일
엔터프라이즈 환경	NetApp 등	고성능 SAN/NAS 장비 연동



NFS서버 유형

대다수 서버들은 아직도 NFS 3/4 서버 기반으로 구성한다. 기

존의 NFS를 사용하여도 상관 없지만, 다중 연결 및 보안을 위해서 Ganesha NFS 서버 사용을 권장한다. 레드햇 계열은 8버전부터 지원하고 있다.

구분	커널 NFS (nfs-server)	NFS-Ganesha (user-space NFS)
동작 계층	커널 공간(Kernel Space)	사용자 공간(User Space)
프로세스 구성	nfsd, rpc.mountd, rpcbind	nfs-ganesha 단일 데몬
구성 파일 위치	/etc/exports	/etc/ganesha/ganesha.conf
지원 FS 유형	로컬 파일시스템만 (ext4, xfs 등)	VFS, Ceph, Gluster, RGW, S3 등
확장성 / 성능	고성능, 커널 I/O 직접 접근	유연성 높음, 성능은 약간 낮음 (user-space 오버헤드)
서비스 재시작	기존 세션 끊김 가능성 높음	설정 Reload 지원(systemctl reload nfs-ganesha)



NFS서버 유형

위의 내용 계속 이어서...

구분	커널 NFS (nfs-server)	NFS-Ganesha (user-space NFS)
NFS 프로토콜 지원	v2, v3, v4	v3, v4, pNFS
보안 및 인증	Kerberos, AUTH_SYS	Kerberos, LDAP, RPCSEC_GSS 등 확장 가능
컨테이너/클러스터 환경 적합성	제한적 (host 기반)	컨테이너/HA 클러스터 친화적
구성 난이도	단순 (/etc/exports)	복잡하지만 세밀한 제어 가능
대표 사용 예	단순 NAS, 로컬 공유	Ceph, Cinder, GlusterFS 백엔드, 클라우드 인프라 (OpenStack 등)



GANESHA 서버 설치 및 구성

패키지 설치 방법만 다르고 수세/데비안/우분투 계열에서 동일하게 설치 및 사용이 가능하다.

```
# dnf -y install nfs-ganesha nfs-ganesha-vfs nfs-utils policycoreutils-python-utils

# mkdir -p /srv/nfs/cinder
# chown -R root:root /srv/nfs/cinder
# chmod 0777 /srv/nfs/cinder # (테스트용 권한 / 운영 시 필요 최소 권한으로 조정)

# semanage fcontext -a -t public_content_rw_t "/srv/nfs/cinder(/.*)?"
# restorecon -Rv /srv/nfs/cinder
# setsebool -P nfs_export_all_rw on
# cp -a /etc/ganesha/ganesha.conf /etc/ganesha/ganesha.conf.bak.$(date +%F)
# vi /etc/ganesha/ganesha.conf
NFS_Core_Param {
    NFS_Protocols = 3,4;
    Bind_Protocol = TCP;
}
```



GANESHA 서버 설치

외부로 노출할 설정 구성.

```
EXPORT_DEFAULTS {
    Access_Type = RW;
    Squash = No_Root_Squash;
    Transports = TCP;
    SecType = sys;
}
EXPORT {
    Export_Id = 101;
    Path = /srv/nfs/cinder;           # 실제 경로
    Pseudo = /cinder;                # NFSv4 가상 경로
    FSAL { Name = VFS; }
    Protocols = 3,4;
}
```



GANESHA 서버 설치

위의 내용 계속 이어서...

```
# systemctl enable --now rpcbind
# firewall-cmd --add-service=nfs --permanent
# firewall-cmd --add-service=mountd --permanent
# firewall-cmd --add-service=rpc-bind --permanent
# firewall-cmd --reload

# systemctl enable --now nfs-ganesha

# systemctl reload nfs-ganesha
```



백엔드 구성

기존에 구성이 되어 있는 cinder-volume에 백엔드 추가한다.

```
# vi /etc/kolla/config/cinder-volume/cinder.conf
[DEFAULT]
enabled_backends = nfs
[nfs]
volume_driver = cinder.volume.drivers.nfs.NfsDriver
volume_backend_name = CINDER_NFS
nfs_shares_config = /etc/cinder/nfs_shares
nfs_mount_point_base = /var/lib/cinder/mnt
nfs_sparsed_volumes = true
nfs_mount_options = vers=3
nas_secure_file_operations = false
```



백엔드 구성

NFS 서버 정보를 cinder설정에 같이 포함하여 배포한다.

```
# vi /etc/kolla/config/cinder-volume/nfs_shares
192.168.1.10:/srv/nfs/cinder
# chown cinder:cinder
# chmod 0640 /etc/cinder/nfs_shares
# kolla-ansible reconfigure -i inventory -t cinder
```



백업 백엔드 구성

백업용으로 사용할 백엔드를 구성하기 위해서 아래와 같이 구성한다.

```
# vi /etc/kolla/config/cinder-backup/cinder.conf
[DEFAULT]
backup_driver = cinder.backup.drivers.nfs
backup_share = 192.168.1.10:/backup
backup_mount_point_base = /var/lib/cinder/backup_mount
# kolla-ansible -i /etc/kolla/inventory deploy --tags cinder-backup
```



백엔드 비교

백업 볼륨 유형을 최종적으로 정리하면 다음과 같다.

시나리오	사용 기능	비고
OS 업그레이드 전 롤백 대비	Snapshot	빠른 시점 복원
주기적 데이터 보호	Backup	증분/전체 백업 가능
다른 존으로 복제	Backup + Restore	지역 간 전송 가능
테스트 환경 복제	Snapshot + Volume Create	빠른 복제 생성



볼륨 생성 및 구성

이미지를 로컬로 다운로드 받기 위해서 아래와 같이 실행한다.

```
# openstack image save --file rocky9.qcow2 rocky9-base
```



볼륨 백업 방법

아래 명령어로 백업 생성이 가능하다.

1. 일반백업
2. 증분백업

```
# openstack volume backup create --volume <볼륨ID> --name <백업이름> --  
incremental <true|false>
```



볼륨 백업 생성

일반 및 증분 백업은 아래와 같이 구성이 가능하다.

```
# openstack volume backup create --volume test-volume --name backup-  
20251012 --description "daily backup"  
# openstack volume backup create --volume test-volume --incremental true --  
name inc-backup-1
```



볼륨 백업 목록

백업 구성이 완료가 되면, 아래 명령어로 확인이 가능하다.

```
# openstack volume backup list  
# openstack volume backup show <백업ID>
```



볼륨 백업 복구

기존 백업 내용을 복구하기 위해서 아래와 같이 명령어를 실행한다.

1. 기존 볼륨 복구
2. 새로운 볼륨 복구

```
# openstack volume backup restore --volume <대상볼륨ID> <백업ID>  
# openstack volume create --backup <백업ID> <새볼륨이름>
```



볼륨 백업 삭제

생성된 백업 볼륨을 삭제 하기 위해서 아래와 같이 실행한다.

```
# openstack volume backup delete <백업ID>
```



부팅 볼륨 생성(O/S 이미지 기반)

가상머신 이미지를 부팅 가능한 볼륨 형태로 구성이 필요한 경우, 아래와 같이 실행한다. 이 방식을 사용하면 부팅 속도 및 고성능 IOPS를 보장하며, 또한 컴퓨트 노드의 워크로드를 줄여 주기도 한다.

```
# openstack volume create --size 10 --image ubuntu-22.04 --bootable ubuntu-volume
# openstack volume show my-volume
# openstack server add volume my-instance my-volume
# openstack server remove volume my-instance my-volume
# openstack volume snapshot create --volume my-volume snapshot-001
# openstack volume create --snapshot snapshot-001 --size 10 cloned-volume
# openstack volume delete my-volume
```



볼륨 암호화

여기서는 볼륨 형식을 추가한다. 다만, 추가 시, 디스크 Lukus암호화를 활성화 한다.

```
# openstack volume type create ssd-type
# openstack volume type set --property volume_backend_name=ssd ssd-type
# openstack volume type create secure-type
# openstack volume type encryption create --cipher aes-xts-plain64 \
  --control-location front-end --provider luks --key-size 256 secure-type
# openstack server show my-instance -f json | jq '.os-extended-
volumes:volumes_attached'
```



백엔드 추가

Cinder

BACKEND 추가

더 많은 백-엔드를 추가하면 좋겠지만, 리소스 문제로 간단하게 LVM2의 백-엔드를 구성한다. 백-엔드는 storage 서버에 구성한다. 여기서는 storage instance에 구성된 /dev/vdb1를 사용하여 LVM2를 구성한다.

만약, 블록 장치가 구성이 안된 사용자는 storage서버에서 아래와 같이 명령어를 실행한다.

```
# lsblk  
# pvcreate /dev/vdb1  
# vgcreate cinder-volumes /dev/vdb1
```



LVM2 BACKEND CONFIG

LVM2 기반으로 백엔드 드라이버 구성한다. `cinder.conf`에 다음과 같이 설정한다.

설정 파일 위치는 deployment에 `/etc/kolla/config/cinder.conf`에 아래와 같이 작성한다.

```
[DEFAULT]
```

```
enabled_backends = nfs,lvm
```

```
[lvm]
```

```
volume_driver = cinder.volume.drivers.lvm.LVMVolumeDriver
```

```
volume_backend_name = LVM
```

```
volume_group = cinder-volumes
```

```
iscsi_protocol = iscsi
```

```
iscsi_helper = lioadm
```



BACKEND CONFIG

다시, cinder container에 설정 파일 적용 및 재시작을 수행한다.

```
# kolla-ansible reconfigure -t cinder -i multimode
```



볼륨 타입 생성

아래 명령어로 오픈스택에서 사용이 가능하도록 볼륨 타입을 생성한다.

```
# openstack volume type create lvm-type  
# openstack volume type set --property volume_backend_name=LVM lvm-type
```

생성이 완료가 되면, 다음처럼 전체적으로 자원을 확인한다.

```
# vgs  
# lvs  
# openstack volume create --type lvm-type --size 1 test-volume
```



앤서블 플레이북 활용

앤서블 기반으로 볼륨 디스크를 생성 시 다음과 같이 작성 및 구성한다.

- name: Create and attach volume in OpenStack
 - hosts: localhost
 - tasks:
 - name: Create volume
 - openstack.cloud.volume:
 - state: present
 - display_name: my-volume
 - size: 10
 - name: Attach volume to instance
 - openstack.cloud.server_volume:
 - server: my-instance
 - volume: my-volume
 - state: present



연습문제

Neutron

기본 기능

Neutron 소개

Neutron은 OpenStack에서 네트워크 서비스를 담당하는 컴포넌트로, 초기에는 Quantum이라는 이름으로 출시되었으나 상표권 문제로 인해 현재의 이름으로 변경되었다.

초기에는 **Linux Bridge**와 **iptables**를 기반으로 네트워크를 구성하였지만, 현재는 **Open vSwitch(OVS)** 및 **OVN** 기반 구성이 권장되며, 더 높은 유연성과 확장성을 제공한다.

Neutron은 API를 통해 네트워크, 서브넷, 포트, 라우터, 보안 그룹 등의 리소스를 정의하고 제어하며, 실제 패킷 흐름은 **OVS/OVN이 L2~L4** 계층에서 처리한다.



Neutron 소개

각 인스턴스의 가상 인터페이스는 네트워크 네임스페이스와 브릿지를 연결하기 위해 veth pair로 구성되며, 이는 모든 드라이버 공통 구조이다.

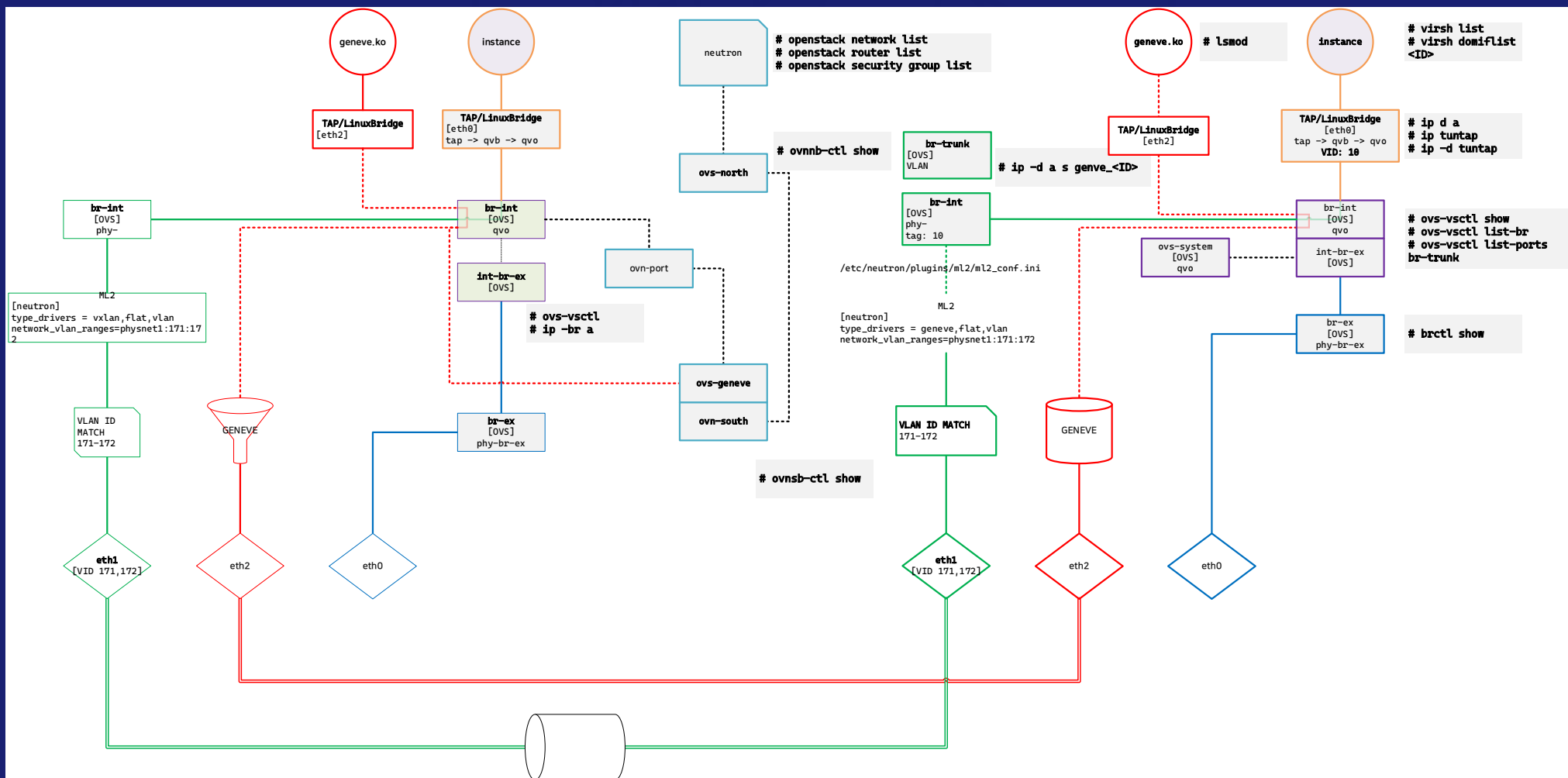
OVN은 고급 SDN 컨트롤 플레인으로 NAT, ACL, DHCP 등 다양한 기능을 포함하며, DPDK나 SR-IOV 등의 기술을 활용하여 성능 저하 없이도 고성능 처리가 가능하도록 설계되어 있다.

오픈스택은 다음과 같은 기술로 네트워크 기능을 제공하고 있다.

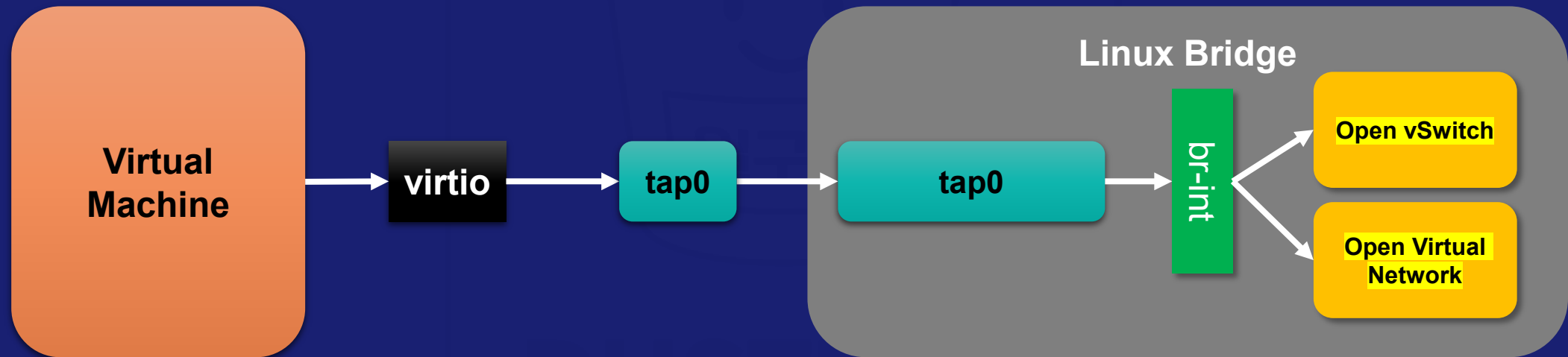
1. Linux Bridge
2. Open vSwitch
3. Open Virtual Network



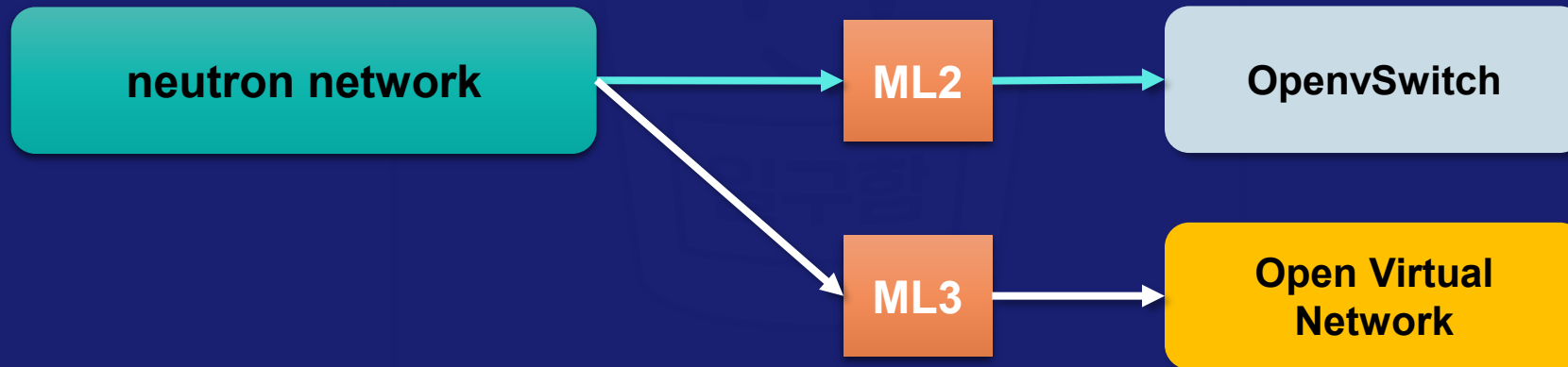
Neutron 아키텍처 설명



Neutron Architecture

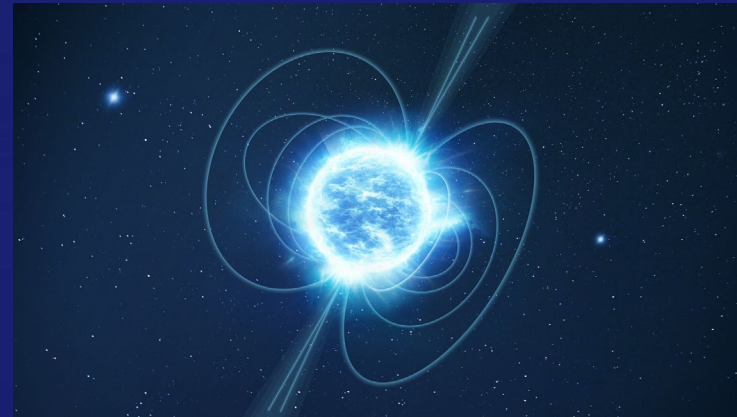


Neutron Architecture



Quantum -> Neutron

이랬던 녀석이...



Neutron 소개

기존 Neutron에서 제공하던 기능과 비교 시, 다음과 같다.

항목	Linux Bridge	Open vSwitch (OVS)	OVN (OVS+SDN Layer)
L2 스위칭	기본 지원	고성능 지원	(OVS 기반)
L3 라우팅	미지원 (외부 라우터 필요)	일부 가능 (DVR 등 활용)	네이티브 지원
보안 그룹	iptables 기반	hybrid 방식	ACL 기반
VLAN 지원	예	예	예
VXLAN/GRE 터널링	아니요	예	예
SDN 지원	아니요	수동 설정 필요	예, 내장 SDN 컨트롤러 (NB/SB DB)



Neutron 소개

항목	Linux Bridge	Open vSwitch (OVS)	OVN (OVS+SDN Layer)
DHCP, NAT, L4 정책	예	아니요 (외부 서비스 필요)	예, OVN 내장 기능
성능	낮음 (커널 경로)	높음 (커널 경로, DPDK 지원)	중간~높음 (구조 복잡)
구성 복잡도	매우 쉬움	보통	복잡 (OVS + OVN DB)
대표 사용 사례	간단한 개발환경, PoC	대부분의 OpenStack 운영 환경	클라우드 네이티브 SDN, NFV



Neutron 성능 비교

기존에 사용하던 SDN 기능과 비교 시, 다음과 같다.

항목	Linux Bridge	OVS	OVN
기본 성능 (단일 노드, L2만)	빠름 (단순 구조)	유사	느릴 수 있음
보안 그룹 + NAT 등 활성화 시	느려짐 (iptables 부하)	hybrid 방식	ACL 기반, 성능은 OVS 기반
VXLAN 등 터널링 사용 시	미지원	지원	지원
대규모 배포 시	비효율적	확장성 우수	SDN 기반 고급 기능 제공
DPDK, SR-IOV 활용 시	지원 안 함	가능	가능 (OVS 기반)



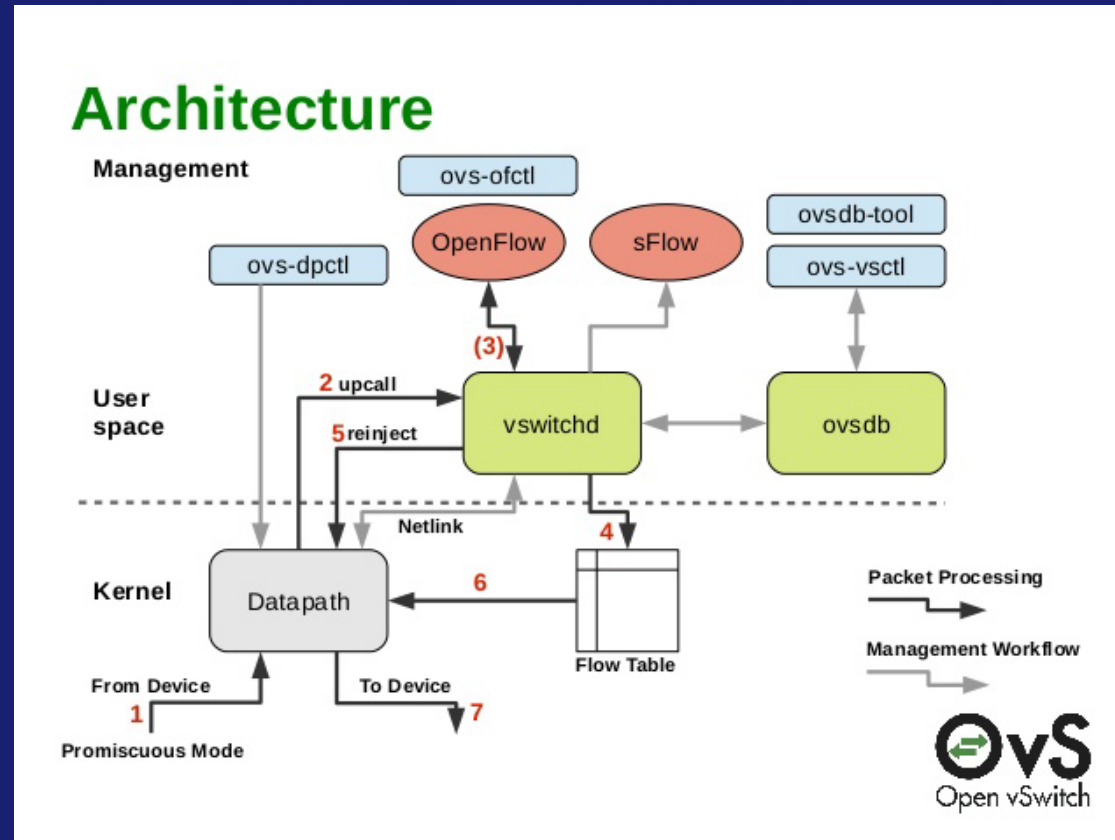
OVS VS OVN

OVS/OVN 성능은 다음처럼 차이가 있다.

항목	ML2/OVS (Seconds)	OVN (Seconds)	% 개선
Average	80.7	23.4	70.9%
95%	163.2	35.3	78.4%
Maximum	221.9	47.8	78.4%
Minimum	18.7	3.8	79.8%



OVS



Open vSwitch(OVS)

Open vSwitch는 가상 스위치로, 하이퍼바이저(예: KVM) 내부에서 가상머신 간 네트워크 트래픽을 스위칭해주는 역할을 한다. 리눅스 커널 기반의 브리지보다 훨씬 유연하며, VLAN, VXLAN, GRE, Geneve 같은 오버레이 네트워크를 지원한다.

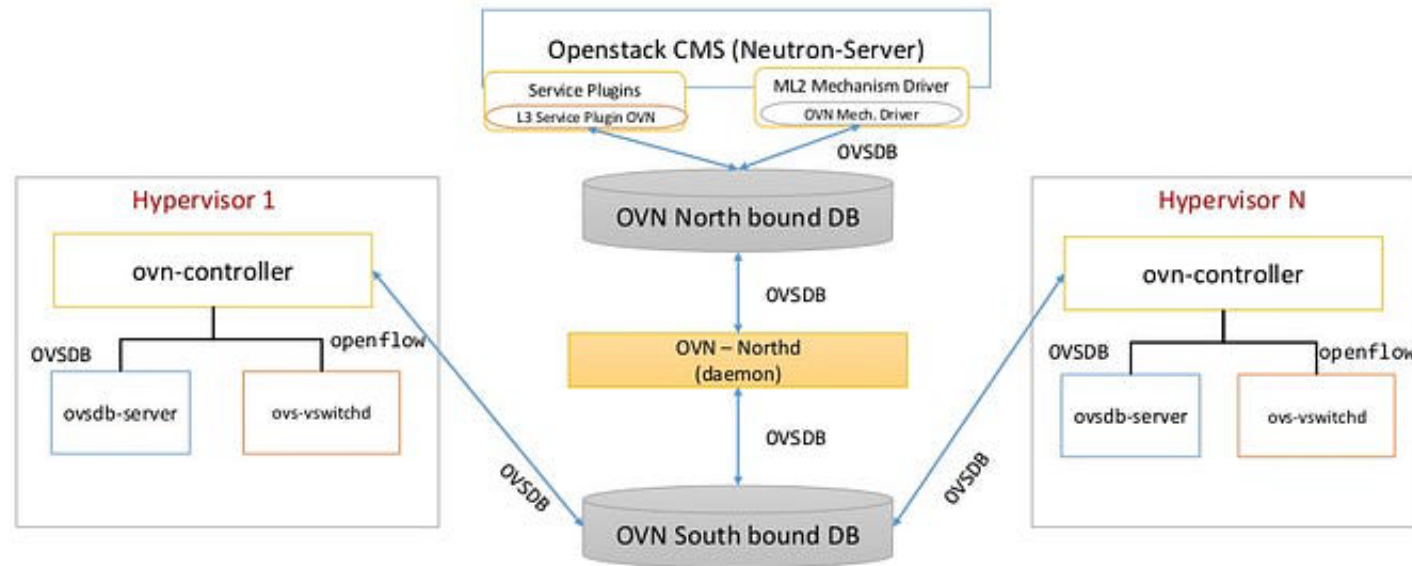
즉, 하나의 서버 안에서 가상 네트워크를 구성하고, 다른 서버와 터널을 통해 연결해주는 핵심 네트워크 레이어다.

1. 네트워크 스위치 구성
2. VM간 트래픽 라우팅
3. Router, DHCP와 같은 기능은 제공하지 않음
4. NAMESPACE, Netfilter, Dnsmasq와 같은 기능이 필요



OVN

OVN – Architecture



ovn-northd

Translate between the logical network elements configured by the CMS to the Northbound DB and model them to the Southbound DB tables, which holds the physical/infrastructure bindings and the logical flows which enable the logical connectivity.



OVN(Open Virtual Network)

OVN은 Open vSwitch 위에서 동작하는 가상 네트워크 제어 Plane이다. 기존 OVS가 단순히 패킷을 스위칭하는 기능만 제공한다면, OVN은 가상 라우터, 스위치, 보안 그룹, NAT, DHCP 등 L2~L3 기능까지 통합하여 제공한다.

즉, Neutron을 대신해 네트워크 토폴로지를 직접 관리할 수 있는 SDN 컨트롤러 역할을 수행한다

1. Route 기능 제공
2. network, subnet 제공
3. DHCP 서버 제공
4. ACL 및 Security Group 제공



ML2(Modular Layer 2 Plugin)

ML2는 OpenStack Neutron에서 사용하는 플러그인 프레임워크로, L2 네트워크 드라이버(예: LinuxBridge, OVS, SR-IOV 등)를 모듈화하여 선택적으로 사용할 수 있게 해준다.

즉, ML2는 "Neutron이 어떤 방식으로 스위칭을 할지 결정하는 뼈대"라고 보면 된다.

현재는 리눅스 브릿지(Linux Bridge) 사용은 권장하지 않는다.



ML3(Modular Layer 3 Plugin)

ML3는 Neutron에서 라우팅 기능(L3)을 제공하는 플러그인 구조로 가상 라우터/NAT/Floating IP/외부 네트워크 연결 등 L3 기능을 구현하는 백-엔드 역할을 담당한다.

OVN을 사용할 경우에는 별도의 ML3 에이전트 대신 OVN의 자체 L3 기능이 이 역할을 수행한다. OVS를 사용하는 경우 커널에서 다음과 같은 기술을 사용하여 자원을 구성한다.

1. NAMESPACE
2. NETFILTER
3. DNSMASQ
4. TUN/TAP DEVICE



네트워크/서브넷 관계

네트워크 및 서브넷 관계를 정리하면 다음과 같다.

구성 요소	설명
네트워크	L2 계층 (MAC 기반 연결) 정의, 브리지 역할
서브넷	L3 계층 (IP 주소 범위) 정의, 라우팅과 DHCP 제공
포트	네트워크 + 서브넷에 연결되는 구체적인 인터페이스 (인스턴스의 NIC)



네트워크 생성

네트워크 생성 명령어는 다음과 같다.

```
# openstack network create my-net
--share          # 공유 네트워크
--provider-network-type vxlan|vlan|flat
--external       # 외부 네트워크로 표시 (floating IP용)
```

옵션	설명
share	생성된 네트워크를 모든 사용자에게 공유한다. 관리자만 사용이 가능.
provider-network-type	구성 시 사용할 네트워크 형식을 선택한다. 기본값은 flat.
external	외부 네트워크와 연결 및 구성이 필요한 경우.



포트 생성

포트는 아이피 생성과 동일하다. 포트가 생성이 되면 OVS에서 브릿지 포트가 생성이 된다.

```
# openstack port create \  
  --network my-net \  
  --fixed-ip subnet=my-subnet,ip-address=192.168.100.10 \  
  my-port
```

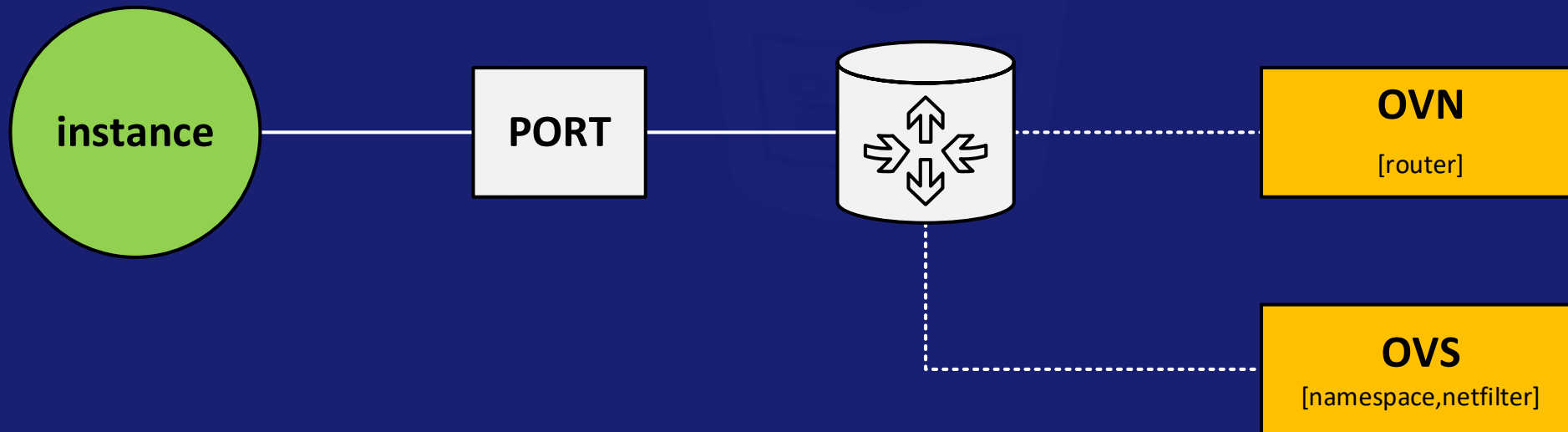
옵션	설명
fixed-ip	dhcp가 아닌, static으로 할당을 원하는 경우
subnet	포트를 생성할 서브넷 및 아이피 주소



네트워크 생성 명령어

라우터는 아래 명령어로 생성한다. OVS를 사용하는 경우, namespace 기반으로 구성이 된다.

```
# openstack router create my-router
```

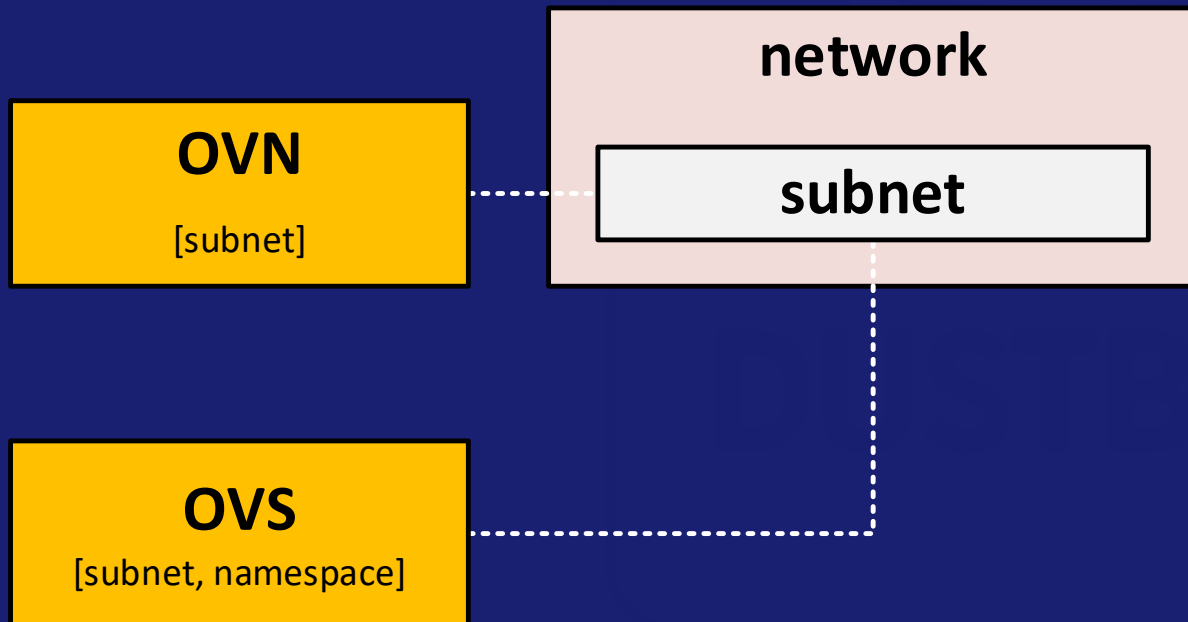


라우터 서브넷 추가

한 개 이상의 네트워크가 서로 통신이 필요한 경우, 라우터를 통해서 라우팅이 필요하다.

서브넷 연결 (인터페이스 추가)

```
# openstack router add subnet my-router my-subnet
```

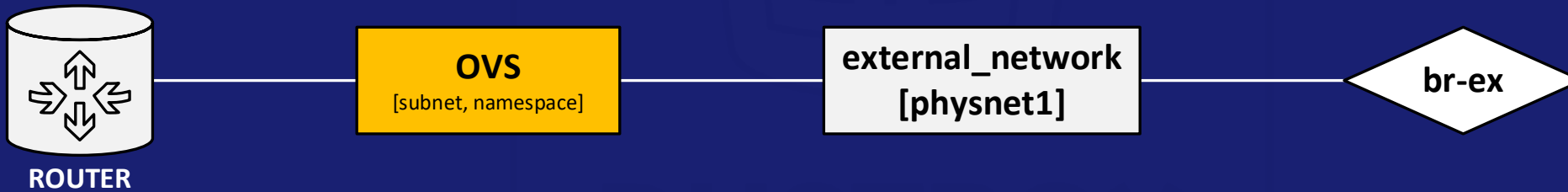


외부 라우터

트래픽을 외부로 전달이 필요한 경우, 아래처럼 생성 및 구성해야 한다. 이 기능은 OVN 혹은 Netfilter에서 구성이 된다.

외부 네트워크 설정

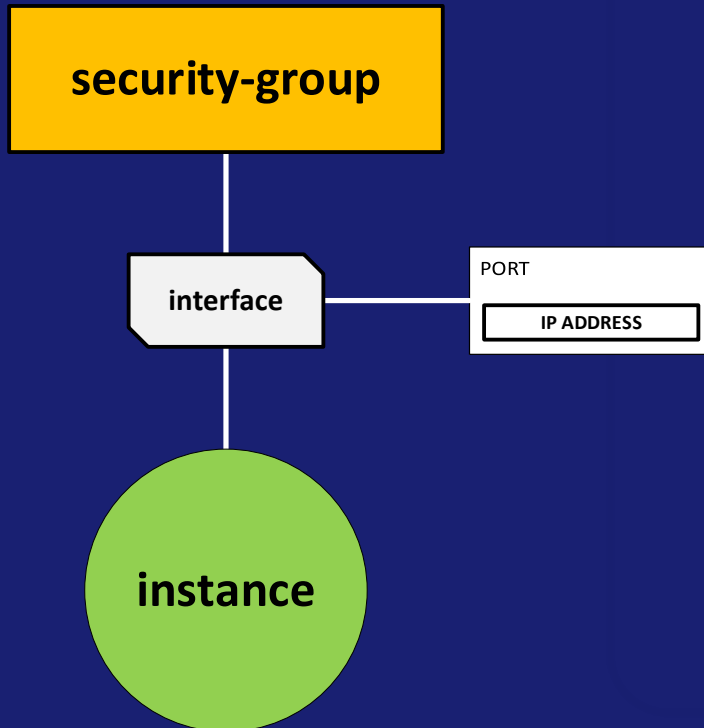
```
# openstack router set --external-gateway public-net my-router
```



보안그룹

네트워크 보안 그룹은 OVN이나 혹은 Netfilter에서 구성이 된다. 보안 그룹 생성 후, 정책을 구성한다.

```
# openstack security group create my-secgroup
```



보안그룹 정책 추가

보안그룹에 정책 추가가 가능하다. 보안 그룹은 다음과 같은 프로토콜에 적용이 가능하다.

1. TCP
2. UDP
3. ICMP

```
# openstack security group rule create \  
  --protocol tcp --dst-port 22 \  
  --ingress --ethertype IPv4 \  
  my-secgroup
```



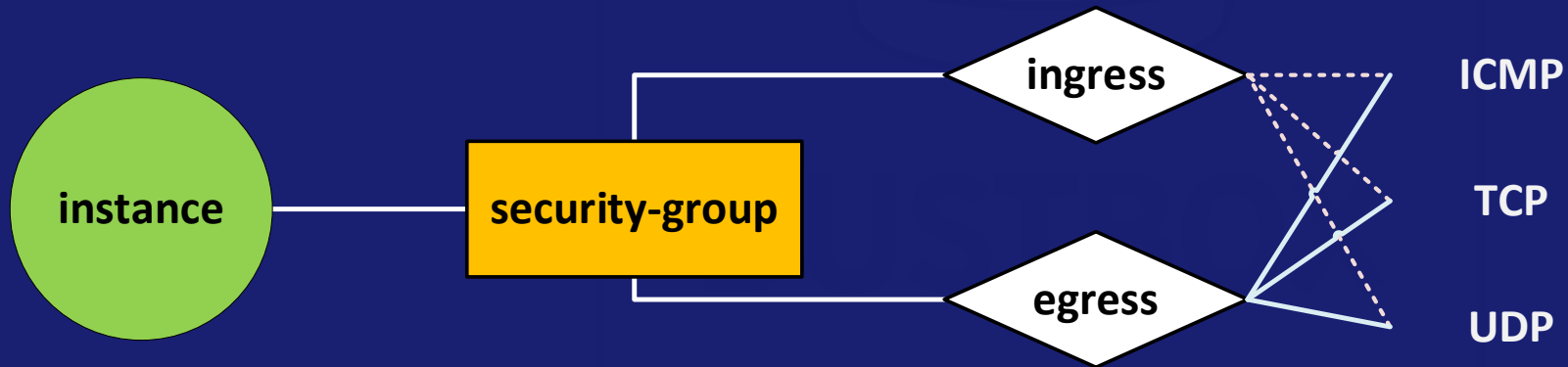
보안그룹 정책 추가

보안 그룹 정책은 두 가지 영역으로 분리 및 구성이 가능하다.

1. Ingress

2. Egress

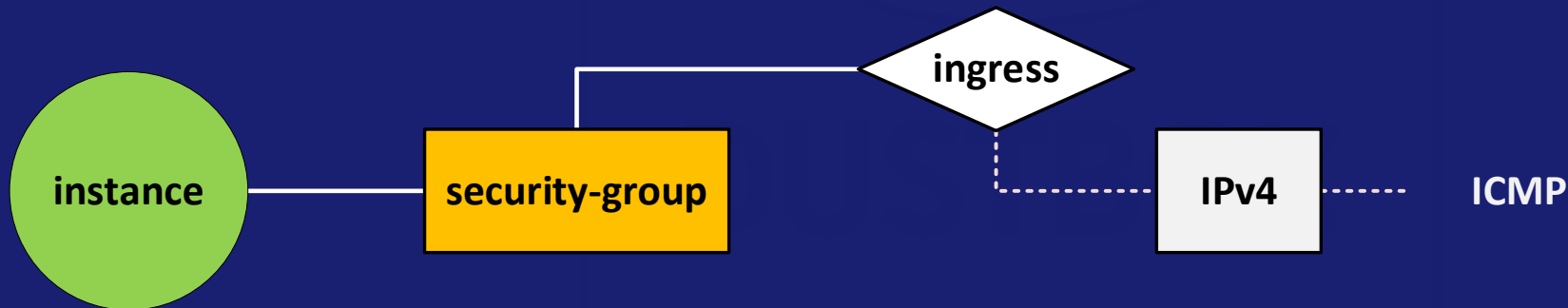
외부에서 들어온 자원은 Ingress, 내부에서 외부로 나가는 자원은 Egress라고 부른다.



보안그룹 정책 추가

만약, ICMP에서 특정 트래픽만 허용이 필요한 경우, 아래처럼 설정이 가능하다.

```
# Ping 허용
# openstack security group rule create \
  --protocol icmp \
  --ingress --ethertype IPv4 \
  my-secgroup
```

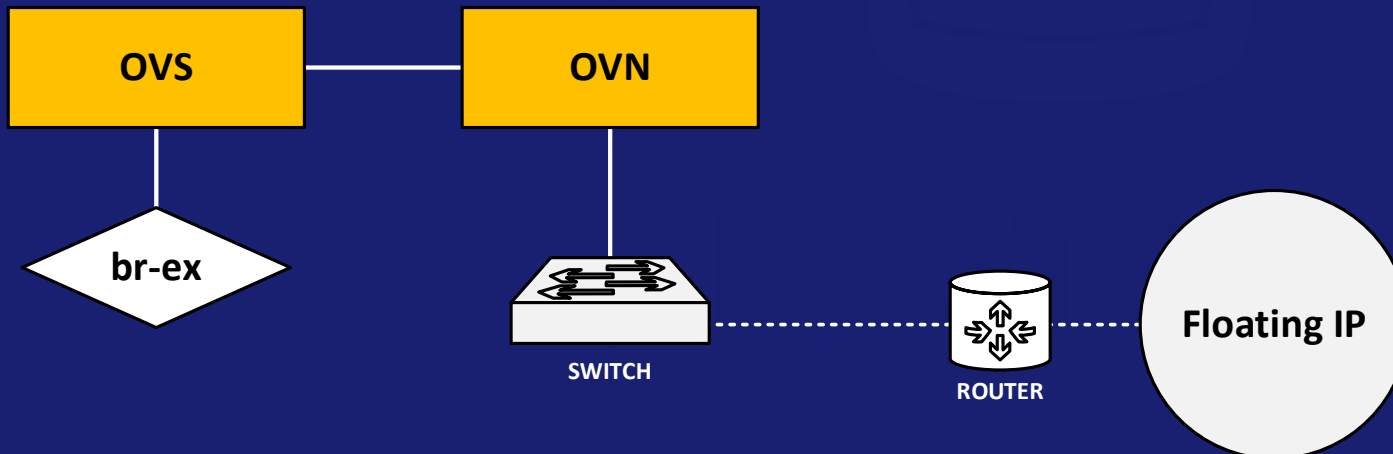


외부 아이피

외부 아이피는 외부와 연결된 네트워크 설정을 통해서 구성이 된다.

```
# openstack floating ip create public-net
```

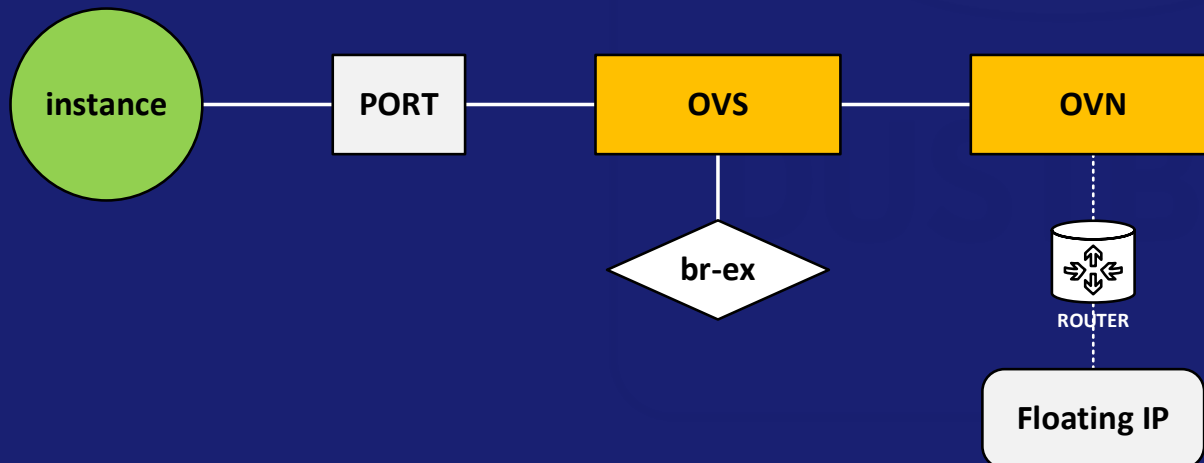
외부 네트워크는 최소 한 개의 physnetX와 연결 및 구성이 되어 있어야, 해당 물리적 인터페이스를 통해서 할당 및 구성이 가능하다.



외부 아이피

Floating IP는 보통 무작위로 할당이 되기 때문에, 특정 아이피를 사용하기 위해서, 아래처럼 Port로 구성 후 인스턴스에 할당한다.

```
# 포트 또는 인스턴스에 할당
# openstack floating ip set \
  --port my-port \
  <FLOATING_IP_ID 또는 IP>
```



QOS 제한

QOS를 구성하여, 특정 인스턴스의 트래픽 제한이 가능하다.

```
# openstack qos policy create my-qos
# openstack qos bandwidth-limit-rule create \
  --max-kbps 1000 --max-burst-kbps 500 \
  my-qos
```

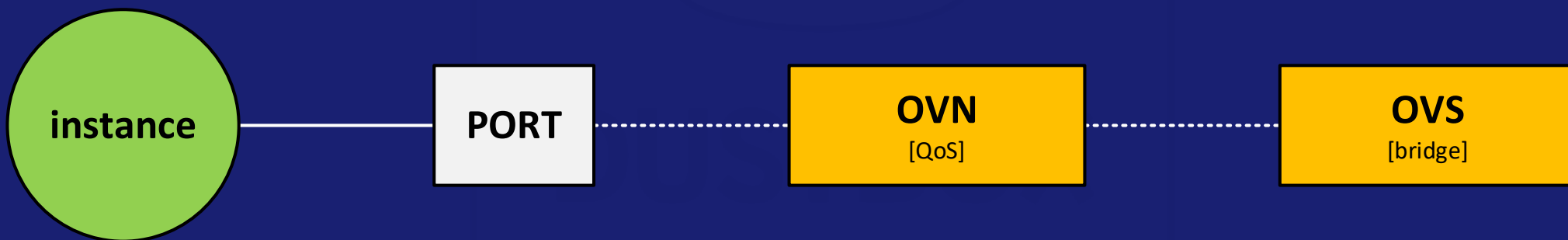


QoS 적용

생성된 QoS를 특정 인스턴스 포트에 적용이 가능하다.

포트에 정책 적용

```
# openstack port set --qos-policy my-qos my-port
```



트렁크 부모 포트 생성

트렁크 포트는 1개 이상의 포트를 묶어서 사용하는 경우 사용한다. 생성 시, 먼저 부모 인터페이스(포트)를 생성한다.

```
# 트렁크의 parent port 먼저 생성
```

```
# openstack port create --network my-net parent-port
```



트렁크 자식 포트 생성

부모 포트에서 사용 할 자식 포트 두 개를 생성한다. 생성 후, 부모에게 할당한다.

```
# 서브포트용 포트 생성
# openstack port create --network my-net subport1
# openstack port create --network my-net subport2
```



최종 트렁크 포트

최종적으로 부모 포트에 두 개의 인터페이스를 연결한다. 세그멘테이션은 인터페이스의 연결 유형이다.

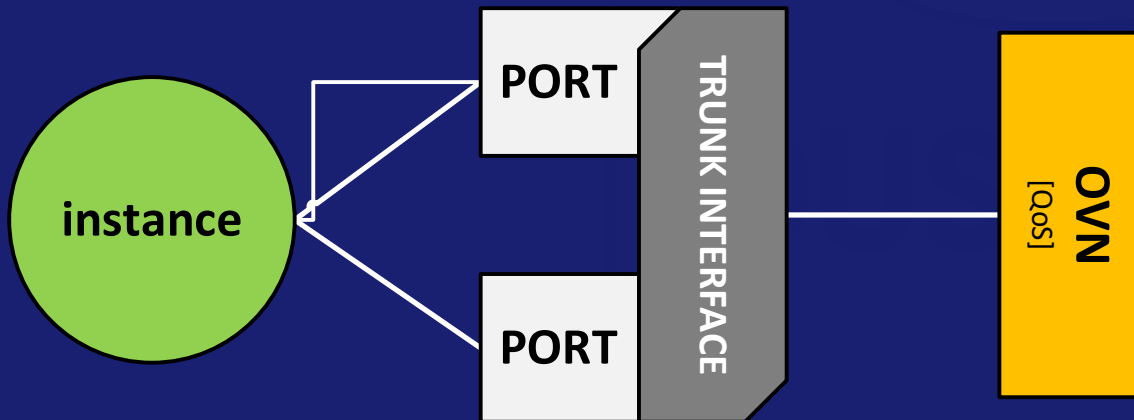
```
# trunk 생성
```

```
# openstack network trunk create my-trunk \
```

```
--parent-port parent-port \
```

```
--subport port=subport1,segmentation-type=vlan,segmentation-id=101 \
```

```
--subport port=subport2,segmentation-type=vlan,segmentation-id=102
```



방화벽 VS 보안그룹

방화벽과 보안그룹의 차이점은 다음과 같다.

항목	Security Group	FWaaS (v1/v2)
적용 대상	인스턴스(VM)의 포트	라우터(L3)
적용 위치	L2/L3 계층의 인스턴스 입출구	라우터를 통한 트래픽 입출구
기반 기술	iptables 또는 OVS 기반 보안 그룹	L3 Agent에 iptables 정책 추가
트래픽 방향	VM ↔ 네트워크	외부 ↔ 내부 네트워크
기본 동작	기본적으로 'DENY ALL', 허용만 명시	기본적으로 'ALLOW ALL', 차단만 명시 가능 (v1 기준)
정책 단위	포트 또는 인스턴스	네트워크 또는 라우터 단위
세분화	세밀한 포트, CIDR, 프로토콜 설정	정책 기반으로 한 묶음 룰 처리
다중 정책 지원	1개의 security group set per port	FWaaS v2에서는 다중 정책 가능



방화벽 VS 보안그룹

위의 내용 계속 이어서...

항목	Security Group	FWaaS (v1/v2)
상태 추적	상태 추적 가능 (stateful)	상태 추적 가능 (v2 기준)
삭제 예정 여부	계속 유지	이미 제거 됨(버전: Wallaby~Xena)
사용 예시	VM 간 접근 제한, SSH/HTTP 제한 등	외부 공격 필터링, 내부망 접근 제어 등



방화벽 VS 보안그룹

- Security Group: VM 단위, 내부 보안 정책. 접근 제어를 인스턴스 단위로 세밀하게 하고 싶을 때.
- FWaaS: 네트워크 경계에서의 방화벽. 트래픽 흐름 전체를 정책 단위로 제어하고 싶을 때 (과거 사용).

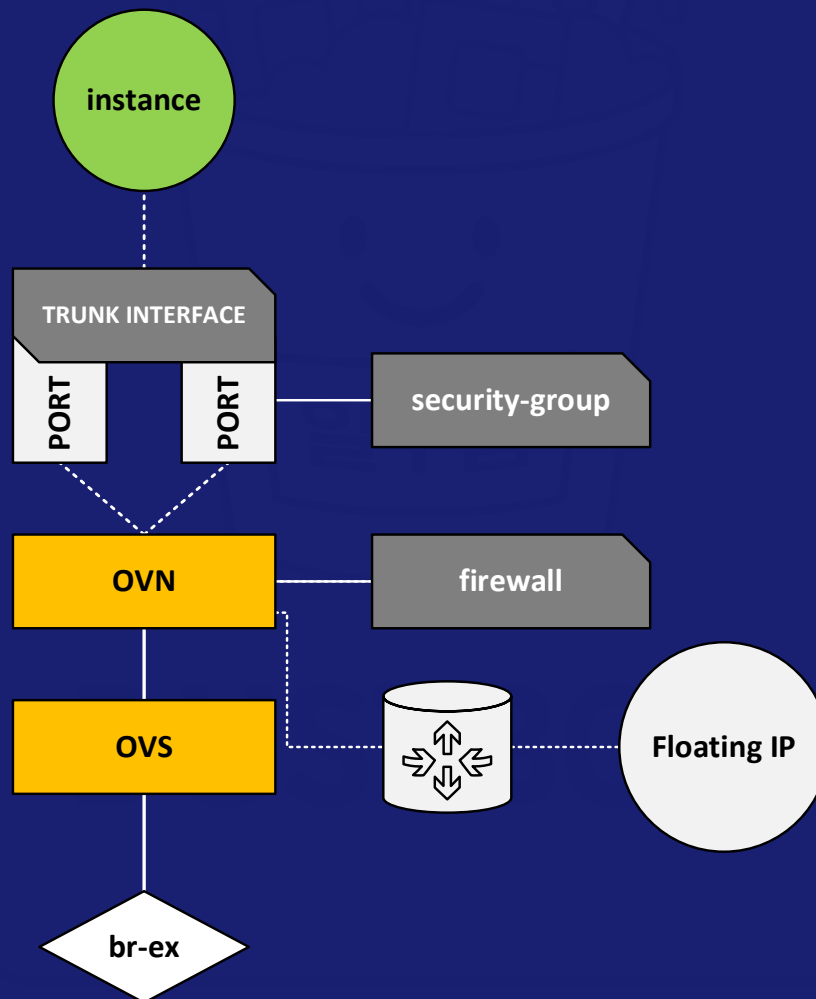
[외부] —▶ [Router (FWaaS 정책 적용)] —▶ [VM (Security Group 적용)]

외부에서 오는 모든 포트 차단,
10.0.0.0/24만 허용

VM의 포트 22, 80만 허용,
같은 네트워크의 VM만 접
속 허용



방화벽/보안그룹



방화벽 생성

보안그룹은 L2에서 생성이 된다.

만약, L3수준에서 방화벽이 필요한 경우 firewalld 서비스를 통해서 구성 및 생성이 가능하다. 아직 이 서비스는 유지가 되고 있지만, 아쉽게도 이걸 유지할 인력도 부족하며 또한, 다른 솔루션을 통해서 구현을 지원하고 있다.

정책과 규칙 생성

```
# openstack firewall group create my-fw --ingress-firewall-policy my-policy
```



방화벽 서비스는 왜 더 이상 사용하지 않는가?

1. 보안 그룹(Security Groups)이 더 광범위하고 유연하게 사용됨
2. FWaaS는 L3에 의존적인 구조로 복잡성과 확장성에 제약이 있었음
3. Neutron의 로드맵 방향이 보안 그룹, BGP, DVR 등과 통합된 네트워크 모델로 전환되었기 때문
4. OpenStack 커뮤니티의 유지 보수 인력 부족도 큰 이유
5. 현재 OVN으로 통합, OVN기반으로 사용을 매우 권장



L3 방화벽 대안/대체 기술

L3 방화벽은 다음과 같은 기술로 대체를 권장하고 있다.

대안 기술	설명
Security Groups	L2/L3 계층에서의 인스턴스 단위 방화벽 제어
OVN ACL (Open Virtual Network)	OVN 네트워크 사용 시 ACL 기반 고급 방화벽 정책
3rd-party 솔루션	Fortinet, Palo Alto, Juniper, A10 등과 연동
Kubernetes 기반 네트워크 정책	OpenStack 위의 K8s에선 Calico/OVN/Cilium 등 가능



방화벽 생성 및 정책 생성

FWaaS에 방화벽 적용하기 위해서 반드시 최소 한 개의 컨테이너 자원이 필요하다. 여기서 말하는 컨테이너는 애플리케이션을 위한 컨테이너가 아닌, 방화벽 자원 컨테이너 개념이다.

```
# openstack firewall policy create my-policy
# openstack firewall group create my-fw --ingress-firewall-policy my-policy
# openstack firewall policy add rule my-policy allow-http
```

명령어	설명
firewall policy create	컨테이너 생성
firewall group create my-fw	컨테이너 밑에 그룹을 생성한다
--ingress-firewall-policy	들어오는 트래픽에 대한 정책을 구성
policy add rule	allow-http 규칙을 추가한다. 80에 대해서 허용한다



방화벽 규칙 추가

아래와 같이 방화벽에 규칙 추가가 가능하다.

```
# openstack firewall rule create --protocol tcp --destination-port 80 \
  --action allow allow-http
```

옵션	설명
openstack firewall rule create	새로운 Firewall Rule 객체 생성
--protocol tcp	TCP 프로토콜에 대한 규칙임을 지정
--destination-port 80	목적지 포트 80 (HTTP) 에 대해 적용
--action allow	해당 조건에 일치하는 트래픽을 허용(allow)
allow-http	이 룰의 이름 (관리자 식별용, 자유롭게 지정 가능)



특정 포트에 방화벽 적용

Ingress 서비스만 허용하는 경우, 아래와 같이 구성이 가능하다.

포트에 FW 적용

```
# openstack firewall group set --ingress-firewall-policy my-policy --port my-port my-fw
```

요소	설명
openstack firewall group set	기존에 만들어진 Firewall Group을 설정함
--ingress-firewall-policy	인그레스(들어오는 트래픽) 정책으로 my-policy를 설정
--port my-port	해당 방화벽 그룹에 Neutron 포트(예: VM NIC 포트 등)를 연결
my-fw	설정할 대상 Firewall Group의 이름 또는 ID



보안그룹 정책

보안그룹에서 제공하는 기능은 두 가지가 있다.

1. Ingress (Inbound): 인스턴스로 들어오는 트래픽 제어
2. Egress (Outbound): 인스턴스에서 나가는 트래픽 제어

보안 그룹은 Open vSwitch에서 제공하는 기능이 아니며, Netfilter에서 제공한다. OVN를 사용하는 경우 OVN 통해서 사용자 영역에서 보안그룹을 제공한다.



보안그룹 적용

여러 개의 아이피를 논리적으로 묶어서 적용한다.

```
# openstack address group create my-ag --address 10.0.0.5/32 --address 10.0.0.10/32
```

요소	의미
openstack address group create	주소 그룹(Address Group)을 생성하는 명령
my-ag	주소 그룹의 이름
--address 10.0.0.5/32	주소 그룹에 포함될 첫 번째 IP 주소
--address 10.0.0.10/32	주소 그룹에 포함될 두 번째 IP 주소



네트워크 생성 및 구성

사용할 네트워크 정보 구성 및 서브넷을 생성 및 구성

1. 네트워크는 서브넷이 사용할 네트워크 유형(VXLAN, VLAN, GRE)와 같은 정보 구성
2. 최소 한 개의 서브넷이 구성이 되어 있어야 인스턴스 사용이 가능
3. 네트워크는 한 개 이상의 서브넷 구성이 가능

```
# openstack network create NET_NAME --project PROJECT_NAME
# openstack subnet create SUBNET_NAME \
  --project PROJECT_NAME --network NET_NAME \
  --subnet-range 192.168.100.0/24 \
  --gateway 192.168.100.1 \
  --dns-nameserver 1.1.1.1 --dns-nameserver 8.8.8.8 \
  --allocation-pool start=192.168.100.50,end=192.168.100.200 \
  --dhcp
```



네트워크 생성 및 구성

네트워크가 구성이 되면 라우터 및 게이트웨이 구성 및 설정한다.

```
# openstack router create ROUTER_NAME --project PROJECT_NAME
# openstack router set ROUTER_NAME --external-gateway PUBLIC_NET_NAME
# openstack router add subnet ROUTER_NAME SUBNET_NAME
# openstack network show NET_NAME -f value -c id
# openstack network rbac create --target-project TARGET_PROJECT_NAME \
  --action access_as_shared NET_ID
```



보안그룹 구성 및 생성

보안 그룹은 아래와 같이 생성 및 구성이 가능하다.

```
# openstack security group rule create SG_NAME --protocol icmp --ingress
# openstack security group rule create SG_NAME --protocol tcp --dst-port 22
--ingress
# openstack security group rule create SG_NAME --protocol tcp --dst-port 80
--ingress
# openstack security group rule create SG_NAME --protocol tcp --dst-port
443 --ingress
# openstack security group rule create SG_NAME --ethertype IPv4 --remote-ip
192.168.100.0/24 --protocol any --ingress
```



정리

방화벽 및 보안그룹 정리하면 다음과 같다.

1. 보안그룹은 가상머신이 사용하는 포트(PORT)에 적용 및 반영이 된다.
2. 방화벽은 가상머신이 사용하는 네트워크에서 적용 및 반영이 된다.
3. 현재 오픈스택은 방화벽보다 보안그룹 기준으로 보안이 구성이 된다.
4. OVS에서만 사용이 가능하고, OVN에서는 사용이 불가능하다.



연습문제

Nova

기본 기능

NOVA 소개

OpenStack의 Nova는 IaaS에서 가장 핵심적인 컴퓨트 서비스로, 가상머신(VM)의 생성을 비롯하여 스케줄링, 삭제, 리사이즈 등 라이프사이클 전반을 관리하는 역할을 수행한다.

Nova는 2010년, NASA와 Rackspace가 협업하여 만든 초기 프로젝트 중 하나로, 코드명은 Nova였다. NASA는 내부적으로 Nebula라는 클라우드 프로젝트를 운영하고 있었으며, Nova는 해당 프로젝트에서 파생된 컴포넌트다. 현재까지도 OpenStack에서 가장 크고 복잡한 프로젝트 중 하나로 자리잡고 있다.

Nova는 다양한 하이퍼바이저(KVM, QEMU, Xen, VMware, Hyper-V 등)를 지원하며, 플러그인 기반 아키텍처를 통해 확장성과 유연성을 제공한다.



NOVA 소개

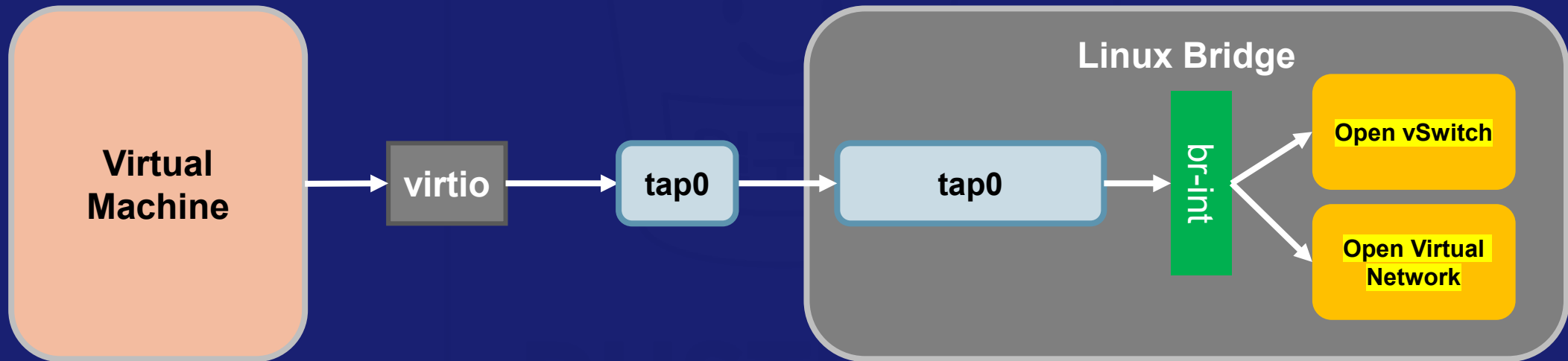
현재 Nova는 다음과 같은 핵심 컴포넌트로 구성된다.

- **nova-api**: 외부에서의 REST API 요청을 수신하고, 인증 및 요청 검증 후 내부 메시지 큐로 전달하는 게이트웨이 역할.
- **nova-conductor**: nova-compute가 직접 데이터베이스에 접근하지 못하도록 RPC를 통해 중재하며, DB에 대한 보안 및 추상화 계층 제공.
- **nova-compute**: 실제 하이퍼바이저(KVM 등)에서 인스턴스를 생성/제거/변경하는 핵심 작업을 담당. 메시지 큐를 통해 명령을 수신한다.

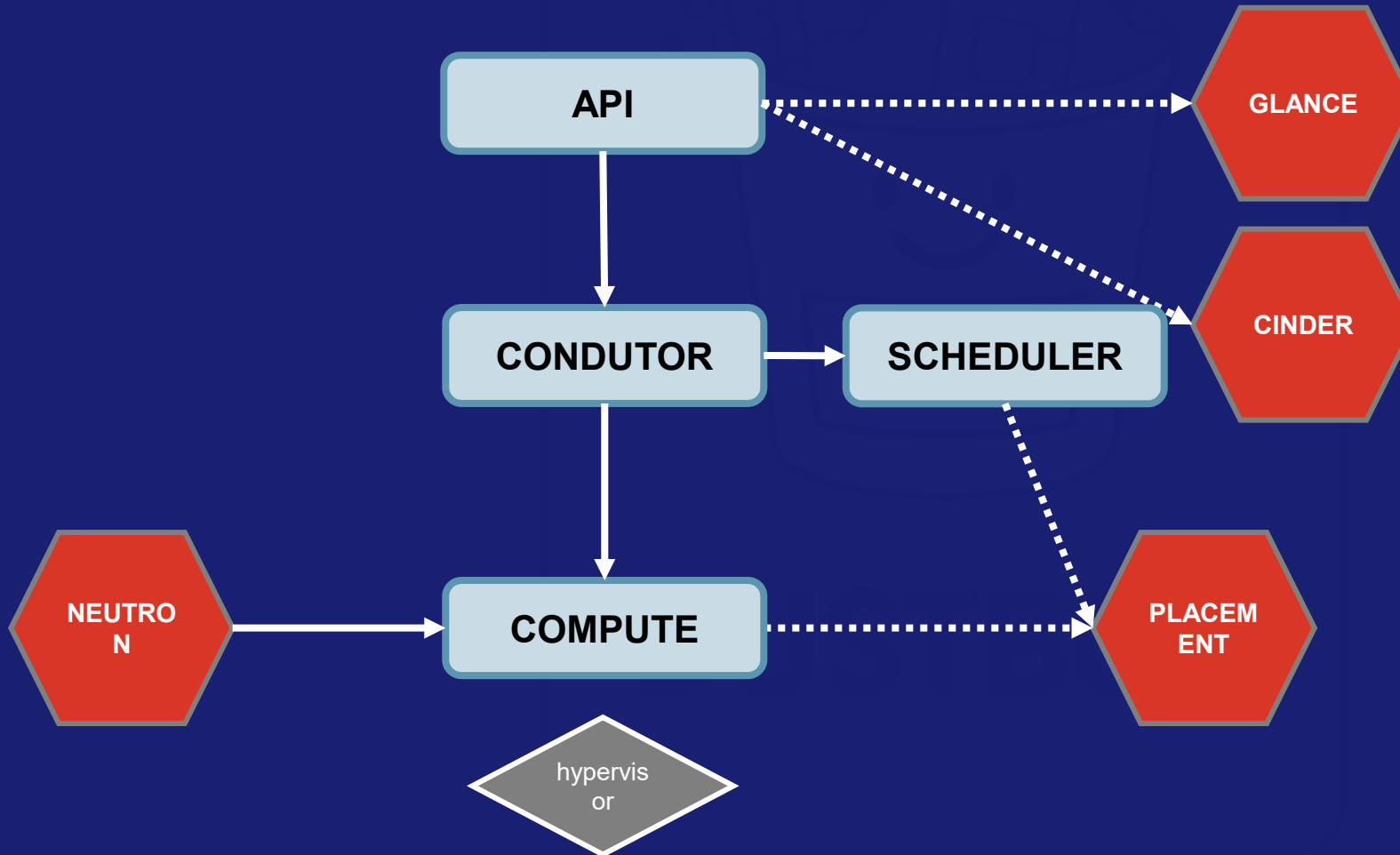
이 외에도 **nova-scheduler**, **nova-novncproxy** 등 다양한 구성 요소가 함께 동작하여 전체 컴퓨팅 서비스를 완성한다.



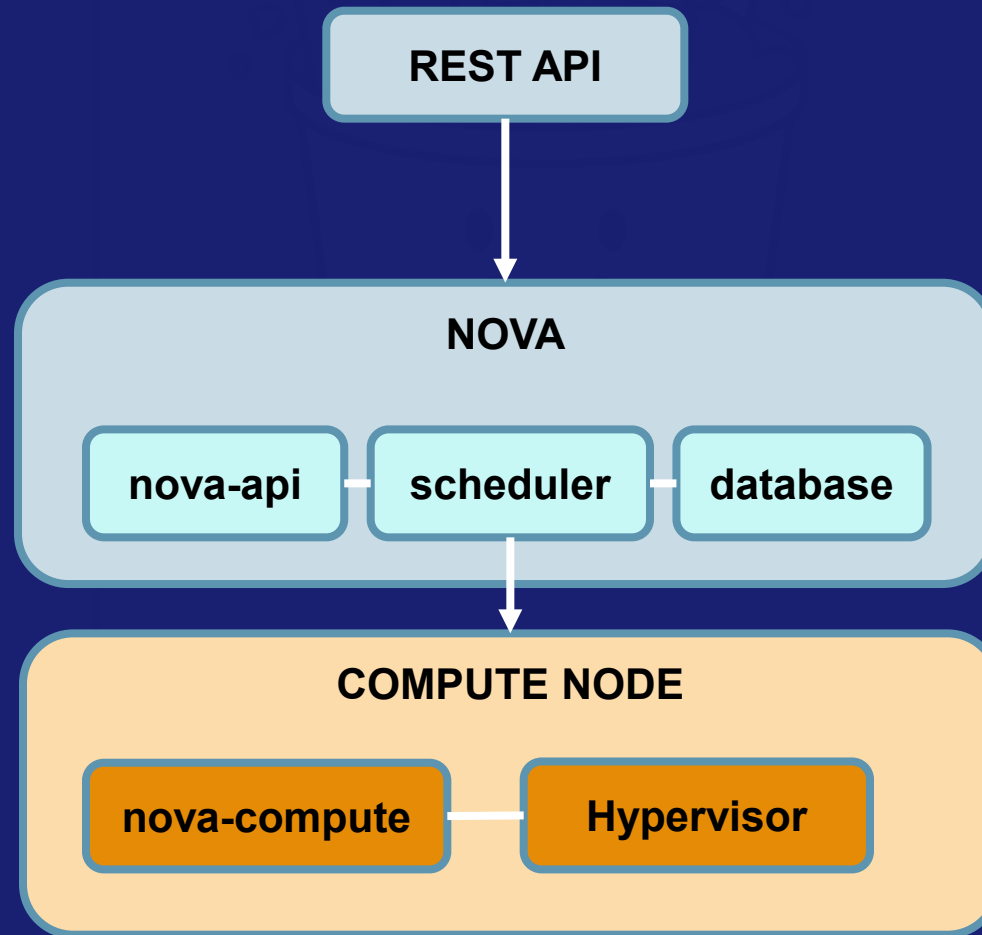
NOVA ARCHITECTURE



NOVA ARCHITECTURE



NOVA ARCHITECTURE



NOVA 자원

NOVA에서 기본적으로 제공하는 자원은 다음과 같다.

1. 보안그룹(Security Group)
2. 키-페어(Key Pair)
3. 플레이버(Flavor)

이외 나머지 자원들은 뉴트론/신더/글랜스와 같은 서비스에 제공 받는다.



NOVA 보안그룹

노바에서 제공하는 보안 그룹은 보통 두 가지 위치에 대해서 제한 기능을 제공한다.

1. Ingress: 외부에서 접근하는 트래픽
2. Egress: 내부에서 외부로 나가는 트래픽

보안그룹 리눅스 커널 라우팅 혹은 OVS/OVN 수준에서 정의 되는 보안 수준이다. 보안그룹을 올바르게 사용하는 경우, 굳이 시스템에서 제공하는 **방화벽(firewalld, windows firewall)** 사용이 필요하지 않다.

OVN를 사용하는 경우 Open vSwitch와 다르게 커널 수준의 iptables/nftables에서 제공하지 않으며, OVN ACL수준에서 Security Group를 제공한다.

AWS 및 GCP 플랫폼에서는 보안그룹/ACL를 둘 다 제공하지만, 오픈스택에서는 기본적으로 SecurityGroup 수준에서 처리한다.



NOVA 보안그룹

구분	처리 위치	특징
OVS 기반 보안그룹	커널 수준(iptables, conntrack)	리눅스 네임스페이스 안에서 패킷 필터링
OVN 기반 보안그룹	OVN 논리 스위치의 ACL 규칙	커널 대신 OVN 컨트롤 플레인에서 처리
AWS/GCP 구조	Security Group + Network ACL	두 계층으로 분리되어 작동
OpenStack 구조	Security Group = ACL	단일 정책 계층에서 양방향 제어



NOVA 키 페어

NOVA 키 페어는 사용자가 생성한 인스턴스에 SSH키 혹은 윈도우 서버의 비밀번호 구성 및 설정을 도와준다. 현재 키 페어는 기본적으로 다음과 같은 암호형식을 제공한다.

1. RSA
2. ECDSA
3. ED25519

현재 대다수 오픈소스는 RSA 기반을 사용하지 않으며 ED25519방식으로 SSH키를 생성 및 관리한다.



NOVA 공개/비공개 키

구분	설명
Private Key	사용자의 로컬 PC에 저장됨 (~/.ssh/id_rsa, ~/.ssh/id_ed25519 등). OpenStack은 저장하지 않음.
Public Key	OpenStack Nova에 등록됨. 인스턴스 생성 시 cloud-init이 /home/<user>/.ssh/authorized_keys에 삽입.
역할	SSH 접속 시 비밀번호 대신 공개키/개인키 쌍으로 인증 수행.



NOVA 키 페어 명령어

특정 형식으로 키 생성을 원하는 경우, 아래와 같이 생성 및 관리한다.

```
# ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa
# ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519
# openstack keypair create --public-key ~/.ssh/id_ed25519.pub mykey

# openstack keypair create mykey > mykey.pem
# chmod 600 mykey.pem
```



NOVA 키 페어 기능비교

구성 요소	저장 위치	내용
Public Key	Nova DB (nova.key_pairs 테이블)	키 이름, 타입, 공개키 데이터
Private Key	사용자 로컬 PC (또는 PEM 파일)	절대 OpenStack에 저장되지 않음
Authorized Key	인스턴스 내 /home/<user>/.ssh/authorized_keys	cloud-init이 자동 삽입



NOVA 하이퍼바이저 목록

노바에서 제공하는 하이퍼바이저는 다음과 같다.

하이퍼바이저 종류	설명
KVM (기본)	Nova의 기본 설정이며, 대부분의 배포판에서 권장됨
QEMU	KVM 없이도 작동 가능한 하이퍼바이저로, 테스트 및 개발 환경에 사용됨
LXC	리눅스 컨테이너 기반 경량 가상화. 현재는 비활성화 혹은 제한적 지원 (비권장)
Xen	오픈소스 하이퍼바이저로, 도메인 기반 격리 제공. 일부 기업 환경에서 사용됨
VMware vSphere	VMware 환경과의 통합을 통해 ESXi 하이퍼바이저와 연동 가능
Hyper-V	Microsoft Windows 환경용 하이퍼바이저. 일부 윈도우 기반 OpenStack에서 사용됨
PowerVM	IBM Power 시스템에서 사용하는 상용 하이퍼바이저
Virtuozzo	컨테이너 기반 하이퍼바이저. 특수한 목적의 환경에서 사용
Baremetal	하이퍼바이저가 없는 베어메탈 서버 배포. Nova-Ironic 통합으로 지원됨
z/VM	IBM 메인프레임용 하이퍼바이저. 특수 환경에서 제한적으로 사용



컴퓨터 서비스 컴포넌트

컴퓨터 서비스에서 제공하는 컴포넌트는 다음과 같다.

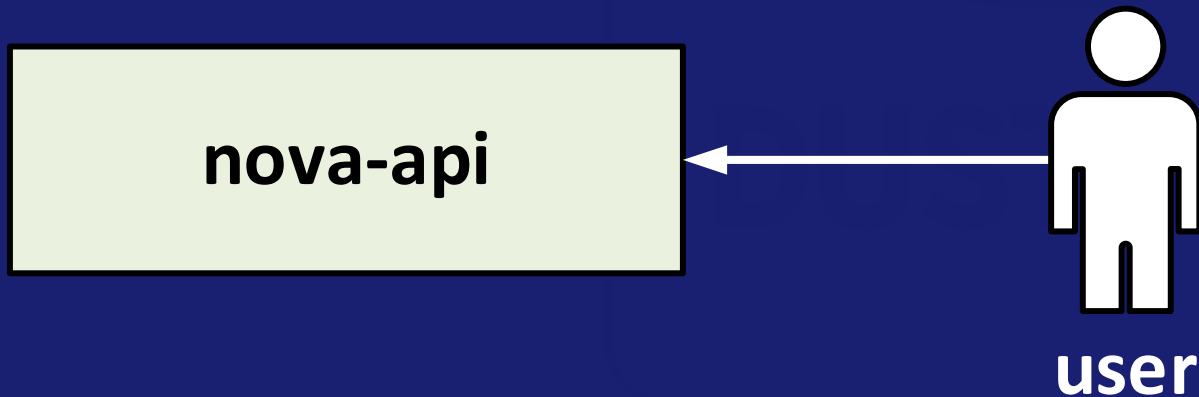
컴포넌트	설명	주요 역할
nova-api	RESTful API를 통해 사용자 요청을 수신	사용자 요청 처리 및 메시지 전송
nova-scheduler	요청된 인스턴스를 적절한 컴퓨터 노드에 스케줄링	VM 스케줄링 결정
nova-conductor	compute와 DB 사이의 간접 통신 수행	보안 중재자 역할
nova-compute	하이퍼바이저와 통신하여 인스턴스를 직접 생성/관리	VM 생성 및 관리
nova-placement	리소스 가용성과 정책 기반으로 배치 최적화 수행	자원 배치 결정을 위한 정보 제공
nova-consoleauth	콘솔 접근에 대한 인증 수행	VNC 콘솔 인증
nova-novncproxy	VNC 콘솔을 웹으로 제공	VNC 중계
nova-database	모든 메타데이터 및 상태 정보 저장	중앙 상태 저장소 역할
nova-metadata	최근에 추가된 서비스. 메타데이터를 관리	cloud-init



NOVA-API

사용자의 인스턴스 생성, 삭제, 조회 요청을 **OpenStack REST API**로 수신하고, 내부 메시지 큐 또는 데이터베이스를 통해 다른 Nova 서비스로 요청을 전달한다.

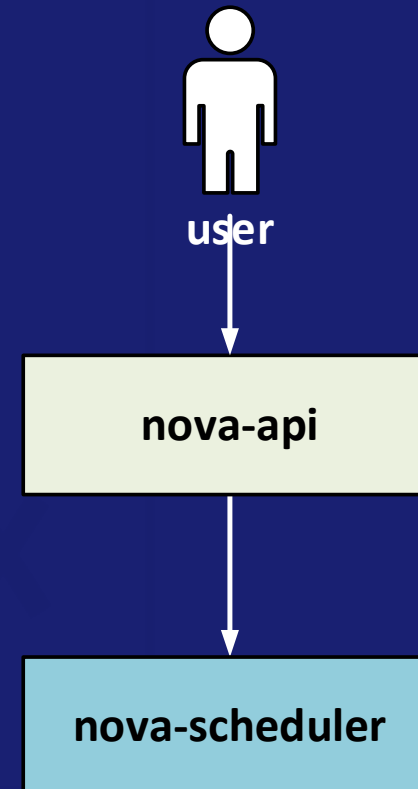
- API Gateway 역할
- 인증 토큰 처리 (keystone 연동)
- 비동기 작업을 큐에 전달 (RabbitMQ 등)



NOVA-SCHEDULER

nova-api로부터 전달된 인스턴스 생성 요청을 수신하고, 가용 자원 상태를 고려하여 인스턴스를 배치할 적절한 컴퓨트 노드를 선택한다.

- 필터(Filter) + 가중치(Weigher) 기반
- nova-placement에서 리소스 사용량 정보 조회



NOVA-SCHEDULER

Nova Scheduler는 인스턴스 생성 요청을 받고, 이를 적절한 compute node에 배치하기 위한 단계는 다음과 같다.

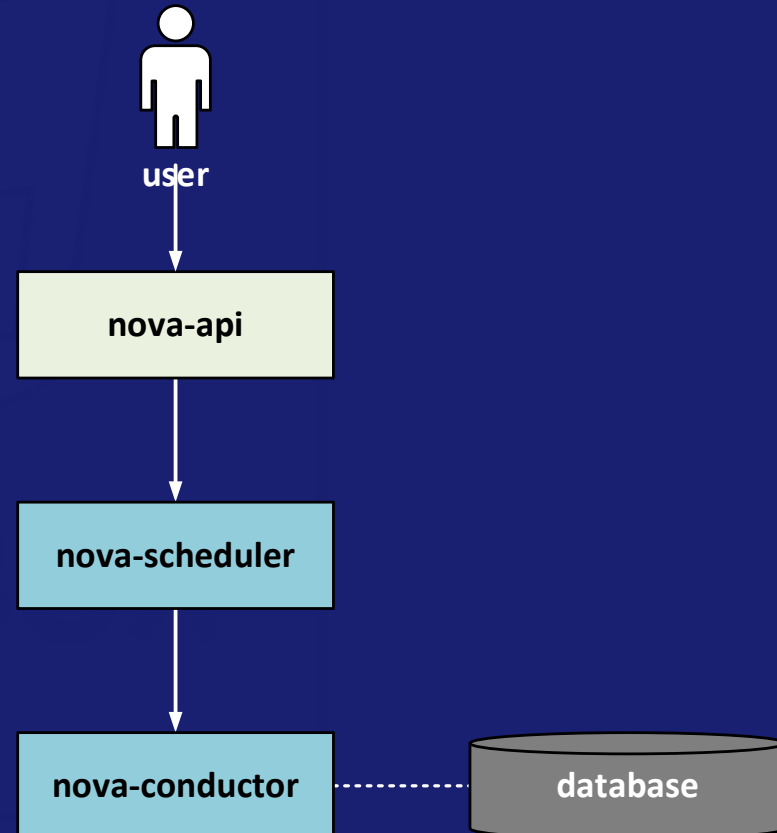
사용자가 인스턴스 생성 요청을 보내면, nova-api를 통해 scheduler까지 전달된다. 여기서 요청에는 인스턴스에 필요한 CPU, 메모리, 디스크 등의 자원 요구사항과 기타 정책(예: 가용 영역, affinity/anti-affinity 규칙 등)이 포함된다.



NOVA-CONDUCTOR

nova-compute가 직접 데이터베이스와 통신하지 않도록 중개자 역할을 수행하고, 보안을 강화하며 DB 연산을 위임받는다.

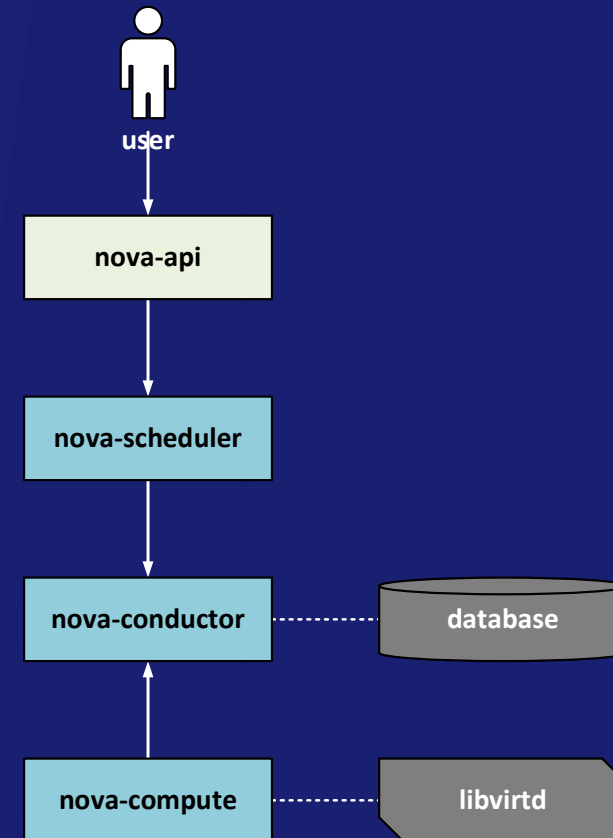
- nova-compute와 DB 사이의 간접 통신 담당
- RPC를 통해 동기/비동기 DB 작업 실행



NOVA-COMPUTE

nova-scheduler로부터 배정받은 인스턴스 생성 요청을 처리하며, libvirt 등 하이퍼바이저 인터페이스를 통해 실제 인스턴스를 생성하고 관리한다.

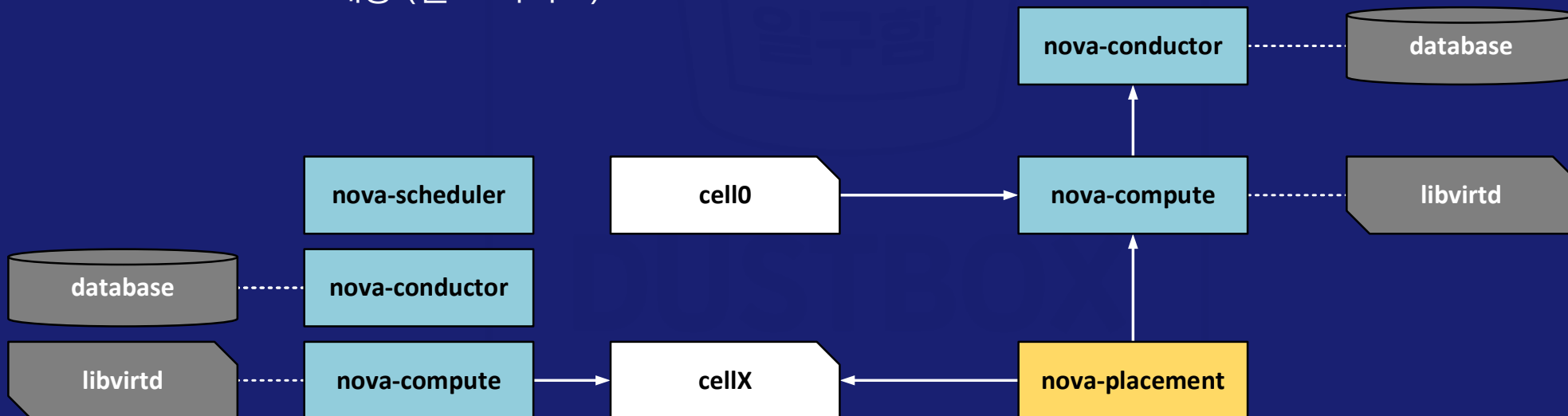
- Libvirt, Xen, VMware 등 다양한 드라이버 지원
- 각 노드에 배치됨 (agent 역할)



NOVA-PLACEMENT

가용 자원(메모리, vCPU, NUMA 등)을 추적하고 보고하여, 스케줄러가 인스턴스를 최적으로 배치할 수 있도록 정보를 제공한다.

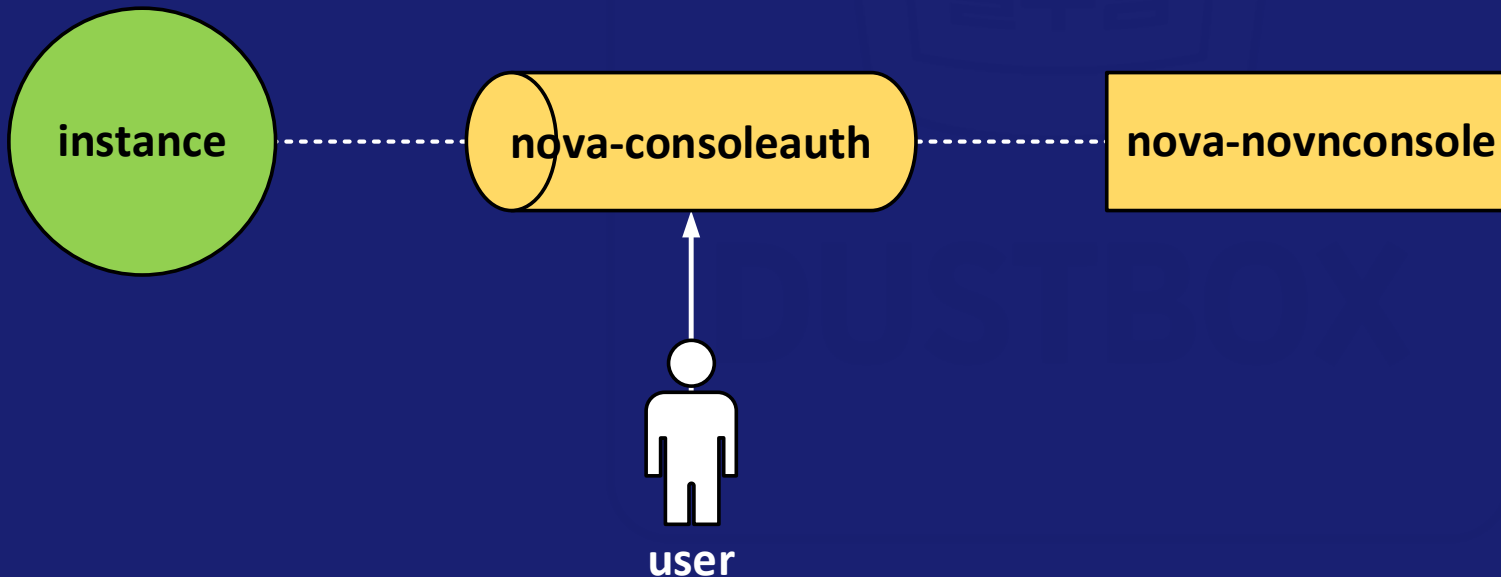
- 리소스 공급자(Resource Provider) 구조
- Placement API 제공 (별도 서비스)



NOVA-CONSOLEAUTH

사용자의 콘솔 접속 시 인증 정보를 검증하고, nova-novncproxy가 세션을 중계할 수 있도록 인증 토큰을 발급한다. 현재는 이 서비스는 더 이상 존재하지 않는다. 2020년도 TRAIN에서 완전히 중단.

- 콘솔 접근 요청 유효성 검증
- 단시간 유효한 세션 기반 인증 처리



NOVA-NOVNCPROXY

웹 기반 VNC 접속을 브라우저로부터 받아 해당 인스턴스의 콘솔과 중계하여 사용자가 VNC콘솔을 사용 할 수 있도록 한다.

- VNC over WebSocket 지원
- 사용자 ↔ novncproxy ↔ 인스턴스 VNC 연결
- 웹 기반으로 제공하는 VNC화면



스케줄러 필터링

nova.conf에 다음과 같이 기본값 구성이 되어 있다.

순서	필터 이름	설명
①	AvailabilityZoneFilter	요청된 AZ에 포함된 호스트만 통과
②	RamFilter	인스턴스에 필요한 RAM이 충분한 호스트만 통과
③	DiskFilter	디스크 공간이 충분한 호스트만 통과
④	ComputeFilter	nova-compute 서비스가 살아있는 호스트만 통과
⑤	ComputeCapabilitiesFilter	호스트의 기능(capabilities)과 요청 일치 확인
⑥	ImagePropertiesFilter	이미지 메타데이터 기반 필터링
⑦	CoreFilter	CPU 코어 수가 충분한 호스트만 통과
⑧	NUMATopologyFilter	NUMA 구성을 지원하는 경우 NUMA 요구 사항을 확인
⑨	ServerGroupAffinityFilter, AntiAffinityFilter	서버 그룹 정책 기반 (같은 호스트 또는 다른 호스트에 배치)



후보 노드 필터링 (Filter 단계)

Scheduler는 우선 전체 compute 노드 목록 중에서, 요청 조건을 만족하는 노드들을 필터링한다. 이 과정에서 여러 가지 필터들이 사용된다.

- **자원 상태 필터:** 노드의 현재 가용 리소스가 인스턴스 요구사항을 충족하는지 확인한다.
- **정책 필터:** 특정 가용 영역이나 host aggregate, 태그를 기준으로 후보 노드를 제한한다.
- **상태 및 네트워크 가용성 필터:** 해당 노드가 장애 상태가 아니며 네트워크 연결 및 기타 부가 서비스가 정상인지 점검한다.



후보 노드 평가 및 가중치 부여 (Weigher 단계)

필터링된 후보들에 대해 개별적 가중치 점수를 산출한다. 이 과정에서는 각 후보 노드의 현재 부하, 사용 가능 자원, 과거 작업 내역 등 다양한 요소가 고려된다.

가중치 알고리즘은 단순히 자원 양만 고려하는 것이 아니라, 정책에 따른 선호도를 반영하여 최적의 노드를 선택하도록 돕는다.



Placement 서비스와의 연계

Nova Scheduler는 최신 OpenStack 아키텍처에서 Placement API를 호출해, 현재 각 노드의 자원 상태 (Inventory)를 조회하고, Reservation을 통해 자원 할당이 가능한지 확인한다.

이 과정에서 Placement 서비스는 각 compute 노드(또는 Resource Provider)의 CPU, 메모리, 디스크 등 세부적인 자원 정보를 정밀하게 관리하며, 요청 조건을 만족하는 노드를 반환한다.



노드 선택 및 명령 전달(인스턴스)

필터와 가중치 평가 결과를 토대로 가장 적합한 compute 노드가 선택되고, Scheduler는 해당 노드에 인스턴스 생성 명령을 내려 배치 과정을 완료한다.



Nova-conductor

nova-conductor는 Nova의 내부 컴포넌트 간 데이터 흐름 중, 데이터베이스 접근을 안전하게 중개하는 기능을 수행한다. 원래는 각 **nova-compute** 노드가 직접 데이터베이스에 접근했지만, 이는 보안 및 확장성 문제를 야기할 수 있어, 중앙에서 처리하도록 변경되었다.

예를 들어, **compute** 노드는 인스턴스를 생성하면서 상태 정보를 DB에 기록해야 하지만, 직접 DB에 접근하지 않고 **conductor**에 RPC 호출을 보내 DB 갱신을 요청한다. **conductor**는 이 요청을 수신한 후, DB에 직접 접근하여 변경사항을 반영한다.

또한 **conductor**는 스케줄링 된 인스턴스 정보 전달, 마이그레이션 준비, 볼륨 연결 정보 갱신 등 다양한 내부 처리를 맡고 있으며, Nova의 내부 통신에서 데이터 무결성과 보안을 유지하는 핵심 컴포넌트이다.



Nova-conductor

동작 순서를 정리하면 다음과 같이 동작이 된다.

1. 사용자 요청은 nova-api에 도달한다.
2. 요청은 메시지 큐를 통해 nova-scheduler로 전달된다.
3. nova-scheduler는 placement에 자원 목록을 요청한다.
4. Placement는 현재 가능한 compute 노드 후보를 반환한다.
5. Scheduler는 정책에 따라 적절한 노드를 선택한다.
6. 선택된 노드에 인스턴스를 배치하기 위해 nova-compute가 호출된다.
7. nova-compute는 인스턴스 상태 및 메타데이터 갱신을 conductor에 요청한다.
8. nova-conductor는 DB에 변경 사항을 안전하게 반영한다.



Nova-compute

Nova-compute는 OpenStack의 가상머신 생성 및 제어를 담당하는 컴포넌트로, 사용자의 요청을 받아 하이퍼바이저를 통해 인스턴스를 생성한다.

Nova-compute 자체가 직접 하이퍼바이저를 구성하지는 않으며, 하이퍼바이저 인터페이스(예: libvirt, VMware 등)를 통해 VM 생성을 간접 수행한다. 예를 들어, libvirt 드라이버의 경우 XML 기반 설정을 생성해 libvirtd에 전달함으로써 가상머신이 생성된다.

Nova-compute는 가상머신이 정상적으로 네트워크에 연결되도록 Neutron과 연동되며, 이를 위해 최소 하나 이상의 L2 또는 L3 네트워크 드라이버(Open vSwitch, LinuxBridge 등)가 필요하다.



Nova-compute

현재 Nova-compute가 지원하는 주요 하이퍼바이저 드라이버는 다음과 같다:

1. libvirt (KVM, QEMU, LXC 등)
2. VMware vSphere
3. XenAPI
4. Hyper-V



Placement, Cell

Placement

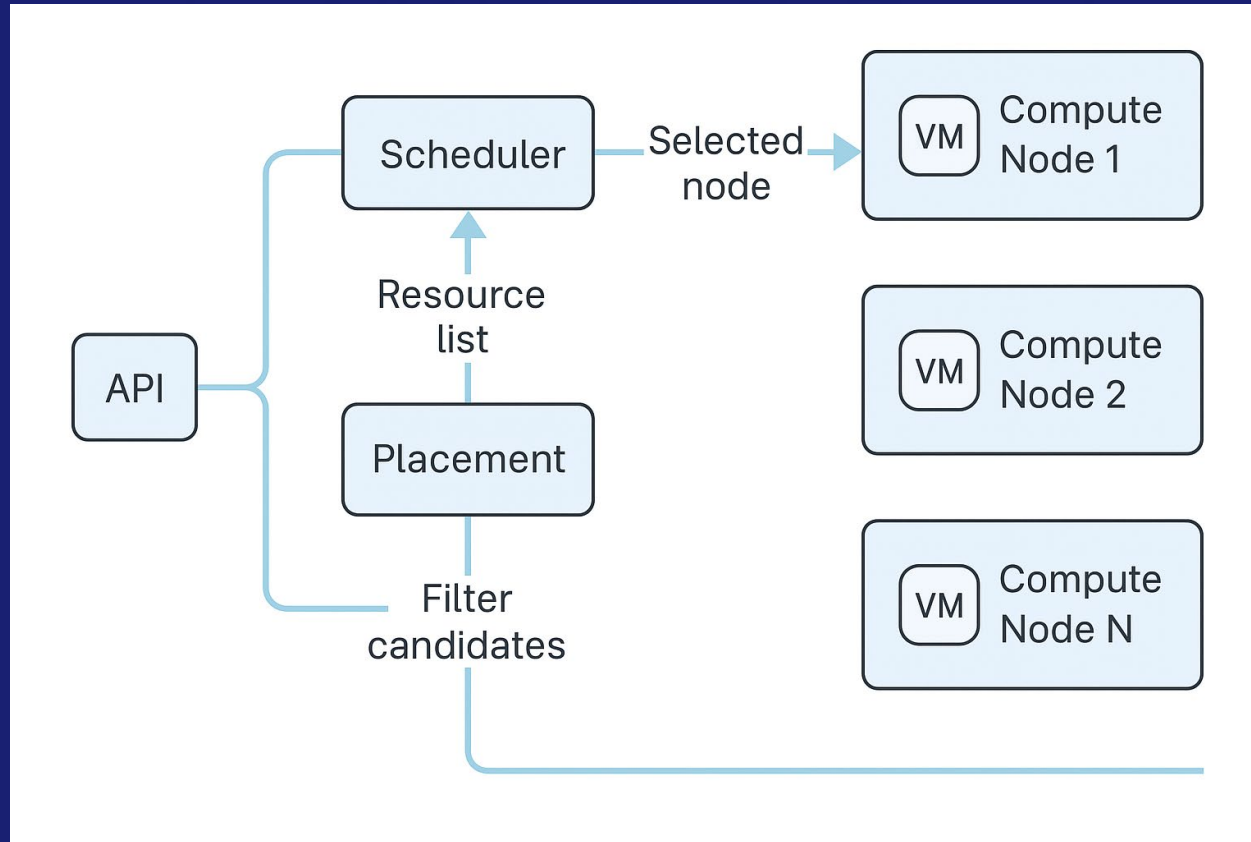
Placement는 OpenStack Nova의 리소스 추적(Resource Tracking)과 할당(Resource Allocation)을 위한 독립 서비스로, Nova 컴포넌트들과 분리되어 동작한다. OpenStack Pike 릴리즈 이후 정식 컴포넌트로 분리되었으며, Nova뿐 아니라 Ironic, Cyborg 등의 다른 서비스에서도 활용 가능하다.

Cell

Cell은 Nova의 스케일 아웃(Scale-Out)을 위한 논리적 구조로, 수천 대의 compute 노드를 효율적으로 관리하기 위해 도입되었다. Nova는 처음부터 단일 DB 구조로 운영되었지만, 대규모 환경에서는 성능과 확장성에 한계가 있어, cell이라는 구조로 분산 배치가 가능하도록 설계되었다.



Placement, Cell 아키텍처



Placement, Cell

Placement는 다음과 같은 기능을 한다.

1. 가상머신 생성 시 필요한 CPU, RAM, DISK 등의 리소스 요구사항을 계산하고, 현재 각 compute 노드에서 사용 가능한 리소스를 비교하여, 할당 가능한 노드를 필터링한다.
2. NUMA, hugepages, PCI devices, GPU 등 고급 리소스까지 추적이 가능하며, 리소스 제공자(Resource Providers), 소비자(Consumers), 할당량(Allocations)을 추상화 기반으로 확장 가능한 자원 모델을 제공한다.



Placement, Cell

노바는 최소 한 개의 cell0번을 가지도 동작한다. 이후 cell 실제로 운영 시 사용하는 셀로 구성이 된다.

- Cell0: 스케줄링 실패 시, 실패한 인스턴스의 메타데이터를 보관하는 비 운영 셀이다.
- Cell1, Cell2...: 실제 compute node들이 존재하는 운영 셀이다. 각 셀은 자체 메시지 큐(RabbitMQ)와 데이터베이스를 가진다.
- 각 Cell은 nova-conductor, nova-compute 등의 컴포넌트를 독립적으로 갖추고, 독립적으로 스케일 가능하다.
- nova-api는 중앙에 위치하고, 셀 간 라우팅을 담당하는 nova-cells 컴포넌트를 통해 요청을 전달한다.



Placement/Cell 정리

추가된 기능에 대해서 정리하면 다음과 같다.

항목	Placement	Cell 구조
기능	자원 추적, 자원 할당	Nova 컴포넌트를 논리적으로 분산하여 확장성 확보
구조	독립 API 서비스, DB 포함	각 셀마다 독립 메시지 큐 및 DB 구성
사용 목적	정확한 리소스 매칭과 스케줄링 최적화	대규모 인프라에서 Nova 확장성과 장애 격리 지원
핵심 요소	Resource Provider, Allocation, Inventory	Cell0, Cell1~N, nova-cells 라우팅

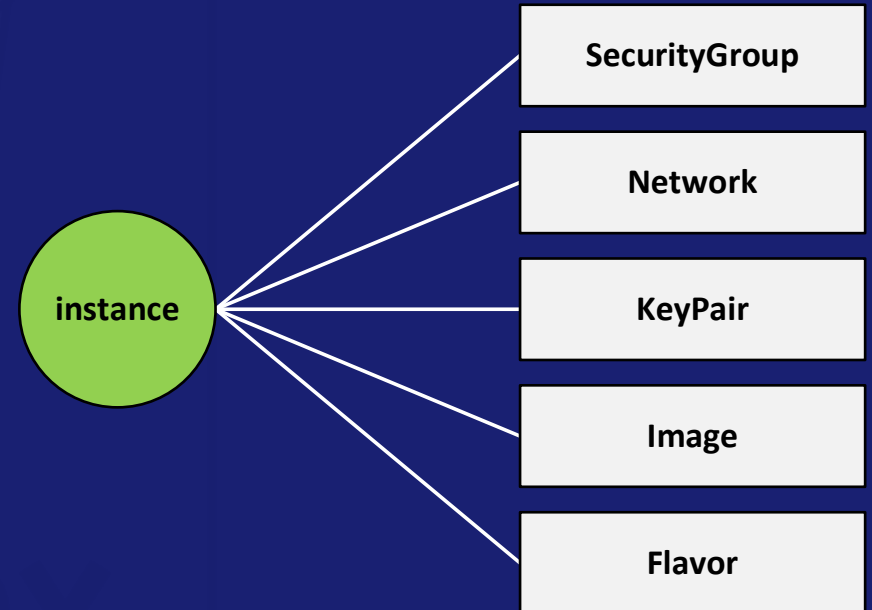


인스턴스 생성 준비

인스턴스 생성을 위해서 다음과 같은 조건을 충족해야 한다.

1. 네트워크
2. 네트워크 서브넷
3. 보안그룹
4. 키페어
5. 운영체제 이미지
6. 플레이버(인스턴스 사양)

이와 같은 조건을 충족하지 못하는 경우, 올바르게 인스턴스 생성이 불가능하다.



인스턴스 생성 준비

인스턴스 생성을 위해서 사용이 가능한 자원 목록을 아래 명령어로 확인한다.

```
# openstack image list
# openstack flavor list
# openstack network list
# openstack router list
# openstack security group list
# openstack keypair list
# openstack floating ip list
```



인스턴스 생성

라우트 생성이 필요한 경우, 아래 명령어로 생성한다.

```
# openstack network create net-demo
```



라우터 생성

라우트 생성이 필요한 경우, 아래 명령어로 생성한다. 라우터에는 다음과 같이 자원을 구성 및 할당한다.

1. 라우터에 내부 서브넷 연결 및 구성
2. 외부로 트래픽 라우팅이 필요한 경우 `--external-gateway` 설정

```
# openstack router create router-demo
# openstack router set router-demo --external-gateway public
# openstack router add subnet router-demo subnet-demo
```



보안그룹

외부에서 접근을 허용하기 위해서 보안 그룹을 생성 및 구성한다. 가급적이면 default 보안 그룹 사용은 사용하지 말고, 새로운 보안 그룹을 생성한다.

```
# openstack security group create lab
# openstack security group rule create --proto icmp lab
# openstack security group rule create --proto tcp --dst-port 22 lab
```



키페어

보안그룹이 올바르게 생성이 되면, 인스턴스 접근 시 사용할 SSH 공개/비공개 키를 생성한다.

```
# ssh-keygen -q -N "" -f ~/.ssh/id_rsa
# openstack keypair create --public-key ~/.ssh/id_rsa.pub mykey
# openstack keypair list
```



플레이버

인스턴스 생성을 최소 한 개의 플레이버(인스턴스 사양)을 구성해야 한다.

```
# openstack flavor create m1.small \  
--id auto \  
--vcpus 2 \  
--ram 4096 \  
--disk 20 \  
--swap 0 \  
--ephemeral 0 \  
--rxtx-factor 1.0
```



이미지 및 플레이버 확인

사용이 가능한 이미지 및 플레이버 목록을 확인한다.

```
# openstack image list
```

```
# openstack flavor list
```



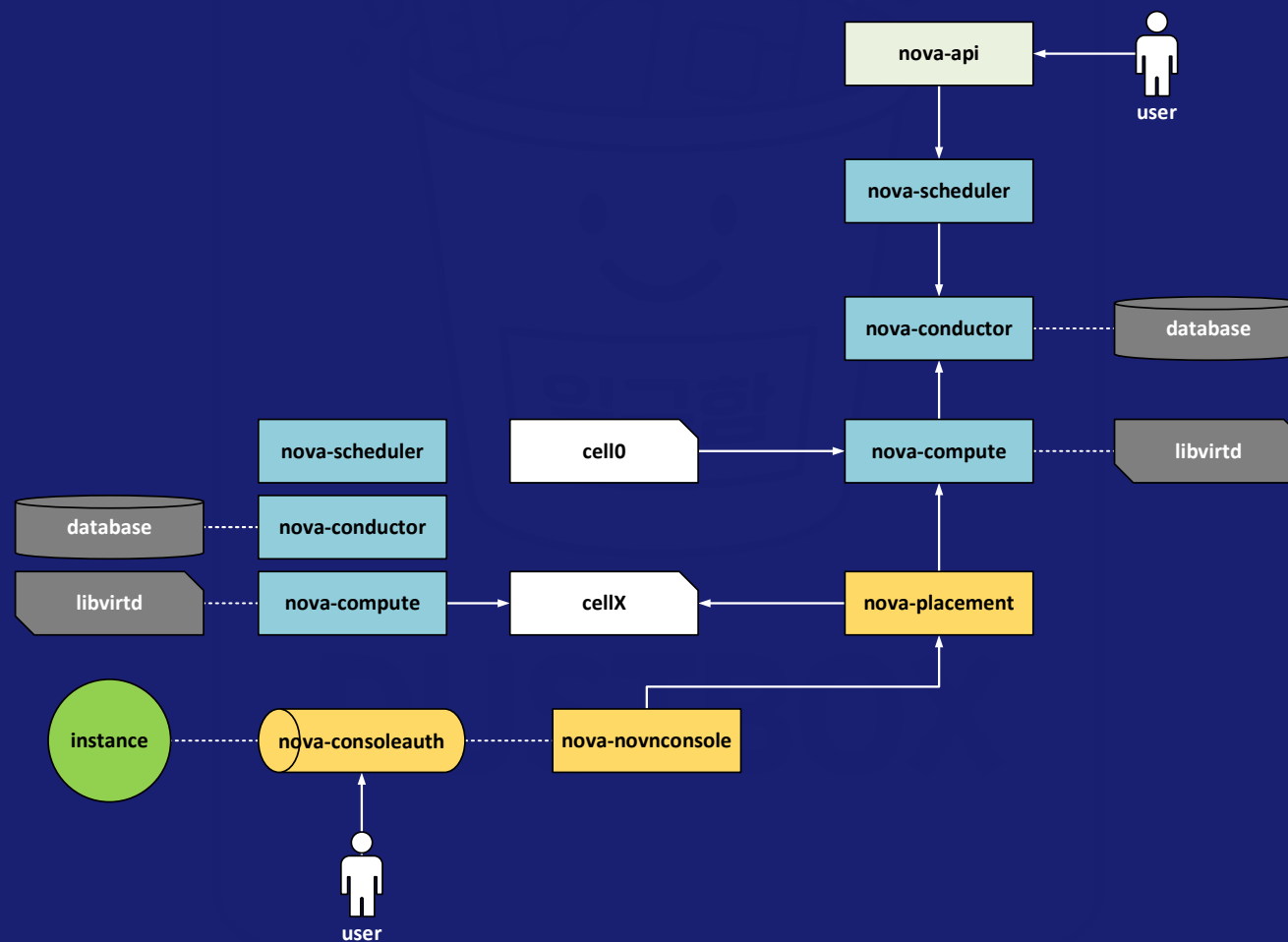
인스턴스 생성

인스턴스 생성을 위해서 네트워크 생성부터 시작한다. 최소 한 개의 내부 네트워크가 필요하다.

```
# openstack server create vm-demo \  
--flavor m1.small \  
--image RockyLinux-9 \  
--nic net-id=$(openstack network show net-demo -f value -c id) \  
--security-group lab \  
--key-name mykey  
# openstack server list
```



정리



연습문제

Swift

기본 기능

Swift 소개

Swift는 OpenStack의 오브젝트 스토리지 컴포넌트로, 대규모 정적 파일 저장에 적합하며, AWS S3와 유사한 기능을 제공한다.

Swift는 Ring 구조를 기반으로 데이터를 분산 저장하며, 각 오브젝트는 해시 함수를 통해 파티션과 저장 장치에 매핑 된다. 오브젝트 데이터는 파일 형태로 저장되며, 리눅스의 VFS와 로컬 파일시스템을 그대로 활용한다.

Swift의 주요 강점은 구조의 단순성과 가벼움이며, 특히 CDN이나 정적 콘텐츠 제공에 적합하다. 반면, 고성능 분산 블록 스토리지나 고가용성 환경에서는 Ceph이 더 많이 활용된다.

Swift는 기본적으로 자체 API를 사용하며, S3 API와는 직접적인 호환성을 제공하지 않지만, 일부 미들웨어를 통해 제한적 호환이 가능하다.



Swift 역사

1. Swift는 2010년 Rackspace와 NASA가 함께 시작한 OpenStack 프로젝트의 초기 구성요소 중 하나이다.
2. OpenStack의 Compute(Nova), Image(Glance)와 함께 오브젝트 스토리지를 담당하는 핵심 컴포넌트로 출발했다.
3. 초기에는 GlusterFS와 함께 오브젝트 스토리지 대안으로 고려되었으나, Swift는 보다 클라우드 네이티브한 구조로 설계되었다.
4. 2010~2014년 사이에는 주요 기업(HP, Rackspace 등)이 Swift 클러스터를 상용 운영했지만, 이후 Ceph의 부상으로 다소 영향력을 잃었다.
5. 최종적으로 kolla-ansible에서는 2025.1 릴리즈 부터는 더 이상 공식 릴리즈에 포함이 되지 않는다.



하지만, Swift...

현재 Swift는 점점 위기가 오고 있다. 자세한 이유는 아래 표를 참고한다.

항목	Swift	Ceph (RGW 기반)	MinIO
구조	전용 오브젝트 스토리지	유니파이드 스토리지 (블록/파일/오브젝트)	경량 고성능 오브젝트 스토리지
주요 구성요소	Proxy Server, Account/Container/Object DB	MON, OSD, RGW	단일 바이너리로 구성
데이터 중복	N-way 복제 기반	EC(Erasure Coding) 및 복제 지원	복제 및 EC 가능
확장성	수평 확장 가능하나 설계 복잡도 존재	뛰어난 수평 확장성	매우 쉬운 확장



하지만, Swift...

현재 Swift는 점점 위기가 오고 있다. 자세한 이유는 아래 표를 참고한다.

항목	Swift	Ceph (RGW 기반)	MinIO
커뮤니티 지원	점점 약화되고 있음	매우 활발	활발, 특히 Kubernetes 친화적
S3 호환성	기본 미지원 (Swift API 중심)	완전 지원 (RGW)	완전 지원
설치/운영 난이도	높음	높음 (구성 요소 다양)	낮음 (간단한 바이너리 실행)
사용 사례	더 이상 지원하지 않음	대규모 스토리지 통합 시스템	개발환경, 클라우드 네이티브 앱 백엔드



왜 점점 사용하지 않을까?

S3 호환성이 부족하다.

현재 대부분의 클라우드 애플리케이션이 AWS S3 API에 맞춰져 있으나, Swift는 고유 API를 사용한다.

운영이 복잡하다.

Swift는 "ring"이라는 구조로 구성되며, 장애 조치/데이터 복제/정합성 유지 등을 위해 많은 컴포넌트와 노력이 필요하다.



왜 점점 사용하지 않을까?

Ceph의 유니파이드 스토리지 모델이 더 매력적이다.

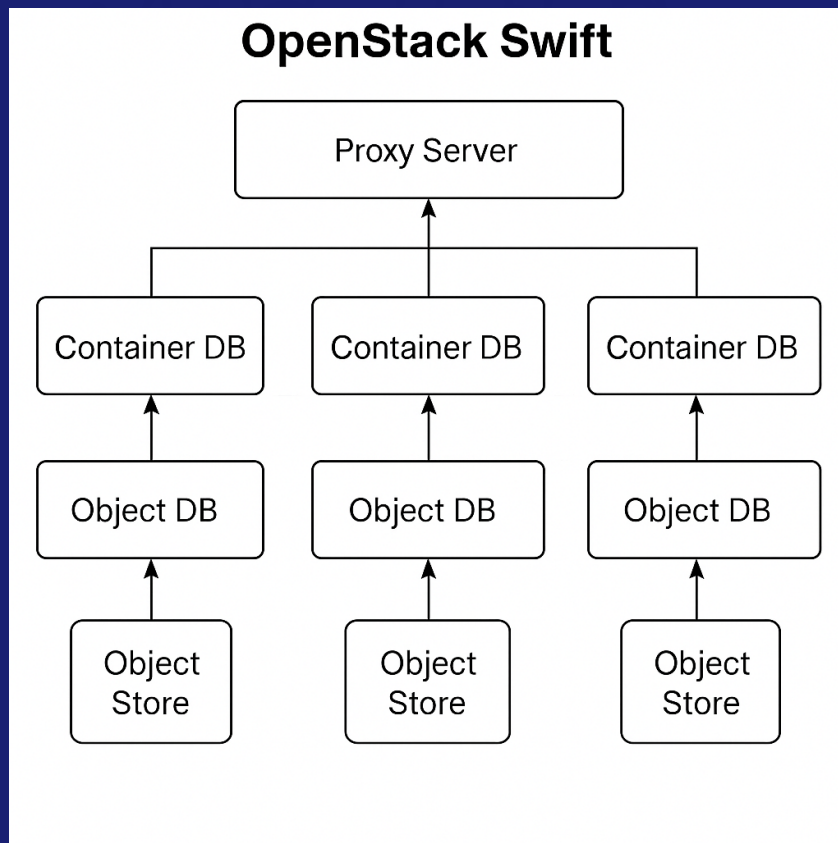
하나의 백엔드로 블록(Cinder), 파일(Manila), 오브젝트(RGW)를 지원할 수 있는 Ceph이 데이터센터 통합에 더 유리하다.

개발 및 커뮤니티가 활발하지 않다.

Swift의 개발 속도가 느리고, 주요 오픈소스 스토리지 트렌드에서 밀려나 있다. 현재는 개발이 중지가 되었다.



Swift 아키텍처



Ceilometer

기본 기능

개요

OpenStack에서 미터링은 인프라 자원의 **사용량, 이벤트, 경보**를 수집하고 분석하기 위한 핵심 기능이다. 이 기능은 운영자에게 **청구, 모니터링, 자동 대응**을 가능하게 하며, 주로 다음 네 가지 서비스가 역할을 분담하여 구성된다.

구성요소	역할 서술	사용 목적
Ceilometer	OpenStack 자원에서 수집된 측정 데이터(meter) 를 통합 수집하는 기능을 담당한다. 다양한 에이전트(nova, neutron 등)를 통해 데이터를 받아드린다.	자원 사용량 측정 및 수집
Gnocchi	Ceilometer가 수집한 원시 메트릭 데이터를 저장하고, 집계 및 검색 기능을 제공한다. 시계열 데이터베이스(TSDB)로 동작하며 Ceilometer와 연동된다.	시계열 기반 데이터 저장 및 분석



개요

위의 내용 계속 이어서...

구성요소	역할 서술	사용 목적
Panko	OpenStack의 이벤트(event) 데이터를 저장하고 조회할 수 있는 이벤트 레코더 역할을 수행한다. 로그성 정보 저장에 가까우며, 이벤트 추적용이다.	이벤트 기반 감사 및 모니터링
Aodh	Ceilometer/Gnocchi에서 수집된 데이터를 바탕으로 경보(alarm) 을 생성하고, 조건 충족 시 알림을 제공한다. 자동화 액션(ex. Heat과 연동)도 가능하다.	경보 기반 자동화/알림



Ceilometer 소개

Ceilometer는 OpenStack의 **모니터링 및 계량(metering)** 서비스를 위한 컴포넌트로, 사용자의 클라우드 자원 사용량을 측정하고 데이터를 수집하여 **빌링(billing)** 또는 모니터링 목적으로 활용할 수 있게 해준다.

이 서비스는 OpenStack의 Telemetry 프로젝트 중 하나로, 처음에는 리소스 사용량을 수집하고, 이를 통해 과금 체계를 구현하기 위한 목적에서 시작되었다.

Ceilometer는 가상머신 인스턴스, 네트워크 사용량, 블록 스토리지 등의 다양한 OpenStack 자원에 대한 정보를 수집하고 이를 중앙화 된 데이터베이스에 저장한다.

수집된 데이터는 대시보드나 외부 툴을 통해 시각화 하거나, 알람 조건으로 활용할 수 있다.



Ceilometer 소개

Ceilometer 자체는 수집만을 담당하며, 데이터 저장 및 시각화는 외부 도구에 위임한다.

대표적으로 **Gnocchi**가 시계열 데이터베이스로 사용되며, **Grafana**를 통해 시각화 할 수 있다. 필요에 따라 **collectd, fluentd**와 연동하거나, **Prometheus**등 외부 모니터링 도구로 대체되기도 한다.

이러한 구성은 사용자의 리소스 사용량 모니터링, 과금 시스템, 성능 분석 등 다양한 목적에 활용된다.



Ceilometer의 간단한 기능 분리 역사

Ceilometer는 2012년 OpenStack Folsom 릴리스에서 처음 도입되었다. 당시에는 클라우드 인프라의 리소스 사용량을 계량할 수 있는 기능이 필요하다는 수요가 많았고, Ceilometer는 이 요구를 충족시키는 핵심 프로젝트로 부상했다.

이후 OpenStack은 Telemetry 프로젝트를 확장하면서 **Aodh(알람 서비스), Panko(이벤트 저장소), Gnocchi(시간 기반 시계열 데이터 저장소)** 등의 하위 프로젝트로 Ceilometer의 기능을 분산시켰다.

Ceilometer는 주로 데이터를 수집하고, Gnocchi는 저장, Aodh는 경고 트리거, Panko는 이벤트 로깅이라는 역할 분담 구조를 갖게 된다.



Ceilometer 사용이 감소한 이유

하지만 시간이 지나면서 **Ceilometer**는 점점 덜 사용되기 시작했다. 주요 이유는 다음과 같다:

1. 복잡한 구조와 성능 문제

Ceilometer는 다양한 에이전트를 설치해야 하며, 데이터 수집 주기를 조정하거나 Gnocchi 같은 별도의 백엔드를 설정하는 등 구성이 매우 복잡하다. 또한 수집되는 데이터 양이 많아질 경우, **데이터베이스의 부하가 급격히 증가**하고, 성능 저하로 이어지는 사례가 많았다.

2. 대안의 등장

Prometheus, Grafana, Telegraf 같은 **경량화된 모니터링 툴들이 등장**하면서, 사용자들은 Ceilometer보다 더 간단하고 효율적인 도구로 넘어가기 시작했다. 특히 Kubernetes의 도입 이후, 클라우드 인프라 전체를 Prometheus 기반으로 통합 모니터링하는 사례가 늘었다.



Ceilometer 사용이 감소한 이유

3. Gnocchi의 한계 및 유지보수 문제

Ceilometer의 시계열 데이터 백 엔드로 많이 사용된 Gnocchi는 한때 OpenStack 공식 프로젝트였지만, 점차 커뮤니티 유지보수가 약화되면서 불안정해졌다. 이에 따라 Ceilometer 전체의 신뢰도도 떨어졌다.

4. 복잡한 알람 설정과 낮은 직관

Aodh와의 연동을 통해 알람 설정이 가능하지만, 설정 방식이 어렵고 사용자 친화적이지 않다는 평가가 많았다.



Ceilometer 대안?

Ceilometer는 여전히 사용하고 있으나, 아키텍처 및 구성원이 너무 점점 많아지면서 운영에 많은 어려움이 발생. 이러한 이유로, 현재는 Prometheus 기반으로 통합 및 운영을 하고 있다.

대체 도구	목적	상태	비고
Prometheus	모니터링/알람/메트릭	매우 활발	현재 가장 많이 사용됨
Gnocchi	시계열 저장소	유지보수 중단	Ceilometer 백엔드였음
Monasca	통합 텔레메트리 시스템	거의 중단	복잡하고 무겁지만 과거 SUSE 등 사용
Telegraf 등	경량 수집기	활발	Prometheus와 함께 사용
Zabbix/Nagios	시스템 모니터링	유지보수 활발	알람 및 상태 모니터링 용도 중심



Ceilometer 역할 및 관계

현재 모니터링 및 미터링 도구는 다음과 같다.

컴포넌트	역할	의존 대상	주요 기능
Ceilometer	메트릭 수집기	OpenStack 서비스, Gnocchi	리소스 사용량 수집, 알람 트리거 조건 전달
Gnocchi	시계열 데이터 저장소	Ceilometer	메트릭 저장 및 요약, API 제공
Aodh	알람 서비스	Ceilometer, Gnocchi	메트릭 기반 조건 만족 시 알람 발생 (예: CPU > 90%)



Ceilometer 역할 및 관계

위의 내용 계속 이어서...

구성 요소	역할	주요 연결 대상
Ceilometer	메트릭 수집 (agent 기반)	Gnocchi
Gnocchi	시계열 메트릭 저장소	Grafana, Aodh
Aodh	알람 조건 평가 및 트리거	Gnocchi, Nova
Exporter	메트릭 노출 (HTTP endpoint)	Prometheus
Prometheus	주기적 메트릭 수집 및 저장	Exporter, Grafana
Grafana	시각화 도구	Gnocchi, Prometheus



Ceilometer – 메트릭 수집기

Ceilometer는 다음처럼 자원 조회가 가능하다.

```
# openstack metric resource list
# openstack metric measures show <resource-id> --metric <metric-name>
# openstack metric list
# openstack metric resource list --type instance
```



메트릭 조회 결과

특정 메트릭 자원을 확인 시, 아래와 같이 출력이 된다.

```
$ openstack metric resource list --type instance
```

id	type	started_at	user_id	project_id
1b2c3d4e- ...	instance	2025-05 ...	user-a	project-a



메트릭 조회 결과

메트릭 자원 목록은 아래처럼 조회가 가능하다.

```
$ openstack metric list --resource-id 1b2c3d4e-...
```

+	+	+
id	name	
+	+	+
aabbccdd- ...	cpu_util	
+	+	+



메트릭 조회 결과

현재 수집된 정보를 아래처럼 수치 확인이 가능하다.

```
$ openstack metric measures show aabbccdd-...
```

timestamp	granularity	value
2025-06-02T12:00:00+00:00	300.0	32.5



Gnocchi

Gnocchi를 사용하게 된 이유는, 기존에 사용하던 MongoDB를 더 이상 사용하지 않고, Celiometer는 데이터를 전달하는 Publisher만 하고, 실제 데이터 저장 및 조회는 Gnocchi에서 한다.

Gnocchi는 이전 Ceilometer와 동일하게 MongoDB 기반으로 시계열 데이터를 저장한다.



Gnocchi - 시계열 메트릭 저장소

Gnocchi 명령어를 사용하기 위해서 다음과 같이 PIP 패키지가 설치 되어야 한다. 인스턴스 정보를 확인한다.

```
# pip3 install python3-gnocchiclient
```

```
# openstack metric resource list --type instance
```

id	type	started_at
1b2c3d4e-5f6g-7h8i-9j0k-abcde1234567	instance	2025-06-01T01:00:00



Gnocchi - 특정 리소스의 메트릭 목록 확인

아래 명령어로 특정 자원, 여기서는 인스턴스에서 사용하는 자원 상태를 확인한다.

```
# openstack metric list --resource-id 1b2c3d4e-5f6g-7h8i-9j0k-abcde1234567
```

id	name	created_by_proje
f0e1d2c3-b4a5-6789-cdef-12345678....	cpu_util	project-x
a1b2c3d4-e5f6-7890-ghij-98765432	memory.usage	project-x



Gnocchi - 특정 리소스의 메트릭 목록 확인

아래 명령어로 특정 자원, 여기서는 인스턴스에서 사용하는 자원 상태를 확인한다.

- 기본적으로 5분 간격(300초)으로 집계된 시계열 데이터.
- granularity는 메트릭을 수집한 시간 간격 (Gnocchi 설정에 따라 다름)

```
# openstack metric measures show f0e1d2c3-b4a5-6789-cdef-123456789abc
```

timestamp	granularity	value
2025-06-02T09:00:00+00:00	300.0	28.53
2025-06-02T09:05:00+00:00	300.0	31.10



Gnocchi – 자원 조회 방법

조회를 하기 위해서 보통 다음과 같은 순서로 조회 및 접근한다.

단계	목적	명령어
1단계	인스턴스 자원 조회	<code>openstack metric resource list --type instance</code>
2단계	해당 자원의 메트릭 ID	<code>openstack metric list --resource-id <resource-id></code>
3단계	메트릭 값 확인 (시계열)	<code>openstack metric measures show <metric-id></code>

메트릭 이름	설명
cpu_util	CPU 사용률 (%)
memory.usage	메모리 사용량 (bytes)
disk.read.bytes	디스크 읽기 (bytes)
network.incoming.bytes	수신 네트워크 (bytes)



Panko – 이벤트 레코더

오픈스택에서 발생한 이벤트를 조회하기 위해서 Panko를 통해서 조회가 가능하다. Panko는 openstack명령어에서 event를 통해서 조회가 가능하다.

```
# openstack event list
# openstack event show <event-id>
[
  {
    "event_type": "compute.instance.create.end",
    "generated": "2025-06-02T12:00:00",
    "traits": {
      "instance_id": "abc123",
      "user_id": "user1",
      "project_id": "proj1"
    }
  }
]
```



Aodh - 알람 및 자동화 트리거

Aodh는 이전에 Ceilometer로 구성하였던 부분을 분리하여, 별도의 알림 시스템을 구성하였다. 사용 및 구성 방법은 조금 복잡하지만, 기본적으로 다음과 같은 조건으로 동작한다.

1. 제한 수치
2. 자원 유형
3. 자원 이름

이 기능은 웹 hook 연동이 가능하기 때문에, 사용자가 원하는 경우 언제든지 웹 hook으로 알림 전달이 가능하다. 다만, 상대적으로 사용하기가 어렵기 때문에, 익숙하지 않는 사용자는 사용하기가 어려운 편이다.



Aodh - 알람 및 자동화 트리거 생성

아래처럼 알람 구성이 가능하다.

```
# openstack alarm create \  
  --name cpu_high \  
  --type gnocchi_resources_threshold \  
  --metric cpu_util \  
  --threshold 80 \  
  --resource-id <id> \  
  --resource-type instance \  
  --comparison-operator gt \  
  --aggregation-method mean \  
  --evaluation-periods 1 \  
  --granularity 300 \  
  --alarm-action 'http://alert.local/webhook'
```



Aodh – 알람 및 자동화 트리거 생성

구성이 완료가 되면, 아래 명령어로 조회가 가능하다.

```
# openstack alarm list
[
  {
    "name": "cpu_high",
    "state": "ok",
    "type": "gnocchi_resources_threshold",
    "threshold": 80,
    "aggregation_method": "mean"
  }
]
```



정리

아키텍처를 정리하면 다음과 같다.

[Ceilometer Agent]



측정 수집 API



[Gnocchi] → 시계열 DB



[Aodh] → 알람 생성 / 액션 실행

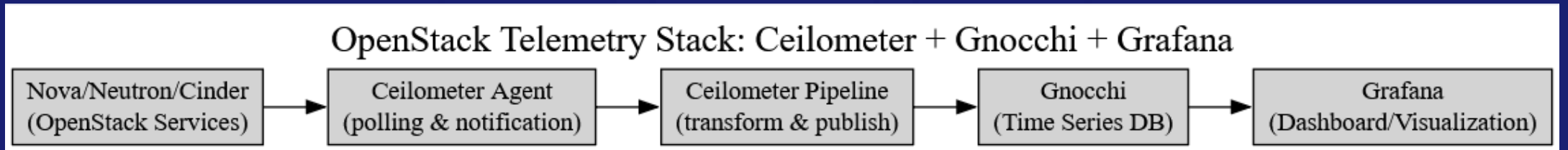


[Webhook / Heat]

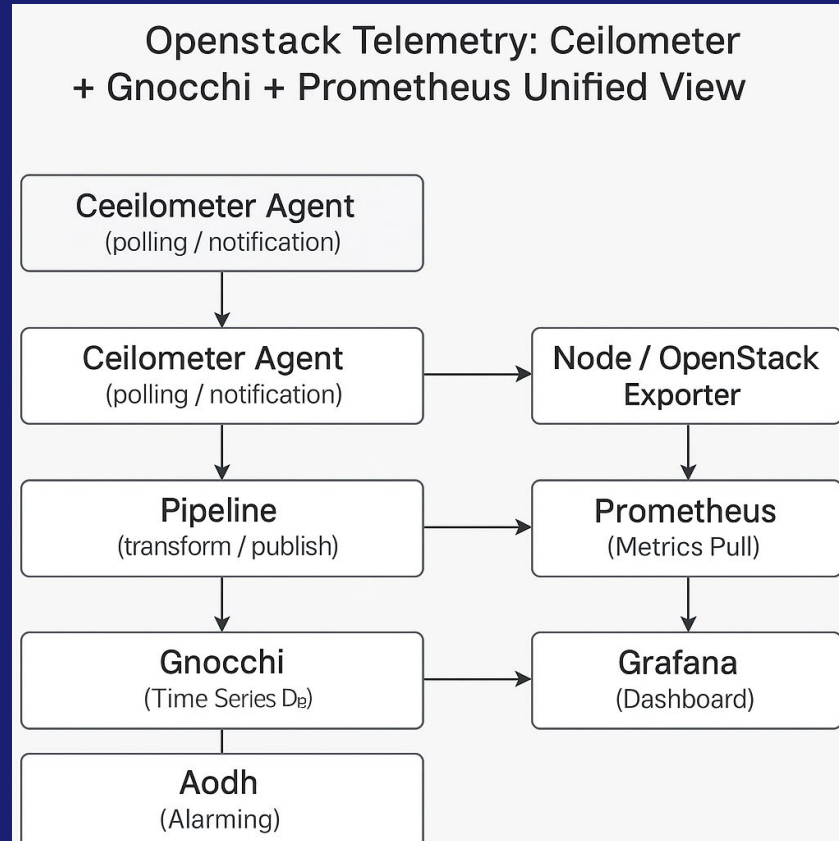
[Panko] ← [Ceilometer Event API]
(이벤트 수집 저장)



Ceilometer 통합 구조



Ceilometer 통합 구조



연습문제

Heat

기본 기능

OpenStack Heat란 무엇인가?

Heat는 OpenStack에서 제공하는 **오케스트레이션(Orchestration)** 서비스로, 클라우드 자원의 배치 및 설정을 자동화하기 위한 도구이다.

사용자는 YAML 형식의 템플릿 파일을 통해 가상머신, 네트워크, 볼륨, 보안 그룹, 키페어 등 다양한 OpenStack 자원을 한 번에 정의하고 배포할 수 있다.

Heat의 목적은 AWS의 **CloudFormation**과 유사하게, **인프라스트럭처 코드관리**(Infrastructure as Code, IaC) 하여, 복잡한 인프라 구성을 재현 가능하고 일관되게 만들 수 있도록 돕는 것이다.

이 방식은 특히 **교육/테스트/개발 환경**을 자동화하고, 반복적 작업을 줄이는 데 유용하다.



Heat의 역사

Heat는 OpenStack의 초기 릴리스인 Grizzly (2013년) 에 포함되었으며, 오케스트레이션이라는 개념을 OpenStack에 도입한 선구적 도구였다.

당시에는 인프라를 구성하기 위해 CLI 또는 Horizon 대시보드를 통해 하나씩 리소스를 만드는 수작업이 많았고, 이를 자동화하는 도구가 절실했다.

Heat는 이후 HOT(Heat Orchestration Template)라는 자체 템플릿 문법을 발전시켜 YAML을 기반으로 한 간결한 구성을 가능하게 했고, 외부 연동(Resource Plugin) 기능도 제공하며 다양한 시나리오에 활용되었다.



OpenStack Heat란 무엇인가?

Heat는 크게 다음과 같은 요소로 구성된다.

- Heat Engine: 템플릿을 해석하고 스택(stack)을 실제 자원으로 변환
- Heat API: 사용자 요청을 받아 엔진에 전달
- Heat Template (HOT): YAML로 작성되는 템플릿 파일
- Stack: 템플릿을 통해 생성된 실제 리소스 집합



HEAT 개요

Heat에서 제공하는 자원은 다음과 같다.

기능	설명
스택(Stack) 관리	리소스 집합을 스택 단위로 생성, 업데이트, 삭제
템플릿 기반 구성	HOT 또는 AWS CloudFormation 호환 JSON 사용 가능
리소스 추적	생성된 리소스 상태를 모니터링하고 롤백 기능 지원
파라미터화	템플릿에 사용자 정의 변수와 조건 사용 가능
리소스 간 의존성	depends_on 또는 참조 관계를 통해 생성 순서 자동 관리



laC 도구 비교

오픈스택에서 사용이 가능한 laC도구에 대해서 기능 비교이다.

항목	Heat	Ansible	SaltStack	Terraform
소속	OpenStack 전용	범용 자동화	범용 자동화	HashiCorp
목적	인프라 선언 및 배포	구성 관리 중심 + 일부 배포	구성 및 원격 실행	클라우드 인프라 선언적 배포
언어	YAML (HOT)	YAML (Playbook)	YAML (SLS)	HCL
작동 방식	스택 기반, OpenStack API 호출	SSH 또는 에이전트 기반 원격 실행	마스터-미니언 또는 에이전트리스	API 기반 상태관리
상태관리	자체적으로 스택 상태 추적	비상태 (Idempotent 아님)	비상태 또는 상태 저장 가능	상태 파일(state) 기반



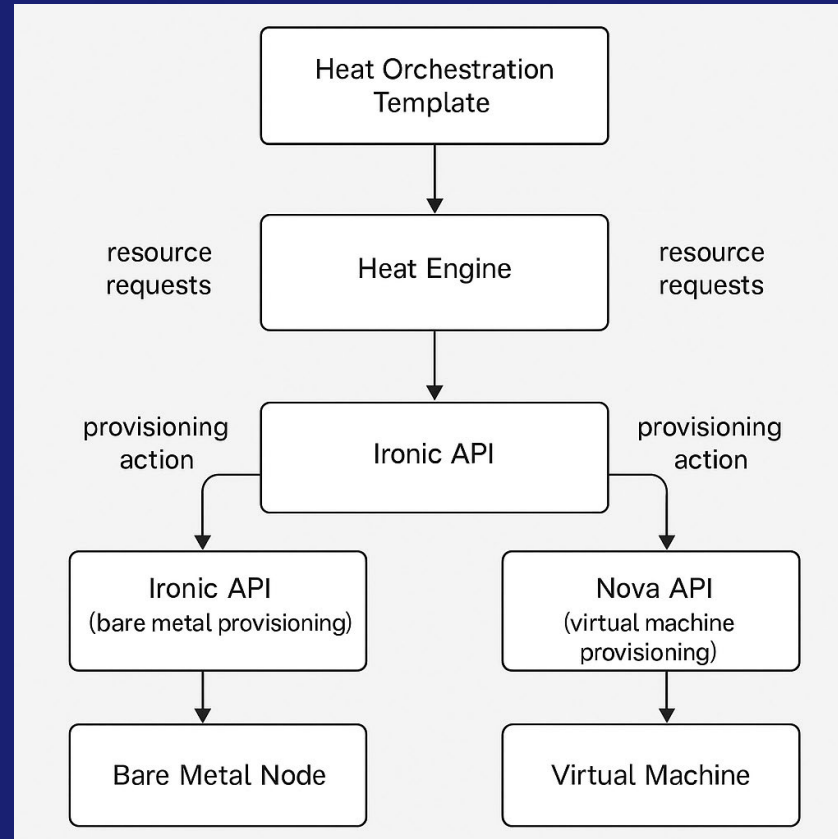
laC 도구 비교

오픈스택에서 사용이 가능한 laC도구에 대해서 기능 비교이다.

항목	Heat	Ansible	SaltStack	Terraform
의존성 처리	리소스 간 depends_on 및 참조 자동 추론	순서 명시 필요	순서 제어 수동	리소스 간 그래프 추론
장점	OpenStack 최적화, 스택 전체 추적 가능	다양한 시스템 구성에 유리	대규모 시스템에 빠른 명령 전파	멀티 클라우드 지원, 선언적 laC
단점	타 플랫폼 미지원, 템플릿 러닝 커브	복잡한 의존성 자동화 어려움	YAML 작성이 복잡	상태 파일 충돌 가능



TRIPLE-O HEAT 아키텍처



Heat 소개

Heat는 OpenStack의 오케스트레이션 도구로, YAML 기반의 HOT 템플릿을 사용하여 서버, 네트워크, 볼륨 등 자원의 자동 구성을 지원한다.

`OS::Nova::Server`, `OS::Cinder::Volume` 등과 함께 `OS::Ironic::Node` 같은 베어메탈 자원도 정의할 수 있으며,

Ironic API와 연동되어 템플릿 기반으로 베어메탈 인프라를 프로비저닝 할 수 있다.

이처럼 Heat는 OpenStack 클러스터 내부 자원 자동화에 특화되어 있으며, 다양한 리소스를 통합 관리하는 경우에는 Ansible과의 병행 사용이 추천된다.



Heat가 점점 사용되지 않는 이유

하지만 시간이 흐르면서 Heat의 사용은 점차 줄어들게 되었는데, 그 배경에는 여러 가지 현실적인 이유가 있다.

1. 기능 제한과 복잡한 템플릿 구조

- Heat는 OpenStack 자원에만 초점을 맞추고 있어, 외부 시스템 (예: DNS 설정, CI/CD, 인증 서버 등)과의 연동이 어렵다.
- 템플릿이 복잡해질수록 의존성과 순서 문제, 디버깅 어려움이 생기며, 관리가 힘들어진다.
- Jinja와 같은 템플릿은 사용하기가 어렵다.



대체 도구의 발전

현재, IaC 도구는 전체적으로 범용성 및 멀티 클라우드 시스템 위주로 되어가고 있다. 심지어, AWS, GCP에서 사용하는 IaC도구도 요즘은 점점 사용율이 낮아지는 추세이다.

- Terraform, Ansible, Pulumi, Crossplane 등 더 범용적이고 클라우드 독립적인 IaC 도구들이 Heat를 대체하게 되었다.
- 특히 Terraform은 OpenStack뿐 아니라 AWS, Azure, GCP, Kubernetes 등 멀티 클라우드 리소스를 통합 관리할 수 있어, 기업 환경에서 Heat보다 훨씬 선호된다.



Horizon UI 통합 기능 한계

Heat는 오픈스택에 통합이 되어 있지만, 어중간한 형태로 기능 제공 및 대시보드 기반에서는 거의 확인 용도만 가능하며, 관리적인 부분은 거의 지원하지 않는다.

상세한 기능은 CLI를 사용해야 되기 때문에, CLI 사용이 불가능한 사용자는 애초부터 접근이 불가능하다.

- Horizon 대시보드에서 Heat Stack을 생성하거나 관리할 수 있지만, UI에서 가능한 기능은 매우 제한적이며, 템플릿 오류 시 상세한 로그 확인이 어렵다.
- 사용자 친화적인 경험이 떨어지고, 결과적으로 CLI를 병행해야 하는 불편함이 있다.



커뮤니티 활력 저하

Heat는 여전히 OpenStack 공식 프로젝트지만, 최근 몇 년간 주요 기능의 변화나 활발한 릴리스는 줄어들었다. 대신 다른 IaC 도구들이 활발히 발전하며 커뮤니티 중심이 옮겨갔다.

다음과 같은 도구로 커뮤니티는 전환을 시작 하였다.

도구 이름	언어/표현 방식	OpenStack	MCP	특징	활동성	대체 성공사례
Terraform	HCL	공식	지원	멀티 클라우드	매우 활발	OVH, Red Hat
OpenTofu	HCL	지원	지원	오픈소스	활발	리눅스재단 후원
Ansible	YAML	지원	일부 지원	멀티 클라우드	매우 활발	CERN, OPNFV
SaltStack	YAML	지원	일부 지원	이벤트 자동화	중간 이상	SUSE OpenStack
Pulumi	Python, Go 등	지원	지원	프로그래밍 기반	활발	Snowflake 등
Crossplane	K8S CRD	제한적 지원	지원	K8s 기반	활발	Upbound 활용



Heat 대체 도구와 그 근거

1. Terraform / Opentofu

- OpenStack 공식 provider 문서: registry.terraform.io/providers/terraform-provider-openstack
- GitHub Stars (OpenTofu 기준): 10K 이상
- 많은 기업이 Heat 대신 사용, 멀티클라우드/CI-CD에 최적화

2. Ansible

- openstack.cloud 모듈 공식 문서: docs.ansible.com/collections/openstack/cloud/
- 단순 자동화에서부터 Playbook 기반 대규모 프로비저닝까지 대응



Heat 대체 도구와 그 근거

3. Pulumi

- 코드 기반 인프라를 선호하는 개발자 대상, GitOps와 잘 통합됨

4. Crossplane

- GitOps와 Kubernetes를 동시에 사용하는 환경에서 매우 빠르게 채택 중
- CNCF 소속, 활발한 PR 및 커밋 진행



HEAT 구조

Heat파일 구조는 YAML과 동일하다. 다만, 여러가지 버전이 있기 때문에 상단에 추가적으로 Heat버전에 대해서 선언이 필요하다.

```
heat_template_version: 2018-08-31    # 템플릿 버전 명시
```

```
description: 간단한 인스턴스 생성 예제
```

```
parameters:
```

```
  image:
```

```
    type: string
```

```
    default: cirros
```

```
  flavor:
```

```
    type: string
```

```
    default: m1.small
```



HEAT 구조

앞에 내용 계속 이어서...

```
network:
  type: string
  default: internal
resources:
  my_instance:
    type: OS::Nova::Server
    properties:
      name: test-vm
      image: { get_param: image }
      flavor: { get_param: flavor }
      networks:
        - network: { get_param: network }
```



HEAT 구조

앞에 내용 계속 이어서...

outputs:

instance_name:

value: { get_attr: [my_instance, name] }



HEAT 적용

Heat 생성 및 구성이 완료가 되면 오픈스택에 적용한다.

```
# openstack stack create -t simple-vm.yaml --parameter network=private-net  
demo-stack  
# openstack stack create -t simple-vm.yaml --stack-name demo-stack  
# openstack stack list  
# openstack stack show demo-stack  
# openstack server list
```



HEAT 적용

명령어 결과 화면 하나씩 구성하면 다음과 같이 나온다.

```
$ openstack stack create -t simple-vm.yaml --stack-name demo-stack
```

Field	Value
id	01a3f83e-e1ad-4a88-93c3-9086c4cc80a3
stack_name	demo-stack
stack_status	CREATE_IN_PROGRESS
creation_time	2025-06-02T12:00:00



HEAT 적용

명령어 결과 화면 하나씩 구성하면 다음과 같이 나온다.

```
$ openstack stack list
```

ID	Stack Name	Stack Status	Creation Time	Updated Time
01a ...	demo-stack	CREATE_COMPLETE	2025-06-02T12:00	None



HEAT 적용

명령어 결과 화면 하나씩 구성하면 다음과 같이 나온다.

```
$ openstack server list
```

ID	Name	Status	Networks	Image	Flavor
a9e123	test-vm	ACTIVE	pr=10.0	cirros	m1.small



연습문제

대표적인 확장 컴포넌트

Octavia

Designated

개요

오픈스택에서 확장 컴포넌트를 사용하기 위해서 컨테이너 런타임은 가급적이면 containerd기반으로 구성을 권장한다. Podman기반으로 구성이 가능하지만, Podman은 rootless기반으로 구성이 되어 있기 때문에 몇몇 기능은 권한 부족으로 올바르게 동작하지 않는다.

많이 사용하는 확장 서비스는 보통 다음과 같다.

1. Octavia
2. Designated

이러한 이유로, 가볍게 코어 서비스만 사용하는 경우, Podman. 그게 아니면 Containerd기반으로 구성한다.



Octavia 설명

교육 마지막에 추가할 서비스는 앞에서 말한 두 가지 서비스를 확장할 예정이다.



Designated 설명

교육 마지막에 추가할 서비스는 앞에서 말한 두 가지 서비스를 확장할 예정이다.



자동화

laC(ansible)

설명

Ansible은 IT 인프라의 자동화, 구성 관리, 애플리케이션 배포, 오케스트레이션을 위한 오픈소스 도구로서, YAML 기반의 '플레이북(playbook)'을 통해 작업을 자동화하는 선언적 도구이다.

1. SSH를 기반으로 동작하며, 별도의 에이전트를 설치할 필요가 없다.
2. 관리 대상 노드에 명령어를 실행하거나 설정 파일을 배포하고, 서비스 상태를 제어하는 등 시스템 운영 작업을 자동화할 수 있다.
3. 코드처럼 인프라를 정의하는 방식, 즉 Infrastructure as Code(IaC) 개념을 따르며, 사람도 읽기 쉬운 형식으로 시스템을 구성할 수 있다.



Ansible과 OpenStack의 관계

Ansible은 OpenStack의 리소스를 프로그래밍 방식으로 관리할 수 있도록 여러 모듈을 제공한다. 대표적으로 다음과 같은 관계가 있다.

1. 오픈스택 클라우드 자원 생성

- 네트워크, 서브넷, 라우터, 보안 그룹, 키페어, 이미지, 볼륨, 인스턴스 등 모든 기본 자원을 Ansible로 정의하고 배포할 수 있다.
- `openstack.cloud.os_server`, `os_network`, `os_image` 등의 모듈을 활용하여 리소스를 조작한다.

2. 클라우드 배포 자동화

- `kolla-ansible`, `openstack-ansible`, `kayobe` 등 다양한 OpenStack 배포 도구가 Ansible 기반으로 작성되어 있다.
- 따라서 Ansible은 OpenStack을 설치하는 도구이자, 운영하는 도구이기도 하다.



Ansible과 OpenStack의 관계

3. CI/CD 및 DevOps 연계

- OpenStack과 연동하여 Jenkins, GitLab CI 등의 파이프라인에서 Ansible을 호출해 클라우드 환경에 자동으로 리소스를 배포하거나 회수하는 작업을 할 수 있다.

4. Heat의 보완 도구

- Heat는 OpenStack 내부에서만 작동하지만, Ansible은 외부 시스템(OS 레벨 설정, 패키지 설치, 파일 배포 등)까지 다룰 수 있기 때문에 Heat + Ansible을 함께 사용하는 하이브리드 자동화 전략도 가능하다.



앤서블 작성

디렉터리 구조는 다음과 같다.

```
openstack_vm_setup/  
├─ create_resources.yml      # 메인 플레이북  
├─ vars.yml                  # 변수 정의  
└─ requirements.txt          # collections 정의 (openstack.cloud)  
# ansible-galaxy collection install openstack.cloud
```



변수 파일

다음과 같이 변수 파일을 작성한다.

```
# vi vars.yaml
auth:
  auth_url: http://<keystone>:5000/v3
  username: admin
  password: secret
  project_name: admin
  domain_name: default
image_name: cirros
image_url: http://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-
disk.img
flavor_name: m1.tiny
network_name: demo-net
```



변수 파일

위의 내용 계속 이어서...

```
subnet_name: demo-subnet
subnet_cidr: 192.168.100.0/24
router_name: demo-router
security_group: allow-ping-ssh
volume_name: demo-volume
instance_name: demo-vm
```



메인 플레이북

다음과 같이 변수 파일을 작성한다.

```
# vi create_resources.yaml
- name: OpenStack 기본 자원 생성 및 VM 배포
  hosts: localhost
  gather_facts: no
  vars_files:
    - vars.yaml
  collections:
    - openstack.cloud
```



메인 플레이북

앞에 내용 계속 이어서...

```
tasks:
```

```
- name: 네트워크 생성
```

```
  os_network:
```

```
    cloud: undercloud
```

```
    state: present
```

```
    name: "{{ network_name }}"
```

```
    external: no
```

```
    validate_certs: no
```

```
    auth: "{{ auth }}"
```



메인 플레이북

앞에 내용 계속 이어서...

```
- name: 서브넷 생성
  os_subnet:
    cloud: undercloud
    state: present
    name: "{{ subnet_name }}"
    network_name: "{{ network_name }}"
    cidr: "{{ subnet_cidr }}"
    ip_version: 4
    dns_nameservers: [8.8.8.8]
    auth: "{{ auth }}"
```



메인 플레이북

앞에 내용 계속 이어서...

```
- name: 라우터 생성
  os_router:
    cloud: undercloud
    state: present
    name: "{{ router_name }}"
    external_gateway: public # external 네트워크 이름
    interfaces:
      - "{{ subnet_name }}"
    auth: "{{ auth }}"
```



메인 플레이북

앞에 내용 계속 이어서...

```
- name: 보안 그룹 생성
  os_security_group:
    cloud: undercloud
    state: present
    name: "{{ security_group }}"
    description: "Allow ping and ssh"
    auth: "{{ auth }}"
```



메인 플레이북

앞에 내용 계속 이어서...

- name: 보안 그룹 룰 추가 - SSH

os_security_group_rule:

cloud: undercloud

security_group: "{{ security_group }}"

protocol: tcp

port_range_min: 22

port_range_max: 22

remote_ip_prefix: 0.0.0.0/0

state: present

auth: "{{ auth }}"



메인 플레이북

앞에 내용 계속 이어서...

```
- name: 보안 그룹 룰 추가 - Ping
  os_security_group_rule:
    cloud: undercloud
    security_group: "{{ security_group }}"
    protocol: icmp
    remote_ip_prefix: 0.0.0.0/0
    state: present
    auth: "{{ auth }}"
```



메인 플레이북

앞에 내용 계속 이어서...

```
- name: 이미지 등록
  os_image:
    cloud: undercloud
    name: "{{ image_name }}"
    container_format: bare
    disk_format: qcow2
    filename: "/tmp/{{ image_name }}.qcow2"
    state: present
    auth: "{{ auth }}"
  when: not lookup('file', '/tmp/' + image_name + '.qcow2',
errors='ignore')
```



메인 플레이북

앞에 내용 계속 이어서...

```
- name: 이미지 다운로드 (필요 시)
  get_url:
    url: "{{ image_url }}"
    dest: "/tmp/{{ image_name }}.qcow2"
  when: not lookup('file', '/tmp/' + image_name + '.qcow2',
errors='ignore')
```



메인 플레이북

앞에 내용 계속 이어서...

```
- name: 볼륨 생성
  os_volume:
    cloud: undercloud
    display_name: "{{ volume_name }}"
    size: 1
    state: present
    auth: "{{ auth }}"
```



메인 플레이북

앞에 내용 계속 이어서...

```
- name: 인스턴스 생성
  os_server:
    cloud: undercloud
    state: present
    name: "{{ instance_name }}"
    image: "{{ image_name }}"
    flavor: "{{ flavor_name }}"
    key_name: default # 사전에 키페어 생성되어 있어야 함
    network: "{{ network_name }}"
```



메인 플레이북 (create_resources.yml)

앞에 내용 계속 이어서...

```
- name: 인스턴스 생성
```

```
...
```

```
...
```

```
  security_groups:
    - "{{ security_group }}"
  boot_from_volume: yes
  volume_size: 1
  terminate_volume: yes
  auth: "{{ auth }}"
```



메인 플레이북 실행 및 적용

앞에 내용 계속 이어서...

```
# ansible-playbook -I localhost, create_resources.yaml
TASK [네트워크 생성] *****changed: [localhost]
TASK [서브넷 생성] *****changed: [localhost]
TASK [이미지 등록] *****ok: [localhost]
TASK [인스턴스 생성] *****changed: [localhost]
```



오픈스택 서비스 확장

CI/CD

Grafana/Prometheus

확장 서비스 설치 및 구성

CI/CD

ZUUL

개요

OpenStack과 Kubernetes는 각각의 목적과 구조가 다른 인프라 플랫폼이기 때문에, 그 위에서 운용되는 CI/CD 도구와 이미지 빌드 방식도 명확한 차이를 가진다.

OpenStack의 CI/CD는 인프라 자체의 구성 요소를 개발·테스트·배포하기 위한 목적이 중심이며, Kubernetes의 CI/CD는 해당 플랫폼 위에서 동작하는 애플리케이션의 개발 및 운영 자동화에 초점을 둔다.

OpenStack은 일반적으로 Zuul, Gerrit, Nodepool 등의 도구를 통해 CI/CD를 구성한다. 개발자가 Gerrit에 코드 변경을 등록하면 Zuul이 이를 감지하여 자동 테스트를 트리거하며, Nodepool을 통해 테스트 환경용 VM이 OpenStack 상에 동적으로 생성된다.

생성된 VM 위에는 DevStack 또는 Kolla-Ansible을 이용해 OpenStack의 각 컴포넌트가 설치되고, 이후 기능 테스트가 진행된다. 이 모든 과정은 코드 리뷰 기반의 파이프라인으로 설계되어 있으며, 결과는 다시 Gerrit에 피드백으로 기록된다.



개요

반면, Kubernetes에서는 보다 일반적인 CI/CD 워크플로우와 GitOps 접근 방식이 주로 사용된다.

Jenkins, GitLab CI, Tekton 같은 CI 도구는 코드 저장소(Git)에 커밋이 발생했을 때 컨테이너 이미지를 자동으로 빌드하고 Docker Registry 등에 저장한다.

이후 ArgoCD나 Flux와 같은 GitOps 도구가 Git 저장소의 상태를 감시하다가 변경 사항을 감지하면 자동으로 Kubernetes 클러스터에 해당 애플리케이션을 배포한다. 이러한 방식은 빠른 피드백과 반복적인 배포에 최적화되어 있으며, 애플리케이션 중심의 CD 환경을 제공한다.



개요

이미지 빌드 방식에서도 양자는 철학이 다르다. OpenStack은 kolla-build라는 도구를 통해 Ansible과 Jinja2 기반 Dockerfile 템플릿으로 운영자용 OpenStack 서비스 컨테이너 이미지를 빌드한다. 이 빌드는 서비스별로 다양한 설정이 내포되어 있으며, 대규모 인프라 구성에 적합하다.

반면 Kubernetes 기반 워크로드에서는 일반적인 docker build, buildah, kaniko 등을 통해 Dockerfile에서 애플리케이션 이미지를 간단히 빌드하고 경량화하는 것을 선호한다. 이는 마이크로서비스의 민첩한 배포와 업데이트를 위한 핵심 전략이다.



개요

결론적으로, OpenStack은 인프라 소프트웨어 개발과 테스트를 위한 복잡한 CI/CD 체계를 갖추고 있으며, VM 기반 환경과 코드 리뷰 기반 파이프라인을 중심으로 설계되어 있다. 반면 Kubernetes는 애플리케이션 개발을 위한 민첩한 배포 자동화 체계를 제공하며, 컨테이너 기반 CI/CD 및 GitOps 모델이 중심이다. 이로 인해 동일한 CI/CD라는 용어를 사용하더라도, 실제 구현 방식과 목적은 플랫폼의 철학에 따라 크게 달라진다.



ZUUL CI/CD

ZUUL기반으로 CI/CD를 구성하기 위해서 다음과 같은 흐름으로 작성 및 구성한다.

- [1] Git 커밋 (Gerrit or GitHub)
↓
- [2] Zuul이 감지 → 파이프라인 실행
↓
- [3] 이미지 빌드 (Packer 또는 Diskimage-builder)
↓
- [4] 빌드한 QCOW2 이미지 Glance에 등록
↓
- [5] Heat 스택으로 배포 테스트 (또는 Ansible)
↓
- [6] 결과 Gerrit에 피드백 (Success / Fail)



ZUUL COMPOSE

podman-compose를 사용해서 아래 내용을 구성한다.

```
# vi docker-compose.yaml
```



ZUUL 설치 및 구성

데비안 계열은 DEB로 제공하지만, 레드햇 계열 경우에는 RPM으로 제공하지 않는다. 사용하기 위해서 Podman이나 혹은 Docker 기반으로 사용을 권장한다.

```
# dnf install -y podman podman-compose
# git clone https://opendev.org/zuul/zuul.git
# cd zuul/tools/compose
# cp .env-sample .env
# podman-compose up -d
```



Grafana/Prometheus

모니터링

Prometheus

- Prometheus는 시계열(Time-series) 데이터베이스 시스템 및 애플리케이션의 메트릭 데이터를 주기적으로 수집(scrape) 하고, 이를 저장, 질의, 경고(Alerting)할 수 있도록 설계된 모니터링 수집기이다.
- Prometheus는 "pull 기반" 모니터링 시스템으로 설정된 endpoint(예: node-exporter, OpenStack Exporter 등)를 일정 주기로 조회하여 CPU, 메모리, 디스크, 네트워크 등의 실시간 성능 지표를 가져온다.
- 또한 자체 쿼리 언어인 PromQL을 사용하여 메트릭에 대해 복잡한 조건 검색과 경고 조건을 정의할 수 있다.



Grafana

- Grafana는 Prometheus가 수집한 시계열 데이터를 시각적으로 표현할 수 있는 대시보드 기반 시각화 도구이다.
- Grafana는 다양한 데이터 소스(Prometheus, Loki, InfluxDB 등)를 연동하여 사용자가 원하는 방식의 차트, 그래프, 알람 대시보드를 구성할 수 있도록 돕는다.
- 또한 Grafana는 경고(Alert Rule)를 시각적으로 정의하고, Slack, Email 등으로 경고 메시지를 전송할 수 있다.



Ceilometer

OpenStack의 Telemetry 서비스로, 클라우드 인프라 내 자원(예: 인스턴스, 볼륨, 네트워크 등)의 상태와 이벤트를 수집, 저장, 전달하는 역할을 수행한다. 과거에는 모니터링+미터링+알람을 모두 Ceilometer가 처리했지만, 이후 기능이 분리.

1. 메트릭 저장은 Gnocchi
2. 이벤트 저장은 Panko
3. 알람은 Aodh
4. 수집은 Ceilometer Agent



정리

Prometheus는 OpenStack과 독립적인 모니터링 스택이다. 구조적으로 다음과 같다.

1. Ceilometer 계열은 OpenStack 내부의 API, MQ(RabbitMQ), Agent를 통해 OpenStack 서비스와 밀접하게 통합됨.
2. Prometheus는 외부 exporter(node-exporter, openstack-exporter 등)를 통해 OpenStack 외부에서 수집함.

즉, OpenStack이 없어도 Prometheus는 동작 가능하고, OpenStack이 있어도 Prometheus는 OpenStack에 직접 의존하지 않음.



정리

Prometheus는 OpenStack과 독립적인 모니터링 스택이다.

1. Ceilometer 계열은 OpenStack 내부의 API, MQ(RabbitMQ), Agent를 통해 OpenStack 서비스와 밀접하게 통합됨.
2. Prometheus는 외부 exporter(node-exporter, openstack-exporter 등)를 통해 OpenStack 외부에서 수집함.

항목	Ceilometer 계열	Prometheus
연동 방식	OpenStack 서비스 내부 통합	외부 Exporter 연결
예시 수집 대상	Nova, Cinder, Neutron API	node-exporter, libvirt-exporter 등
수집 방식	AMQP(RabbitMQ) 기반 push	HTTP 기반 pull
저장소	Gnocchi	자체 TSDB
필수 구성 요소	ceilometer-agent, aodh, panko 등	Exporter만 있으면 가능



정리

Prometheus & Grafana 와 Ceilometer의 차이 및 관계.

항목	Ceilometer 계열	Prometheus/Grafana
수집 방식	push + agent 기반	pull 기반
수집 대상	OpenStack 리소스 중심	시스템 리소스 중심 (node-exporter, cadvisor 등)
저장소	Gnocchi (시계열 DB)	Prometheus 내장 TSDB
시각화	Horizon 통합 or 외부 도구	Grafana
알람	Aodh	Prometheus Alertmanager 또는 Grafana



모니터링 시각화

kolla-ansible를 사용하는 경우, 아래처럼 구성이 가능하다.

```
# vi gloabls.yaml
enable_prometheus: "yes"
enable_grafana: "yes"

# Optional: 성능 메트릭 수집을 위한 서비스들
enable_node_exporter: "yes"
enable_cadvisor: "yes"
enable_alertmanager: "yes"
grafana_admin_password: "MySecureGrafanaPass"
```

enable_cadvisor는 Docker 전용. Podman에서 사용 불가능.



시각화 적용

deploy를 통해서 기능추가 혹은 기존 설정 업데이트는 reconfigure를 통해서 가능하다.

```
# kolla-ansible -i /etc/kolla/inventory deploy
# kolla-ansible -i /etc/kolla/inventory reconfigure -t prometheus
# kolla-ansible -i /etc/kolla/inventory reconfigure -t grafana
# podman ps | grep prometheus
# podman ps | grep grafana
```

